Federal University of Minas Gerais
School of Engineering
Bachelor's Degree in Control and Automation Engineering

MicroTEDAClus Formulation, Implementation, and Experimental Validation Reassessment and Improvement

Eduardo Henrique Basilio de Carvalho

Advisor: Prof. Janier Arias Garcia, PhD

Belo Horizonte, November 2026

Undergraduate Thesis

MicroTEDAClus Formulation, Implementation, and Experimental Validation Reassessment and Improvement

Undergraduate Thesis submitted to the examination committee designated by the Academic Board of the Bachelor's Degree in Control and Automation Engineering at the Federal University of Minas Gerais, as part of the requirements for approval in the activity Projeto Final de Curso II.

Belo Horizonte, November 2026

Resumo

MicroTEDAClus é um algoritmo de agrupamento evolutivo para fluxos de dados que resume o fluxo em microgrupos por meio da excentricidade e da tipicidade do arcabouço TEDA, e cujos autores relatam ter desempenho equivalente ao de algoritmos consolidados de agrupamento de fluxos e se adaptar melhor do que eles a mudanças nos dados. Este projeto final de curso, desenvolvido na Universidade Federal de Minas Gerais, no curso de Engenharia de Controle e Automação, sob orientação do Prof. Janier Arias Garcia, reavalia o algoritmo e propõe um modelo aprimorado. A reavaliação identifica problemas na formulação do algoritmo, diferenças entre a formulação e a implementação pública que produziu os seus resultados publicados e falhas na metodologia dos seus experimentos, entre elas etapas de pré-processamento que vazam informação sobre os dados e uma avaliação que prediz cada lote depois de aprendê-lo. Sob uma metodologia sem essas falhas, na qual todos os modelos são executados na mesma linguagem sobre os dados sem pré-processamento, avaliados de forma prequencial, ajustados pela mesma busca de hiperparâmetros e comparados por testes estatísticos em muitas execuções, o MicroTEDAClus não tem desempenho equivalente ao dos modelos de referência do trabalho original, nem se adapta melhor do que eles a mudanças de conceito.

O modelo aprimorado, MicroTEDAClus+, resulta de experimentos controlados nos quais cada correção ou modificação é aceita por uma regra definida previamente, em execuções separadas das da sua avaliação. Nos conjuntos de dados sintéticos do trabalho original, ele supera o MicroTEDAClus em qualidade de agrupamento, tempo de execução e memória, nenhum modelo de referência o supera nos conjuntos com mudança de conceito, e ele é o mais rápido desse experimento. Em um experimento modernizado no ambiente, nos modelos de referência e na seleção de hiperparâmetros, ele supera todos os modelos de referência em alguns conjuntos de dados e é superado pela maioria deles em outros. Em oito conjuntos de dados reais, sob o mesmo protocolo, ele supera todos os modelos de referência em um conjunto e é superado pela maioria deles em quatro, prediz ao menos tão rápido quanto todos e seu resumo ultrapassa o limite no fluxo mais longo.

Palavras-chave: Agrupamento de fluxos de dados. Agrupamento evolutivo. TEDA. Mudança de conceito. Metodologia experimental.

Abstract

MicroTEDAClus is an evolving clustering algorithm for data streams that summarizes the stream into microclusters by the eccentricity and the typicality of the TEDA framework, and whose authors report that it performs on par with established stream clustering algorithms and adapts better than them to changes in the data. In this undergraduate thesis, developed at the Federal University of Minas Gerais in the Bachelor's Degree in Control and Automation Engineering, under the supervision of Prof. Janier Arias Garcia, we reassess the algorithm and propose an improved model. We find issues in the formulation of the algorithm, differences between the formulation and the public implementation that produced its published results, and flaws in the methodology of its experiments, among them preprocessing steps that leak information about the data and an evaluation that predicts each batch of samples after learning it. Under a methodology without these flaws, in which we execute every model in the same language on the unprocessed data, evaluate it prequentially, tune it by the same automatic search and compare the models over many runs by nonparametric statistical tests, MicroTEDAClus is not on par with the benchmark models of the original work, nor does it adapt better than them to concept drift.

We obtain the improved model, MicroTEDAClus+, from controlled experiments in which we accept each correction or modification by a predefined rule, on runs separate from those of its evaluation. On the synthetic datasets of the original work it improves on MicroTEDAClus in clustering quality, execution time and memory, no benchmark model is better than it in the datasets with concept drift, and it is the fastest model of that experiment. In an experiment modernized in its environment, its benchmark models and its selection of hyperparameters, it is better than every benchmark model in some datasets and worse than most of them in others. On eight real datasets, under the same protocol, it is better than every benchmark model in one dataset and worse than most of them in four, predicts at least as fast as all of them, and its summary outgrows its limit in the longest stream.

Keywords: Data stream clustering. Evolving clustering. TEDA. Concept drift. Experimental methodology.

Acknowledgments

I would like to thank Professor Janier Arias Garcia for his guidance, mentorship, and support — not only in this work but also throughout many other endeavors during my undergraduate studies. I also thank Professor Cristiano Leite de Castro for his willingness to share his experience and ideas regarding this project. Finally, I am grateful to the authors and researchers who produced and developed the knowledge and resources applied in this work.

Contents

Resumo i

Abstract iii

Acknowledgments v

List of Figures xi

List of Tables xiii

1 Introduction 1

1.1 Motivation . . . 2

1.2 Objectives . . . 3

1.3 Structure of the Thesis . . . 3

2 Background and Related Work 5

2.1 Typicality and Eccentricity-Based Data Analytics . . . 5

2.2 Original MicroTEDAClus . . . 6

2.2.1 Issues of the Original Formulation . . . 10

2.3 Updated MicroTEDAClus . . . 11

2.3.1 Issues of the Updated Formulation . . . 12

2.4 A `C` Implementation of MicroTEDAClus . . . 12

2.4.1 Issues of the Evaluation of the C Implementation . . . 13

3 Methodology Reassessment 15

3.1 Model Implementation . . . 15

3.2 Benchmark Models . . . 19

3.3 Datasets . . . 22

3.3.1 Static Datasets . . . 22

3.3.2 Concept Drift Datasets . . . 25

3.3.3 Scalability Streams . . . 26

3.4 Evaluation Protocol . . . 28
3.5 Metrics . . . 29
3.6 Hyperparameter Selection and Runs . . . 31
3.6.1 Hyperparameter Tuning . . . 31
3.6.2 Multi-run Statistics . . . 31
3.7 Statistical Comparison . . . 33
3.8 MicroTEDAClus Under the Reassessed Methodology . . . 34
3.8.1 Clustering Quality . . . 34
3.8.2 Cost . . . 35
3.8.3 Microcluster Count . . . 37
3.9 Hyperparameters of MicroTEDAClus . . . 40
3.10 Summary . . . 40

4 Model Improvement **43**
4.1 Experimental Design . . . 43
4.1.1 Variants and Controls . . . 43
4.1.2 Protocol and Datasets . . . 43
4.1.3 Development and Validation Runs . . . 44
4.1.4 Comparisons and Decision Rule . . . 44
4.2 Implementation Optimization . . . 45
4.2.1 Clustering Quality . . . 45
4.2.2 Execution Time . . . 46
4.3 Corrections . . . 48
4.3.1 Candidate Corrections . . . 48
4.3.2 Corrections One at a Time . . . 50
4.3.3 Combined Corrections . . . 52
4.4 Improvements . . . 53
4.4.1 Candidate Improvements . . . 55
4.4.2 Improvements One at a Time . . . 56
4.4.3 Sensitivity Function . . . 58
4.4.4 Combined Improvements . . . 59
4.4.5 Refinements of the Improved Model . . . 61
4.4.6 Covariance of the Microclusters . . . 63
4.4.7 Order-Independent Densities in MicroTEDAClus+ . . . 64
4.5 Validation . . . 65
4.5.1 Clustering Quality . . . 65
4.5.2 Cost . . . 67

4.6 Limits of the Hyperparameter Selection . . . 69
4.6.1 Reachability Probe . . . 69
4.6.2 Consequences . . . 73
4.7 MicroTEDAClus+ . . . 73
4.7.1 Microclusters . . . 74
4.7.2 Macroclusters . . . 74
4.7.3 Prediction . . . 75
4.7.4 What Remains of the Original Model . . . 75
4.8 MicroTEDAClus+ in the Main Experiment . . . 76
4.8.1 Clustering Quality . . . 76
4.8.2 Cost . . . 80
4.8.3 Microcluster Count . . . 83
4.8.4 Hyperparameters of MicroTEDAClus+ . . . 83
4.8.5 Summary . . . 84

5 Experiment Modernization 85
5.1 Environment . . . 85
5.1.1 Stream Generators . . . 85
5.1.2 Length and Order of the Streams . . . 87
5.2 Benchmark Models . . . 87
5.3 Hyperparameter Selection . . . 89
5.3.1 Ranges by Rule . . . 89
5.3.2 Objective and Search . . . 89
5.4 Execution and Measurements . . . 91
5.5 Results . . . 92
5.5.1 Failures . . . 92
5.5.2 Clustering Quality . . . 92
5.5.3 The Change of Protocol . . . 95
5.5.4 Cost . . . 95
5.6 Summary . . . 99

6 Real Data 101
6.1 Datasets . . . 101
6.1.1 Features . . . 102
6.1.2 Order . . . 102
6.2 Protocol . . . 103
6.3 Results . . . 104
6.3.1 Failures . . . 104

6.3.2 Clustering Quality 106
6.3.3 Cost 106
6.4 Summary 108

7 Discussion **109**
7.1 The Claims of the Original Work 109
7.2 Synthetic and Real Data 110
7.3 The Weight of the Protocol 111
7.4 Limits of the Evidence 111

8 Conclusions **113**
8.1 Final Remarks 113
8.2 Future Work 114

References **117**

A Complementary Tables of the Model Improvement Experiments **123**
A.1 Implementation Optimization 123
A.2 Covariance of the Microclusters 123
A.3 Validation 124

List of Figures

3.1 Execution time and memory of the benchmark models of the original work through each interface 20
3.2 DS3 through the preprocessing of the original work 23
3.3 ARI and execution time of MicroTEDAClus on DS3 after each preprocessing step 24
3.4 Batches of DS3 in the order of its file and ordered by label 25
3.5 Cassini and one of its batches 26
3.6 The streams with concept drift 27
3.7 The scalability stream of two dimensions 28
3.8 ARI per batch reported in the original work 29
3.9 Three hypothetical evolutions of the ARI over the batches 30
3.10 Error of the median and of the interquartile range against the number of runs 33
3.11 ARI of MicroTEDAClus and the benchmark models in the scalability streams 36
3.12 Execution time, memory footprint and microcluster count of MicroTEDAClus and the benchmark models in the scalability streams 39

4.1 Speedup of the *vectorized* implementation against the number of microclusters 48
4.2 ARI of each evaluation batch for the accepted models 67
4.3 Objective of the hyperparameter search against Overall ARI of 400 configurations of MicroTEDAClus+ 71
4.4 ARI of MicroTEDAClus+, MicroTEDAClus and the benchmark models in the scalability streams 79
4.5 Execution time, memory footprint and microcluster count of MicroTEDAClus+, MicroTEDAClus and the benchmark models in the scalability streams 82

5.1 ARI of MicroTEDAClus+ and the benchmark models in the scalability streams of the modernized experiment 94
5.2 Execution time, memory and summary size of MicroTEDAClus+ and the benchmark models in the scalability streams of the modernized experiment 98

6.1 Batch ARI along the evaluation segment of INSECTS abrupt 106

List of Tables

3.1 Settings of the benchmark models in the experiment on the interfaces 21
3.2 The datasets of the main experiment 22
3.3 Clustering quality of MicroTEDAClus and the benchmark models . . 35
3.4 Clustering quality of MicroTEDAClus and the benchmark models in the scalability streams 37
3.5 Execution time and memory footprint of MicroTEDAClus and the benchmark models 38
3.6 Microcluster count of MicroTEDAClus and the benchmark models . . 40
3.7 Hyperparameters selected for MicroTEDAClus 40

4.1 Implementation optimization: clustering quality 46
4.2 Hyperparameter search of the two implementations 46
4.3 Execution time of the two implementations with the same hyperparameters 47
4.4 Corrections, one at a time: clustering quality 51
4.5 Corrections, one at a time: predictions of a single cluster 51
4.6 Combined corrections: clustering quality 52
4.7 Combined corrections: predicted partitions 53
4.8 Partitions predicted by the *corrected* model 53
4.9 Search range of the variance limit: clustering quality 54
4.10 Search range of the variance limit: selected limits 54
4.11 Improvements, one at a time: clustering quality 57
4.12 Hyperparameters and noise of *noise-label* 57
4.13 Improvements, one at a time: predictions of a single cluster 58
4.14 Reparameterizations of the m function: clustering quality 59
4.15 Hyperparameters selected for the sensitivity function 59
4.16 Combined improvements: clustering quality 60
4.17 Direct comparisons between combinations: clustering quality 61
4.18 Refinements of the improved model: clustering quality 62

4.19 Two hyperparameter searches of a single algorithm . . . 63
4.20 Covariance of the microclusters: clustering quality . . . 64
4.21 Order-independent densities in MicroTEDAClus+: clustering quality . 65
4.22 Clustering quality of the accepted models . . . 66
4.23 MicroTEDAClus+ against the *vectorized* model . . . 66
4.24 Cost of the original implementation and of the accepted models . . . 68
4.25 Reachability probe of two search spaces . . . 70
4.26 The two probed configurations of MicroTEDAClus+ along the stream 72
4.27 Differences between MicroTEDAClus+ and MicroTEDAClus . . . 74
4.28 Clustering quality of MicroTEDAClus+, MicroTEDAClus and the benchmark models . . . 77
4.29 Clustering quality of MicroTEDAClus+, MicroTEDAClus and the benchmark models in the scalability streams . . . 78
4.30 Execution time and memory footprint of MicroTEDAClus+, MicroTEDAClus and the benchmark models . . . 81
4.31 Microcluster count of MicroTEDAClus+, MicroTEDAClus and the benchmark models . . . 83
4.32 Hyperparameters selected for MicroTEDAClus+ . . . 83

5.1 Models with fixed hyperparameters on the streams generated by R and by the port . . . 86
5.2 Share of the samples of the Random RBF stream that carry the noise label . . . 86
5.3 Hyperparameters of the models and their categories . . . 88
5.4 Rule of each category of hyperparameter . . . 90
5.5 Search ranges that the rules give . . . 90
5.6 Failed runs of the modernized experiment . . . 92
5.7 Clustering quality of MicroTEDAClus+ and the benchmark models . 93
5.8 Clustering quality of MicroTEDAClus+ and the benchmark models in the scalability streams . . . 93
5.9 MicroTEDAClus+ under the two protocols of hyperparameter selection 95
5.10 Execution time and memory of MicroTEDAClus+ and the benchmark models . . . 96
5.11 Summary size of MicroTEDAClus+ and the benchmark models . . . 97

6.1 The real datasets . . . 102
6.2 Batches of a single class in the real datasets . . . 103
6.3 Failed runs on the real datasets . . . 104

6.4 Clustering quality of MicroTEDAClus+ and the benchmark models on the real datasets 105
6.5 Execution time and memory of MicroTEDAClus+ and the benchmark models on the real datasets 107

A.1 Implementation optimization: microcluster count and memory footprint 123
A.2 Covariance of the microclusters: microcluster count and memory footprint 124
A.3 Microcluster count and memory footprint of the accepted models . . . 125

Chapter 1

Introduction

Data streams arrive continuously and without a known end, and the process that generates them can change while they are observed. Clustering such a stream requires an algorithm that processes each sample as it arrives, keeps a bounded summary of what it has seen and follows the changes of the data, a family of methods known as stream or evolving clustering [1–3]. MicroTEDAClus [4] is one such algorithm. It summarizes the stream into microclusters by the eccentricity and the typicality of the TEDA framework [5], which assume no distribution for the data and are computed recursively, and it groups the microclusters into macroclusters that represent the clusters of the stream. Its authors report that it performs on par with established stream clustering algorithms, and that it adapts better than them to concept drift.

In this work, we reassess MicroTEDAClus and propose an improved model. We find issues in the formulation of the algorithm, differences between the formulation and the implementation that produced the published results, and flaws in the methodology of the original experiments, among them preprocessing steps that leak information about the stream and an evaluation that predicts each batch after learning it. Under a methodology that corrects these flaws, MicroTEDAClus is not on par with the benchmark models of the original work, nor does it adapt better than them to concept drift. We obtain the improved model, MicroTEDAClus+, from controlled experiments on corrections and modifications of the algorithm. On the synthetic datasets of the original work it improves on MicroTEDAClus in clustering quality, execution time and memory, and no benchmark model is better than it in the datasets with concept drift. In an experiment modernized in its environment, its benchmark models and its selection of hyperparameters, it is better than every benchmark model in some datasets and worse than most of them in others. On real datasets, under the same protocol, it is better than every benchmark model in one dataset and worse than most of them in four, predicts at least as fast as all of them, and its summary outgrows the limit in the

longest stream.

1.1 Motivation

The results of MicroTEDAClus are the basis of later work. The doctoral thesis of Galindo [6] updates the algorithm and applies it to the last-mile problem in logistics, assuming the validity of the original evaluation without examining it. The undergraduate thesis of Cardoso [7] reimplements it in `C` to reduce its execution time, evaluates its quality only on static datasets that the model has already learned, and expects it to handle concept drift because the original work reports that it does. A model whose evaluation supports further work must be verified, and the verification of empirical results in machine learning is not a formality: information leaked from the data into the evaluation, among other errors, has been found to inflate the published results of many studies [8, 9]. The algorithm is also attractive for what it promises. The TEDA metrics require neither a distribution for the data nor a predefined number of clusters, and an algorithm built on them that is accurate, fast and robust to concept drift would be a useful tool for stream clustering.

The datasets of the original work are synthetic streams and benchmarks of static shapes, whose clusters, drifts and noise are known by construction. A model that is to serve applications must also be judged on data recorded from physical processes, whose classes are not drawn from chosen distributions and whose drifts come from the conditions of the recording. An evaluation on real data tests whether the conclusions reached on synthetic data hold.

We verify the claims of MicroTEDAClus before building on them, and improve the model where the verification shows weaknesses. The TEDA framework is that of Angelov [5]. The formulation of MicroTEDAClus and its reference implementation are those of Maia et al. [4], and its updated formulation is that of Galindo [6]. Cardoso [7] had earlier observed that the outlier condition cannot hold for microclusters with two samples, and proposed an intersection condition with a factor. The benchmark models are those of their respective authors. In the main experiment, we execute them through the implementations of the `River` library [10] or, for CEDAS, TEDACloud and AutoCloud, through implementations that we made from the official code or from the descriptions in their papers; in the modernized and real-data experiments, through `River`, `scikit-learn` [11] and the `bico` package [12]. The real datasets are those of the sources cited where they are introduced. The reassessment of the formulation, of the implementation and of the methodology, the methodology that replaces the original one, the vectorized implementation, the experiments of the improvement and

MicroTEDAClus+, the modernized experiment with its ported stream generators and its search ranges by rule, and the real-data experiment are our contribution.

1.2 Objectives

From the motivation presented in the previous section, our objectives are:

a) to examine the formulation of MicroTEDAClus and its public implementation, and identify their issues and the differences between them;

b) to examine the methodology of the experiments of the original work, and define a methodology that evaluates every model alike and without its flaws;

c) to evaluate MicroTEDAClus against benchmark models under that methodology;

d) to propose an improved model, and evaluate it under the same methodology against MicroTEDAClus and the benchmark models;

e) to modernize the experiment and to evaluate the improved model under it;

f) to evaluate the improved model on real datasets.

1.3 Structure of the Thesis

Besides this introduction, the thesis has seven chapters. In Chapter 2, we present the TEDA framework, the formulation of MicroTEDAClus and its update, together with the issues of both. In Chapter 3, we examine the implementation of MicroTEDAClus and the methodology of the original experiments, define our methodology and evaluate MicroTEDAClus and the benchmark models under it. In Chapter 4, we optimize the implementation, evaluate corrections and improvements of the algorithm, define MicroTEDAClus+ and compare it with MicroTEDAClus and the benchmark models in the main experiment. In Chapter 5, we replace the environment, the benchmark models and the selection of hyperparameters, and repeat the main experiment under them. In Chapter 6, we evaluate MicroTEDAClus+ and the benchmark models on real datasets. In Chapter 7, we read the results of these chapters together, and in Chapter 8 we summarize the findings, their limits and the directions of future work. Appendix A collects complementary tables of the experiments of Chapter 4.

Chapter 2

Background and Related Work

This chapter presents the eccentricity and the typicality of the TEDA framework (Section 2.1), the formulation of MicroTEDAClus by Maia et al. (Section 2.2), its update by Galindo (Section 2.3), and the reimplementation of the model in `C` by Cardoso (Section 2.4). After each formulation, we discuss its issues, and we point to where the following chapters examine them.

2.1 Typicality and Eccentricity-Based Data Analytics

Angelov [5] introduces the TEDA framework as an alternative to probability theory for data in non-deterministic settings, built on two metrics, the eccentricity and the typicality. Let

$$C = \{\sum_{i=1}^{k} \pi_i^k > 0, \quad j > 1, \quad k > 2\} \tag{2.1}$$

be the conditions under which the metrics of a sample j are computed at moment k, where π_j^k is the sum of the distances from sample j to all the samples seen up to k. The eccentricity, which measures how much a sample deviates from the others, and the typicality, which measures how well it fits among them, are

$$\xi_j^k = \frac{2\pi_j^k}{\sum_{i=1}^{k} \pi_i^k}, \quad C \tag{2.2}$$

$$\tau_j^k = 1 - \xi_j^k, \quad C \tag{2.3}$$

Both are bounded, $0 < \xi_j^k < 1$ and $0 < \tau_j^k < 1$, and their normalized forms are

$$\zeta_j^k = \frac{\xi_j^k}{2}, \quad \sum_{i=1}^{k} \zeta_i^k = 1, \quad 0 < \zeta_j^k < \frac{1}{2}, \quad C \tag{2.4}$$

$$t_j^k = \frac{\tau_j^k}{k-2}, \quad \sum_{i=1}^{k} t_i^k = 1, \quad 0 < t_j^k < \frac{1}{k-2}, \quad C \tag{2.5}$$

A sample is anomalous if $\zeta_j^k > 1/k$ and typical if $t_j^k > 1/k$. For the squared Euclidean distance, the sum of the distances can be updated recursively from the mean and the mean squared norm of the samples,

$$\mu^k = \frac{k-1}{k}\mu^{k-1} + \frac{1}{k}x_k, \qquad X^k = \frac{k-1}{k}X^{k-1} + \frac{1}{k}\|x_k\|^2, \tag{2.6}$$

since

$$\pi_j^k = k\left(\|x_j - \mu^k\|^2 + X^k - \|\mu^k\|^2\right), \qquad \sum_{i=1}^{k}\pi_i^k = \sum_{i=1}^{k-1}\pi_i^{k-1} + 2\pi_k^k. \tag{2.7}$$

Angelov also states that the metrics suit other proximity functions, among them the cosine similarity and the Mahalanobis distance [13].

2.2 Original MicroTEDAClus

Maia et al. [4] use the eccentricity and the typicality over the Euclidean distance to create and update microclusters, by detecting outliers and typical samples, and to merge the microclusters into macroclusters. They write the eccentricity in the recursive form

$$\xi(x_k) = \frac{1}{k} + \frac{(\mu_k - x_k)^T(\mu_k - x_k)}{k\sigma_k^2}, \tag{2.8}$$

with

$$\mu_k = \frac{k-1}{k}\mu_{k-1} + \frac{x_k}{k}, \qquad \sigma_k^2 = \frac{k-1}{k}\sigma_{k-1}^2 + \frac{1}{k-1}\|x_k - \mu_k\|^2, \tag{2.9}$$

and consider a sample an outlier if

$$\zeta(x_k) > \frac{m^2+1}{2k}, \quad m > 0, \tag{2.10}$$

where m is a function of the number of samples chosen by the authors,

$$m(k) = \frac{3}{1 + e^{-0.007(k-100)}}. \tag{2.11}$$

A microcluster i at moment k is described by:

- S_k^i, its number of samples;
- μ_k^i and $(\sigma_k^i)^2$, the mean and the variance of its samples;
- $\xi^i(x_k)$ and $\zeta^i(x_k)$, the eccentricity and the normalized eccentricity of sample x_k to it;
- $\tau^i(x_k)$ and $t^i(x_k)$, the typicality and the normalized typicality of x_k to it;
- $D_k^i = 1/\zeta^i(x_k)$, its density;
- $m_k^i(S_k^i)$, its threshold function.

The metrics apply to each microcluster as to the whole data. For $S_k^i > 2$, a sample x_k is an outlier to microcluster i if

$$\zeta^i(x_k) > \frac{m_k^i(S_k^i)^2 + 1}{2S_k^i}, \tag{2.12}$$

and, for $S_k^i = 2$, if

$$\left(\zeta^i(x_k) > \frac{m_k^i(2)^2 + 1}{4}\right) \wedge \left(\left(\sigma_k^i\right)^2 < r_0\right), \tag{2.13}$$

where r_0 is a hyperparameter, the variance limit. Every microcluster to which x_k is not an outlier is updated by

$$\begin{aligned} S_k^i &= S_{k-1}^i + 1, \qquad \mu_k^i = \frac{S_k^i - 1}{S_k^i}\mu_{k-1}^i + \frac{x_k}{S_k^i}, \\ \left(\sigma_k^i\right)^2 &= \frac{S_k^i - 1}{S_k^i}\left(\sigma_{k-1}^i\right)^2 + \frac{1}{S_k^i - 1}\left(\frac{2\|x_k - \mu_k^i\|}{d}\right)^2, \\ \xi^i(x_k) &= \frac{1}{S_k^i} + \frac{2\left(\mu_k^i - x_k\right)^T\left(\mu_k^i - x_k\right)}{S_k^i\left(\sigma_k^i\right)^2 d}, \end{aligned} \tag{2.14}$$

where d is the dimension of the data, and a sample that is an outlier to every microcluster creates a new one with

$$S_k^i = 1, \quad \mu_k^i = x_k, \quad (\sigma_k^i)^2 = 0, \tag{2.15}$$

as Algorithm 1 describes.

Algorithm 1 Update of the microclusters in the original formulation

Require: x_k, r_0
Ensure: m_i, $i = 1, 2, \ldots, n$

while new samples are available **do**
  if $k = 1$ **then**
    Create m_1 by Equation (2.15)
  else
    flag $\leftarrow$ true
    for $i = 1$ to n **do**
      if $S_k^i = 2$ **then**
        Decide whether x_k is an outlier to m_i by Equation (2.13)
      else
        Decide whether x_k is an outlier to m_i by Equation (2.12)
      end if
      if x_k is not an outlier to m_i **then**
        Update m_i by Equation (2.14)
        flag $\leftarrow$ false
      end if
    end for
    if flag **then**
      Create m_{n+1} by Equation (2.15)
    end if
  end if
end while

The macroclusters are the connected components of the graph whose vertices are the microclusters and whose edges join the microclusters that satisfy

$$\text{dist}(\mu_k^i, \mu_k^j) < 2\left(\sigma_k^i + \sigma_k^j\right), \quad i \neq j. \tag{2.16}$$

To keep overlapping microclusters from collapsing into a single macrocluster, a microcluster m_l^j takes part in the macrocluster $\mathcal{M}_j = \{m_1^j, \ldots, m_{N_j}^j\}$ only if it is active, that is, if

$$D_k^l \geq \text{mean}\left(D_k^l\right), \quad l = 1, \ldots, N_j. \tag{2.17}$$

Each active microcluster weighs in its macrocluster by its share of the density of the active microclusters,

$$w_k^l = \frac{D_k^l}{\sum_{l \in \mathcal{M}_j} D_k^l}, \tag{2.18}$$

the typicality of a sample x_k to a macrocluster is the weighted sum of its typicalities to the active microclusters,

$$\mathcal{T}_j(x_k) = \sum_{l \in \mathcal{M}_j} w_k^l t_k^l(x_k), \tag{2.19}$$

and the sample is assigned to the macrocluster of highest typicality,

$$j = \arg\max_j \mathcal{T}_j(x_k), \tag{2.20}$$

as Algorithm 2 describes. The formulation defines no decay of the microclusters, although the public implementation has one (Section 3.1).

Algorithm 2 Update of the macroclusters in the original formulation

Require: x_k, m_i, $i = 1, 2, \ldots, n$
Ensure: the macrocluster of x_k
1: **while** new samples are available **do**
2: Group the microclusters that satisfy Equation (2.16) into macroclusters $\mathcal{M}_j$
3: Find the active microclusters of each macrocluster by Equation (2.17)
4: Compute the typicality of x_k to each macrocluster by Equation (2.19)
5: Assign x_k to the macrocluster of highest typicality
6: **end while**

2.2.1 Issues of the Original Formulation

Order-dependent density. The eccentricity and the typicality relate a sample to a microcluster, so the density $D_k^i = 1/\zeta^i(x_k)$ relates them too, and should not be treated as a property of the microcluster. A microcluster with a small variance is dense for any standard clustering algorithm [1, 2, 14], but if the current sample is far from its mean, $\zeta^i(x_k)$ is large and D_k^i small, and the microcluster is less likely to be activated. Conversely, a microcluster with a large variance and a sample close to its mean has a large D_k^i and is more likely to be activated, even if it is not dense at all. Section 4.3 tests two densities that do not depend on the current sample, and Section 4.4.7 tests both again on MicroTEDAClus+.

Outlier condition for microclusters with two samples. For $S_k^i = 2$, the expressions of Section 2.1 give

$$\mu_2 = \frac{1}{2}(x_1 + x_2), \qquad \sigma_2^2 = \|x_2 - \mu_2\|^2, \qquad \xi(x_2) = 1, \qquad \zeta(x_2) = 0.5, \tag{2.21}$$

and, because $m \geq 1$,

$$\frac{m_k^i(2)^2 + 1}{4} \geq 0.5. \tag{2.22}$$

The first term of Equation (2.13) is therefore never satisfied, and neither is the conjunction: no sample can be an outlier to a microcluster with two samples. With the scaled expressions of Equation (2.14), the normalized eccentricity of the second sample becomes $1/4 + d/8$, so the first term is never satisfied with one or two dimensions and always satisfied with three or more, where the condition reduces to $(\sigma_k^i)^2 < r_0$, which holds for the samples close to the mean. Either way, the condition does not detect outliers. Replacing it by

$$\left(\sigma_k^i\right)^2 < r_0 \tag{2.23}$$

would keep the inversion, so the condition should be

$$\left(\sigma_k^i\right)^2 > r_0. \tag{2.24}$$

Cardoso [7] reached the same conclusion earlier, proving that the eccentricity of the second sample is always one, and detects the outliers of microclusters with fewer than three samples by Equation (2.24) alone, as the public implementation does (Section 3.1).

Dimensionality penalty. Equation (2.14) scales the variance and the eccentricity of a microcluster by the dimension d. No paper that introduces the TEDA metrics suggests such a scaling [5, 15], the original work does not justify it, and it lets the normalized eccentricity exceed its bound of one half. Section 3.8 traces the collapse of MicroTEDAClus in high dimensions to it, and Section 4.3 tests its removal.

2.3 Updated MicroTEDAClus

In the PhD thesis *Mineração de Dados Aplicada a Problemas de Last-Mile na Logística*, Galindo [6] updates MicroTEDAClus to apply it to the last-mile problem in logistics; we describe only the changes. The condition for microclusters with two samples becomes

$$\left(\zeta_2^i(x_2) > \frac{m^i(2)^2 + 1}{4}\right) \wedge \left(\left(\sigma_k^i\right)^2 > r_0\right), \tag{2.25}$$

which corrects the direction of the comparison with r_0 but keeps the conjunction.

An explicit decay is introduced. Each microcluster has a vitality v_k^i, set to one when it is created and lowered by

$$v_k^i = v_{k-1}^i - \rho \tag{2.26}$$

whenever the sample x_k is an outlier to it, where $\rho = 1/\text{decay}$ and *decay* is a hyperparameter. A microcluster is removed when $v_k^i \leq 0$. Section 4.4 tests this rule as the candidate *recency*.

To suit the data of the problem, the Euclidean distance is replaced by the cosine similarity, which Angelov [5] accepts for the TEDA metrics. The outlier condition becomes

$$\left(x_k - \mu_k^i\right)^2 > \left(m_k^i\right)^2 \left(\sigma_k^i\right)^2, \tag{2.27}$$

and the update of a microcluster becomes

$$\begin{aligned} S_k^i &= S_{k-1}^i + 1, \qquad \mu_k^i = \frac{S_k^i - 1}{S_k^i}\mu_{k-1}^i + \frac{1}{S_k^i}\frac{x_k}{\|x_k\|}, \quad \|x_k\| \neq 0, \\ \left(\sigma_k^i\right)^2 &= \frac{S_k^i - 1}{S_k^i}\left(\sigma_{k-1}^i\right)^2 + \frac{1}{S_k^i - 1}\left(\frac{2\|x_k - \mu_k^i\|}{d}\right)^2, \\ \xi^i(x_k) &= \frac{1 - \mu_k^i \frac{x_k}{\|x_k\|}}{1 + k\left(\mu_k^i\right)^2}, \quad \|x_k\| \neq 0. \end{aligned} \tag{2.28}$$

The rest of the algorithm, including the formation of the macroclusters, is unchanged.

2.3.1 Issues of the Updated Formulation

Issues that remain. The order-dependent density and the dimensionality penalty remain, and the condition for microclusters with two samples keeps its conjunction.

An outlier condition derived for another distance. Angelov [15] derives Equation (2.10) from Equation (2.27) for the Euclidean distance, and states that a similar condition can be derived for the cosine similarity without deriving it. The updated model uses the Euclidean condition with the cosine similarity without justification.

A Euclidean variance with the cosine similarity. The eccentricity is adapted to the cosine similarity, but the variance of Equation (2.28) is still the Euclidean one, and nothing shows that it is compatible with the cosine similarity.

2.4 A `C` Implementation of MicroTEDAClus

In the undergraduate thesis *Implementação do algoritmo MicroTEDAclus para classificação de dados*, Cardoso [7] reimplements the original MicroTEDAClus in `C`, a language chosen for its speed and for its use in embedded platforms. Along the way, the thesis documents the TEDA expressions that the model uses, with proofs and unit tests, and finds that the recursive eccentricity of Equation (2.8) corresponds to the eccentricity computed over the squared Euclidean distance, as the recursive expressions of Equation (2.7) assume. It also distinguishes two modes of operation: in the online mode, the model updates its macroclusters along the stream and labels each sample as it arrives; in the offline mode, it learns the whole dataset, builds its macroclusters once and then labels every sample.

Cardoso proposes an alternative to the intersection condition of Equation (2.16). The alternative takes $m_k^i(S_k^i)\,\sigma_k^i$ as the radius of a microcluster, the distance from its mean beyond which a sample is an outlier to it, and compares the distance between the means of two microclusters with the sum of their radii, scaled by a factor ϕ that loosens the condition relative to that sum:

$$\operatorname{dist}(\mu_k^i, \mu_k^j) < \phi \left(m_k^i(S_k^i)\,\sigma_k^i + m_k^j(S_k^j)\,\sigma_k^j \right), \quad \phi \geq 1, \quad i \neq j. \tag{2.29}$$

Section 4.4.1 tests a factor of its own on the condition of Equation (2.16).

The experiments of the thesis evaluate the clustering quality by the adjusted Rand index on seven static datasets of two dimensions, two of them taken from the original work. For each dataset and intersection condition, the model runs in the offline mode

with random orders of the samples and with hyperparameters taken from a grid, and the thesis reports the lowest and the highest index and the highest mean index over the configurations [7, Section 4.3]. It concludes that the quality depends strongly on r_0 and ϕ and weakly on the decay of Equation (2.26), and that Equation (2.29) is comparable to Equation (2.16), with a higher index in one dataset.

The thesis also compares the execution time of the reimplementation with that of the `Python` implementation of the original work. It measures each whole program, including the reading of the data, with the `time` command of the operating system on a single machine, and reports no repetitions [7, Section 4.4]. It reports that the reimplementation runs much faster than the original one, and that its execution time grows steeply with the number of samples, which it attributes to the construction of the macroclusters, quadratic in the number of microclusters [7, Sections 4.4 and 4.5].

2.4.1 Issues of the Evaluation of the C Implementation

Hyperparameters selected with the labels of the evaluated data. The highest index and the highest mean index belong to the configurations of the grid that best fit the labels of the evaluated data, and are therefore optimistic estimates of the quality that the model reaches when its hyperparameters are chosen without those labels. The lowest index is no better than chance in every dataset [7, Tables 4.1 and 4.2], so the quality reported depends on a choice made with the labels. The comparison between the two intersection conditions rests on these best cases, without a statistical test.

Scope of the comparison. The model is compared with no benchmark model, and none of the datasets has concept drift, which the thesis leaves out of its scope and expects the model to handle because the original work reports that it does [7, Section 4.2]. The claims of the original work about the benchmark models and about concept drift therefore remain untested.

Chapter 3

Methodology Reassessment

Maia et al. [4] report that MicroTEDAClus performs on par with established stream clustering algorithms and adapts better than them to concept drift. In this chapter, we examine what produced those results: the implementation of the model (Section 3.1), the benchmark models and how they were executed (Section 3.2), the preprocessing of the datasets (Section 3.3) and the evaluation protocol (Section 3.4). We then define the methodology that the rest of the thesis uses: the metrics (Section 3.5), the selection of hyperparameters and the number of runs (Section 3.6), and the statistical comparison (Section 3.7). Finally, we evaluate MicroTEDAClus against six benchmark models under that methodology (Sections 3.8 and 3.9).

3.1 Model Implementation

The original work points to a public repository with the code of its experiments, `https://github.com/cseveriano/evolving_clustering`, which implements MicroTEDAClus in `src/models/evolving/EvolvingClustering.py`. To preserve it, we forked the repository at its last commit, `50dd2b4` [16].

This implementation is not the formulation of the paper, but it is the code that produced the published results. It shares features with the updated formulation of Galindo [6], and it solves the outlier condition of microclusters with two samples (Section 2.2.1), but it departs from the formulation in the decay of the microclusters, in their activation, in the eccentricity and in the membership.

Decay. The constructor turns the hyperparameter `decay` into `fading_factor` = 1/decay, the rate ρ of the decay of Galindo (Equation (2.26)). The factor acts in two places. The pruning (Listing 3.1) lowers, at every update, the vitality of the microclusters that are not active, while Galindo lowers that of the microclusters that

```python
def prune_micro_clusters(self):
    for mc in self.micro_clusters:
        if not mc["active"]:
            mc["life"] = mc["life"] - self.fading_factor
            if mc["life"] < 0:
                self.micro_clusters.remove(mc)
                if mc in self.changed_micro_clusters:
                    self.changed_micro_clusters.remove(mc)
                self.graph.remove_node(mc["id"])
```

Listing 3.1: Pruning of the microclusters in the public implementation of MicroTEDAClus

```python
def update_life(self, xk, micro_cluster):
    previous_mean = micro_cluster["mean"]
    previous_var = micro_cluster["variance"]
    if previous_var > 0:
        d = EvolvingClustering.get_euclidean_distance(xk, previous_mean)
        dist = np.sqrt(np.sum(d))
        rt = np.sqrt(previous_var)
        micro_cluster["life"] = micro_cluster["life"] + (((rt - dist) / rt)
    * self.fading_factor)
    else:
        micro_cluster["life"] = 1
```

Listing 3.2: Update of the vitality of a microcluster in the public implementation of MicroTEDAClus

the current sample does not update; activation is decided only when the macroclusters are built. The vitality update (Listing 3.2) rewards a microcluster when a sample falls within one standard deviation of its mean and penalizes it otherwise, a rule that appears in neither work and that neither explains.

Activation. The paper activates the microclusters whose density is not below the mean density of their macrocluster (Section 2.2 and Equation (2.17)). The implementation builds the macroclusters twice, as Algorithm 3 describes in our reading of the code: it groups the intersecting microclusters into the components of a graph $\mathbb{G}_0$, activates microclusters within each component, removes the inactive ones to obtain a graph $\mathbb{G}_1$, and returns the components of $\mathbb{G}_1$ as the macroclusters. The second pass guards against the collapse of overlapping microclusters into a single macrocluster, which the original work anticipates. Listing 3.3 also shows that only microclusters with more than two samples can be active, while the mean density is computed over all the microclusters of the component.

Algorithm 3 Construction of the macroclusters in the public implementation of MicroTEDAClus

Require: the microclusters m_i, $i = 1, 2, \ldots, n$
Ensure: the macroclusters $\mathcal{M}_j$, $j = 1, 2, \ldots, m$
1: Build the graph $\mathbb{G}_0$ whose edges join the microclusters that satisfy Equation (2.16)
2: Build a macrocluster $\mathcal{M}_j$ for each connected component of $\mathbb{G}_0$
3: **for** each macrocluster $\mathcal{M}_j$ **do**
4: Activate its microclusters by Equation (2.17)
5: **end for**
6: Remove the inactive microclusters from $\mathbb{G}_0$ to obtain $\mathbb{G}_1$
7: Build a macrocluster $\mathcal{M}_j$ for each connected component of $\mathbb{G}_1$

```
def define_activations(self):
    self.active_graph = self.graph.copy()
    for mg in self.macro_clusters:
        num_micro = len(mg)
        total_density = 0
        for i in mg:
            dens = next(item["density"] for item in self.micro_clusters if
    item["id"] == i)
            total_density += dens
        mean_density = total_density / num_micro
        for i in mg:
            mi = next(item for item in self.micro_clusters if item["id"] ==
    i)
            mi["active"] = (mi["num_samples"] > 2) and (mi["density"] >=
    mean_density)
            if not mi["active"]:
                self.active_graph.remove_node(mi["id"])
    self.active_macro_clusters =
    list(nx.connected_components(self.active_graph))
```

Listing 3.3: Activation of the microclusters in the public implementation of MicroTEDAClus

```
def is_outlier(self, s_ik, var_ik, norm_ecc):
    if s_ik < 3:
        outlier = (var_ik > self.variance_limit)
    else:
        mik_sik = 3 / (1 + math.exp(-0.007 * (s_ik - 100)))
        outlier_limit = ((mik_sik ** 2) + 1) / (2 * s_ik)
        outlier = (norm_ecc > outlier_limit)
    return outlier
```

Listing 3.4: Outlier test in the public implementation of MicroTEDAClus

```
def get_eccentricity(x, num_samples, mean, var):
    if var == 0 and num_samples > 1:
        result = (1/num_samples)
    else:
        a = mean - x
        result = ((1 / num_samples) + ((np.linalg.norm(a) * 2 / len(a)) ** 2
    / (num_samples * var)))
    return result
```

Listing 3.5: Eccentricity in the public implementation of MicroTEDAClus

Outlier test. Listing 3.4 applies Equation (2.12) to microclusters with three samples or more, and, below three, the test of the variance alone of Equation (2.24), which replaces the conjunction of Equation (2.13).

Eccentricity. The eccentricity of Listing 3.5 expands to Equation (3.1), which differs from the recursive eccentricity of Equation (2.8) by the factor $4/d^2$ on its second term.

$$\xi^i(x) = \frac{1}{S_k^i} + \frac{4\|\mu_k^i - x\|^2}{d^2 S_k^i \left(\sigma_k^i\right)^2} \tag{3.1}$$

Membership. The membership of a sample to a macrocluster (Listing 3.6) weighs $1-\zeta$ by the densities of the active microclusters, an expression that cannot be rewritten as the normalized typicality of Equation (2.5).

The implementation of the updated formulation, which Galindo provided to us, is not public. It adds partial support for the cosine similarity, refactors the decay and moves the microclusters into a class of their own.

```
def calculate_membership(x, active_micro_clusters):
    total_density = 0
    for m in active_micro_clusters:
        total_density += m["density"]
    mb = 0
    for m in active_micro_clusters:
        d = m["density"]
        t = 1 - EvolvingClustering.get_normalized_eccentricity(x,
    m["num_samples"], m["mean"], m["variance"])
        mb += (d / total_density) * t
    return mb
```

Listing 3.6: Membership in the public implementation of MicroTEDAClus

3.2 Benchmark Models

The original work compares MicroTEDAClus with DenStream [2], CluStream [1] and StreamKM ++ [17], benchmarks common in the literature on stream clustering [3, 18–22]. It executes them from `Python` through `rpy2`, which calls the `R` package `streamMOA`, itself a wrapper of the `Java` framework MOA [23]. These two layers of interface add to the time and memory that the experiment measures.

To quantify them, we timed each model and measured its memory on a stream of the Random RBF generator with 100,000 samples, 10 dimensions, five classes and 50 clusters, eight times, through the bridge and directly in `Java`, with the settings of Table 3.1. Through the bridge, every model takes much longer, while its memory barely changes (Figure 3.1, top). The comparison of the original work therefore measures the interfaces together with the models, and a fair comparison requires every model to run in the environment of the evaluated one. Since MicroTEDAClus is implemented in `Python`, we use the implementations of DenStream, CluStream and StreamKM in the `River` library [10]. Their times are close to those of the native implementations, except for a slower CluStream, and their memory is smaller (Figure 3.1, bottom). The settings in `River` follow those of `streamMOA` as closely as its parameters allow: its `mu` is `streamMOA`'s `mu` times `offline`, and it requires $\mu\beta > 1$, hence $\beta = 0.6$.

The original work cites TEDACloud [24] and CEDAS [18] but does not evaluate them. We ported the official `MATLAB` implementation of CEDAS to `Python` and implemented TEDACloud from the description in its paper. We also implemented AutoCloud [25], which improves on TEDACloud and appeared at about the same time as MicroTEDAClus, from its paper. The three implementations are in the public repository of this work. The benchmark models of the following experiments are thus six: DenStream, CluStream, StreamKM, AutoCloud, CEDAS and TEDACloud.

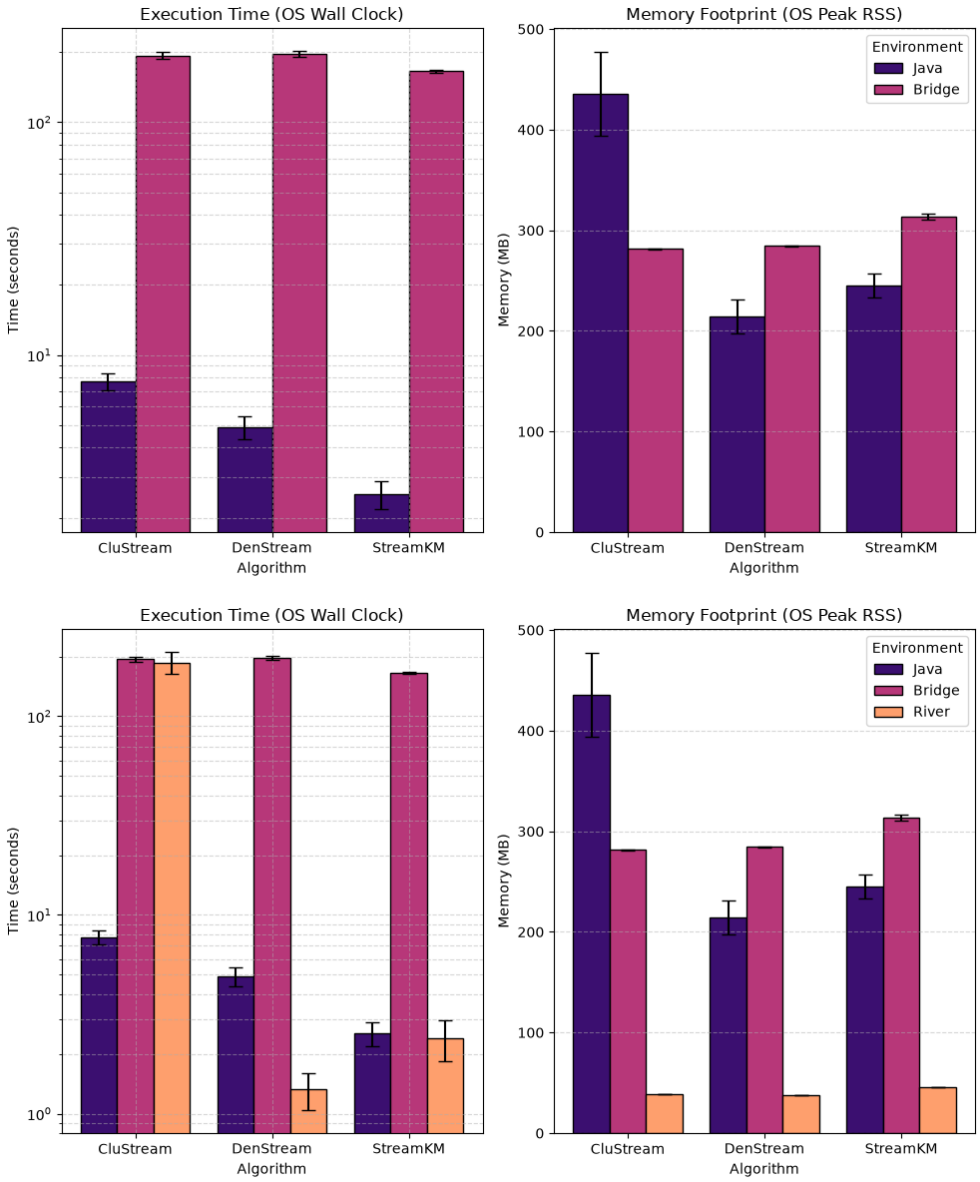


Figure 3.1: Execution time, at left, and memory, at right, of the benchmark models of the original work through the `rpy2` bridge and directly in `Java`, at top, and in `River`, at bottom.

Table 3.1: Settings of the benchmark models of the original work in the experiment on the interfaces.

Model	Package	Parameter	Value
CluStream	streamMOA	maxNumKernels	100
		k	5
	River	max_micro_clusters	100
		n_macro_clusters	5
DenStream	streamMOA	epsilon	0.02
		mu	1.0
		beta	0.2
		lambda	0.001
		initPoints	100
		offline	2.0
	River	epsilon	0.02
		mu	2.0
		beta	0.6
		decaying_factor	0.001
		n_samples_init	100
StreamKM	streamMOA	sizeCoreset	10000
		numClusters	5
		length	100000
	River	chunk_size	10000
		n_clusters	5

3.3 Datasets

The original work evaluates the models on static datasets, on streams with concept drift and on synthetic streams of increasing dimension for its scalability experiments. These datasets are common in the literature [24–28], but the files of the original work hide preprocessing decisions that change the results and that a stream does not allow. Table 3.2 lists the datasets that we use instead, with the abbreviations of the tables.

Table 3.2: The datasets of the main experiment and their abbreviations.

Dataset	Abbreviation	Kind
StreamBenchmark(1)	B1	Concept drift
StreamBenchmark(2)	B2	Concept drift
RandomRBFGeneratorEvents	RBF	Concept drift
Cassini	CSN	Static
DS3	DS3	Static
DS4	DS4	Static
SCL-1 to SCL-64	S1 to S64	Scalability, of 1 to 64 dimensions

3.3.1 Static Datasets

According to the original work, its static datasets ST-D1 and ST-D2 are datasets introduced to evaluate the Chameleon algorithm [29]. Its repository holds the files `chameleon-ds3-clean.csv` and `chameleon-ds4-clean.csv`, while the Chameleon paper names the corresponding raw patterns, of 10,000 and 8,000 samples, DS3 and DS4. We use the raw files `cluto-t7-10k.arff` and `cluto-t8-8k.arff`, and those names. The files of the repository are cleaned of noise, downsampled, scaled and ordered by label. To measure what each of these steps does, we applied them in turn to DS3, as Figure 3.2 shows, and tuned and evaluated MicroTEDAClus on the result of each step by the protocol of this chapter (Figure 3.3).

Noise removal. Removing the samples labeled as noise changes the ARI little. It is, however, a severe leak [8, 9]: a sample is noise only if it belongs to no cluster, which is known after the clustering. In a stream, the model itself must decide which samples are noise, from the samples it has seen, or not handle noise at all [3, 19–22, 26, 28].

Downsampling. Keeping a random half of the samples needs no information about them and is not a leak. It lowers the ARI slightly and the learning time by about half.

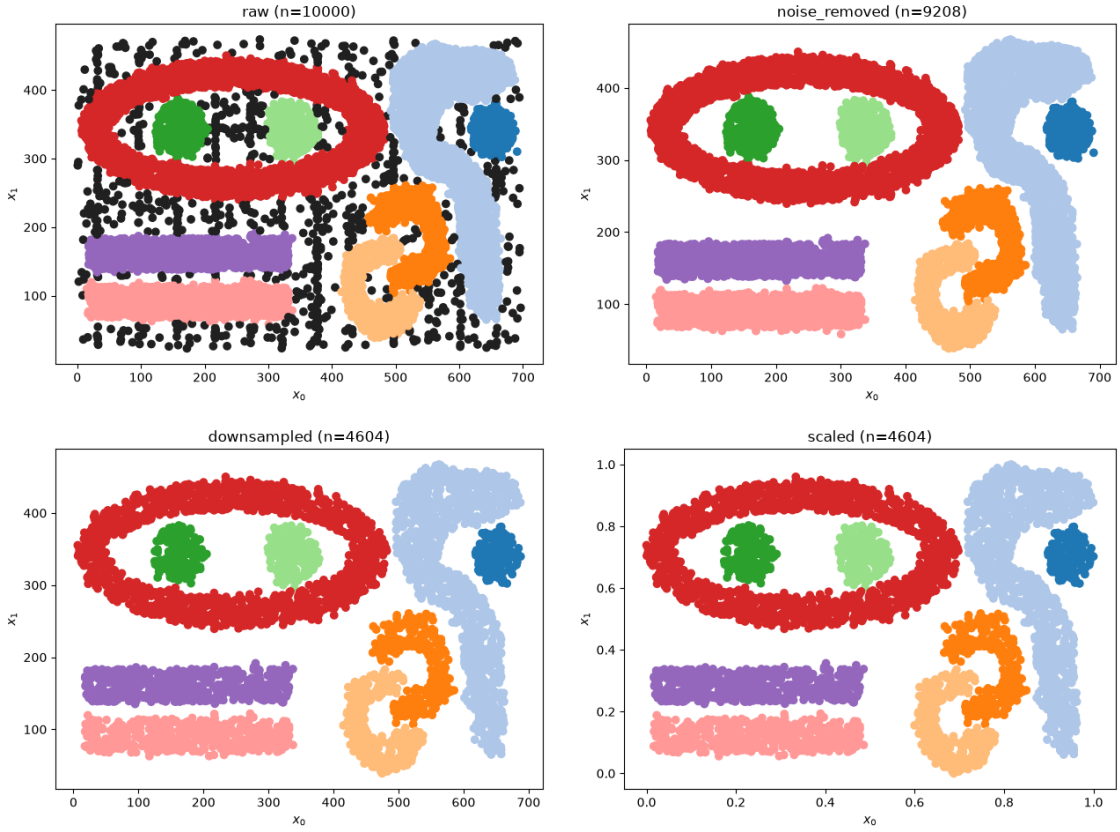


Figure 3.2: DS3 raw, without its noise, downsampled to half of its samples, and scaled to $[0, 1]$, from left to right and top to bottom. Noise samples are black.

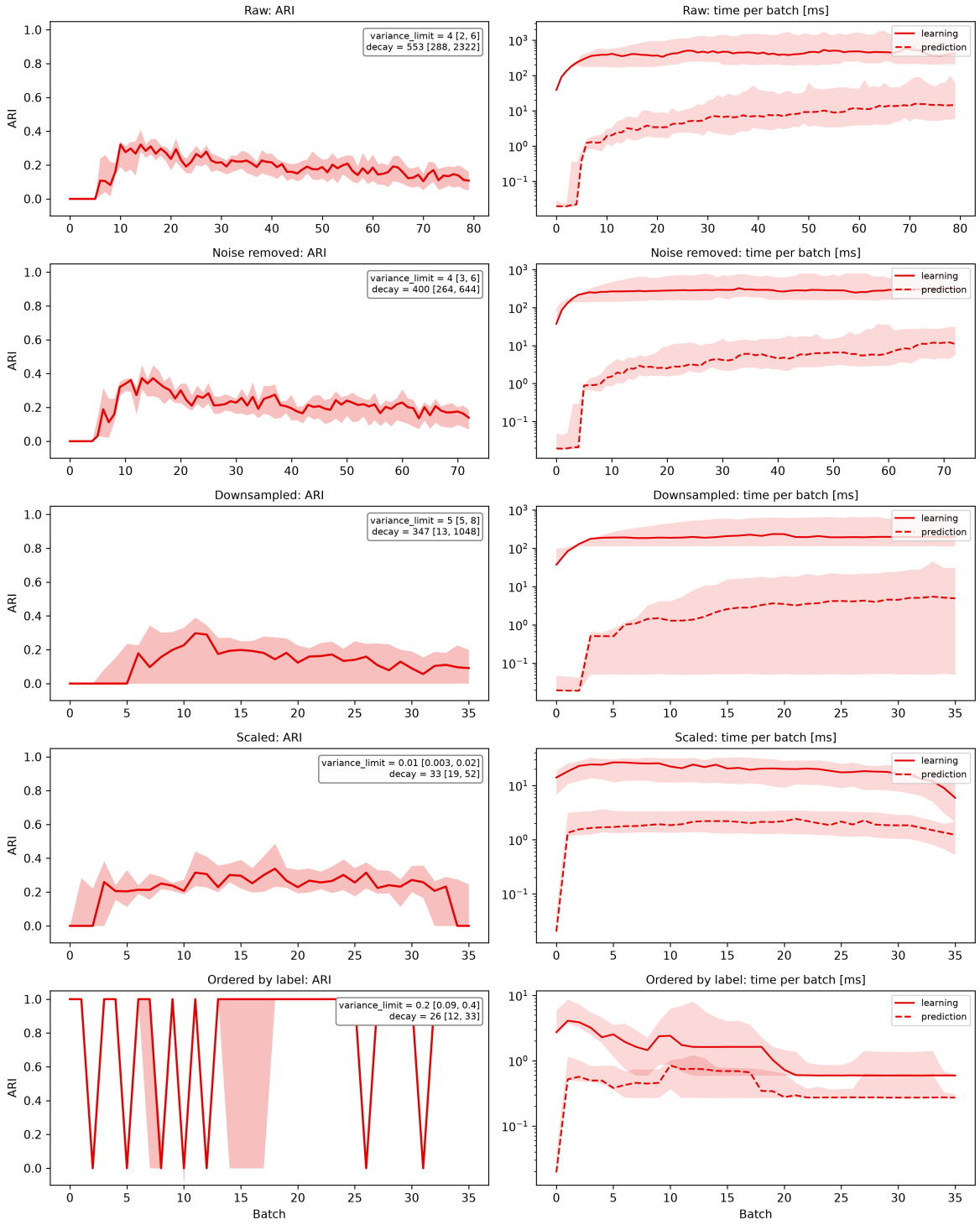


Figure 3.3: ARI and execution time per batch of MicroTEDAClus on DS3 after each preprocessing step, with the hyperparameters selected for it: medians and interquartile bands over 94 runs.

Scaling. Min-max scaling reduces the execution time by an order of magnitude and moves the variance limit that the search selects from a median of about 5 to about 0.01. The model is therefore not robust to the choice of this hyperparameter, as the original work claims: the limit follows the scale of the data. The sensitivity study of Section 2.4 also finds the quality strongly dependent on it. Scaling is expected in any application, but the original work computes the minimum and the maximum of each feature over the whole dataset, which leaks the future of the stream.

Ordering. Ordering the samples by label is the most harmful step. It leaks the labels and discards the order of arrival, and it turns most 100-sample batches into batches of a single cluster (Figure 3.4). The ARI then reaches 1 in most batches, and the learning time falls below a millisecond per batch over most of the stream (Figure 3.3).

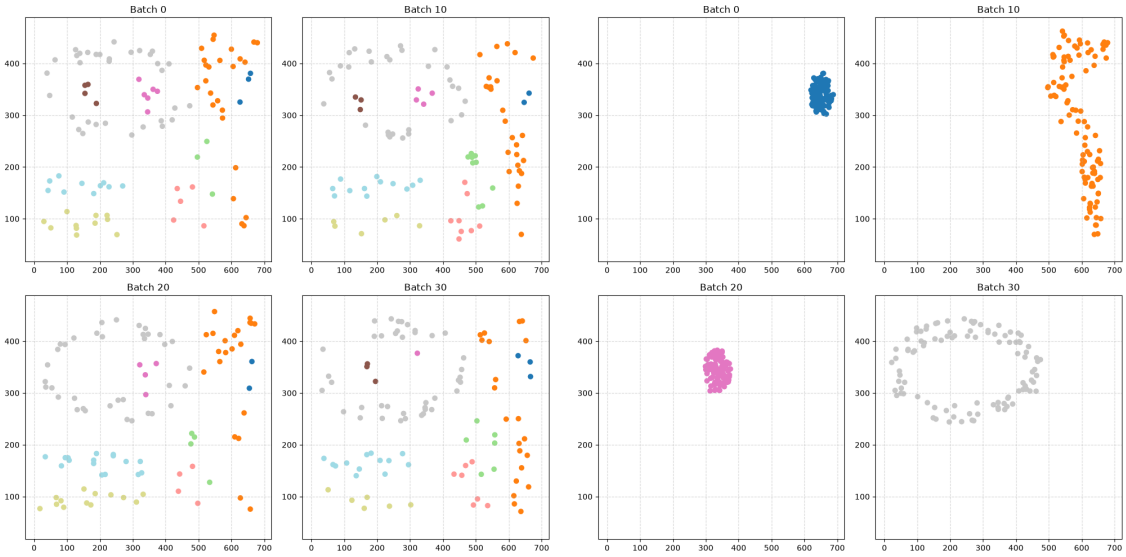


Figure 3.4: Batches 0, 10, 20 and 30 of the downsampled DS3, in the order of its file, at left, and ordered by label, at right.

The experiments of the original work on ST-D1 and ST-D2 therefore do not evaluate online stream clustering, and we use the raw DS3 and DS4 instead. The third static dataset, Cassini, comes from the `R` package `mlbench` without preprocessing, but ordered by label; we shuffle it (Figure 3.5).

3.3.2 Concept Drift Datasets

The streams with concept drift of the original work, StreamBenchmark(1), StreamBenchmark(2) and RandomRBFGeneratorEvents, come from `streamMOA`. Its plots suggest that their noise samples, labeled `NA` by the package, were removed. We keep them, labeled -1 (Figure 3.6).

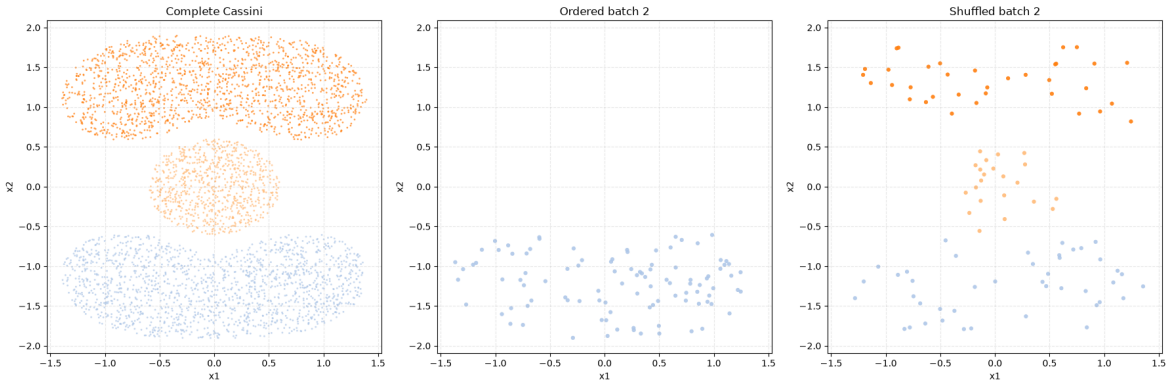


Figure 3.5: Cassini in full, at left, and its batch 2 in the order of the package, at middle, and shuffled, at right.

3.3.3 Scalability Streams

The scalability experiments of the original work, which profile memory and time as the number of samples and the dimension grow, use synthetic streams of `streamMOA` without noise. We generate them with noise (Figure 3.7).

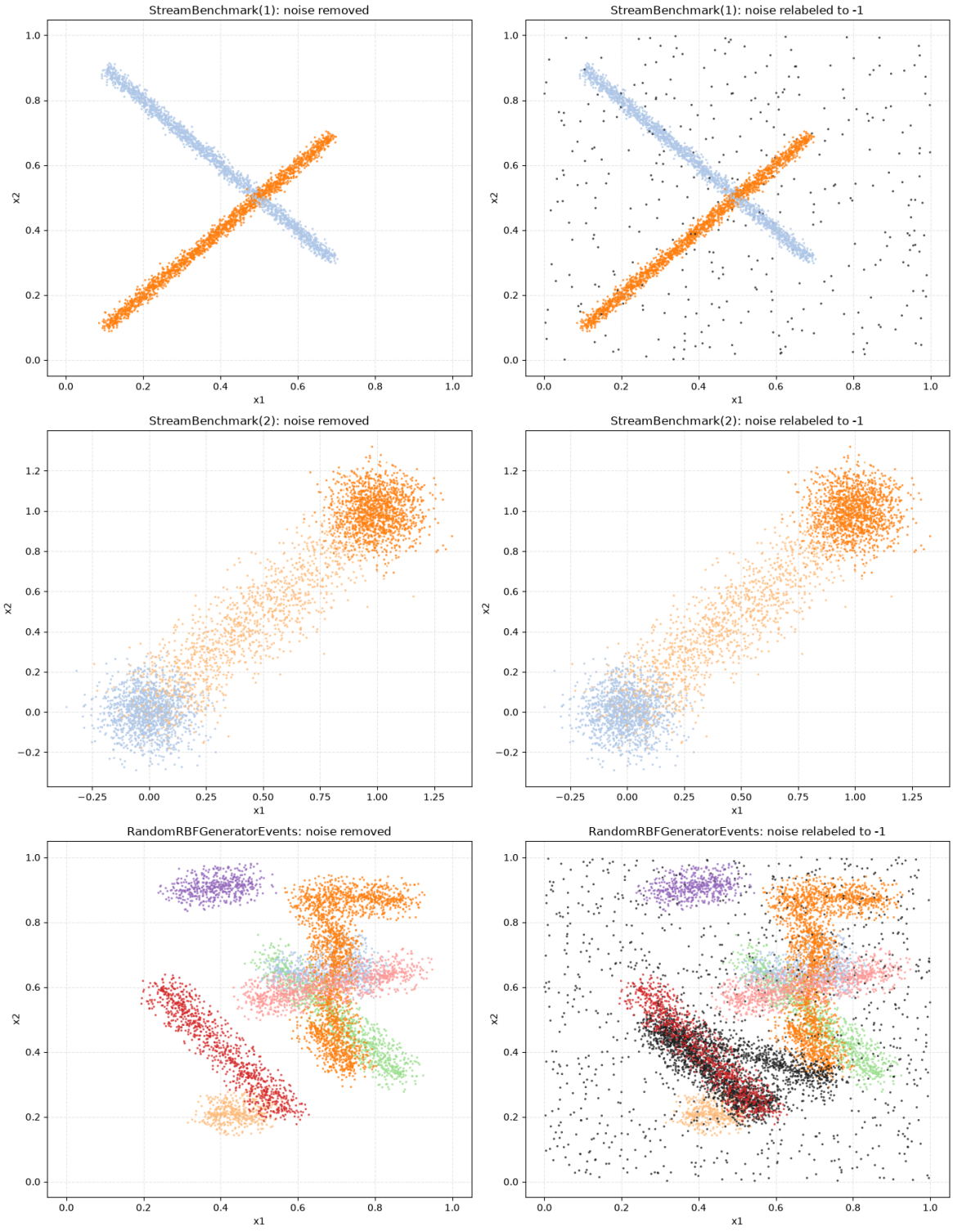


Figure 3.6: The streams with concept drift without their noise samples, at left, and with them, labeled -1, at right.

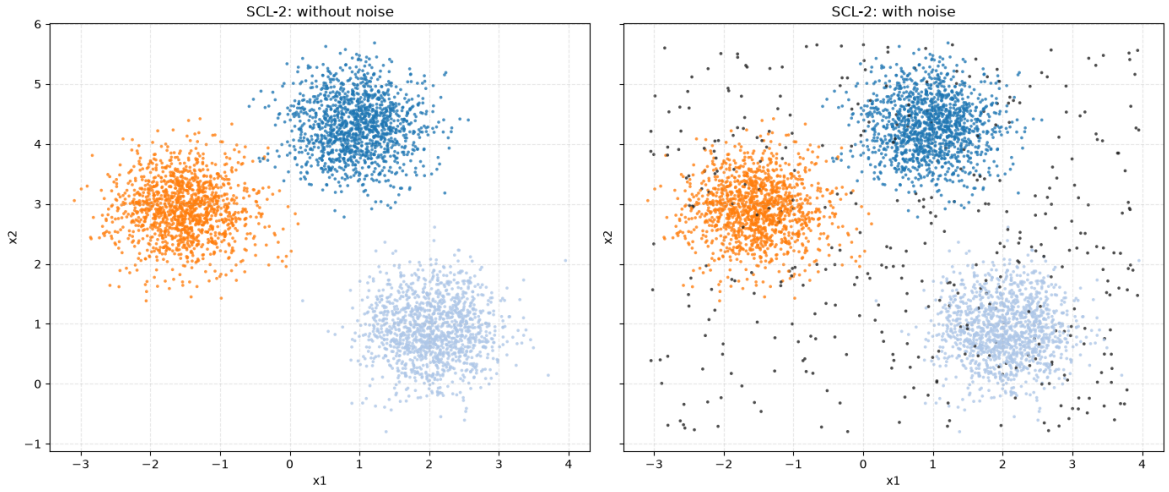


Figure 3.7: The scalability stream of two dimensions without noise, at left, and with noise, at right.

3.4 Evaluation Protocol

The original work states that it applies prequential evaluation, in which "each instance can be used to test before it is used for training" (Maia et al. [4, Section 5.2.1]). That is the standard practice for stream clustering [3, 19–22], in line with Dawid's view of inference as forecasting future observations [30]. Its code, however, implements no prequential evaluation.

For ST-D1 and ST-D2, it learns the whole dataset and then predicts the same samples (Listing 3.7). Even for offline clustering, that is not standard practice [8, 9, 31, 32], which prefers cross-validation [33, 34]. The evaluation of the `C` reimplementation of Section 2.4 follows the same protocol for all of its datasets. For the other datasets, a function named `prequential_evaluation` learns each batch before predicting it (Listing 3.8), the opposite of prequential evaluation.

The published curves confirm it without the code. Under prequential evaluation, the first batch is predicted before anything is learned, so its ARI should be near zero, yet it exceeds 0.5 in every curve of the original work and 0.8 in ten of its fifteen curves

```
X = stream_df[['x', 'y']].values
y = stream_df['class'].values
evol_model = EvolvingClustering.EvolvingClustering(variance_limit=0.0006,
    debug=False)
evol_model.fit(X)
y_hat = evol_model.predict(X)
error = adjusted_rand_score(y, y_hat)
```

Listing 3.7: Evaluation of the static datasets in the original work

```
if elapsed_time:
    time_start = time.time()
method.fit(train_data)
y_hat = method.predict(test_data)
y = labels[test_start:test_end]
```

Listing 3.8: Excerpt of the function named `prequential_evaluation` in the original work

(Figure 3.8). Under the evaluation of the original work, the best model is the one that memorizes the label of each position [35, 36], which does not generalize to unseen samples.

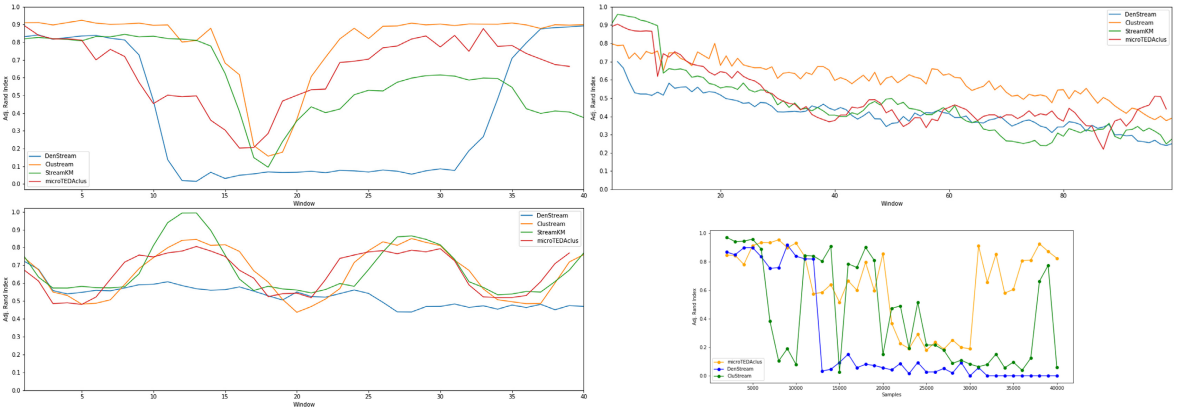


Figure 3.8: ARI per batch reported in the original work: the streams with concept drift StreamBenchmark(1), StreamBenchmark(2) and RandomRBFGeneratorEvents, and, at bottom right, the memory scalability experiment.

Every experiment of this thesis uses prequential evaluation: each 100-sample batch is predicted before the model learns it, and the predicted labels are compared with the true ones.

3.5 Metrics

The original work evaluates clustering quality by the adjusted Rand index (ARI) [37], an external index that depends neither on the distribution of the data nor on the number of clusters [32]. It summarizes the ARI of a run by its mean and standard deviation over the runs, which hides how the quality evolves along a stream: Figure 3.9 shows three curves with the same mean and standard deviation, one that improves slowly, one that peaks and degrades, and one that is random. We summarize the ARI of a run by two metrics:

- the **Overall ARI**, the median of the ARI of the batches of the evaluation segment;

- the **Steady-State ARI**, the ARI of the penultimate batch, which measures the quality after the stream has run its course and any concept drift has occurred. The last batch is avoided because it may be incomplete.

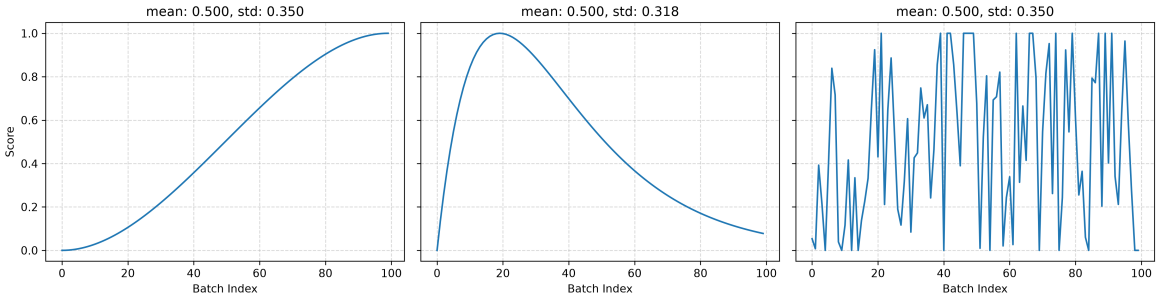


Figure 3.9: Three hypothetical evolutions of the ARI over the batches, with the same mean and standard deviation: a slow improvement, a peak followed by degradation, and a random curve.

The original work reports the execution time only for its scalability streams, as the mean time per sample against the dimension, and the memory only for one run of StreamBenchmark(1). We measure the cost of every model on every dataset:

- the **execution time** of the learning and of the prediction stages, each summed over the batches of the evaluation segment, from the wall clock inside the task. Section 4.2.2 measures the time with each task executed alone, which the modernized experiment adopts (Section 5.4);

- the **memory footprint**, the size of the model object after each batch, at its maximum and at the penultimate batch. The modernized experiment measures memory differently, and its values are not comparable with these (Section 5.4);

- the **microcluster count**, at its maximum and at the penultimate batch, a proxy for the memory that depends less on the implementation. StreamKM keeps no microclusters and is left out of it.

For the scalability streams, we report the quality per dimension and plot the cost against the dimension.

3.6 Hyperparameter Selection and Runs

3.6.1 Hyperparameter Tuning

Every model has hyperparameters that can be tuned, even those whose papers use default values. So that no model is disadvantaged by its hyperparameters, every run begins with a search on the first 20% of its stream, by Bayesian optimization [38], and the tuned model is evaluated on the remaining 80%, which the search never sees [8, 9]. The original work, in contrast, selects the parameters that maximize the score over the whole dataset. The search maximizes the objective of Algorithm 4: the ARI of a prequential execution on the tuning segment, without its first 5% of predictions, less a penalty of $\lambda = 10^{-3}$ per second of execution.

Algorithm 4 Objective of the hyperparameter search

Require: the tuning segment (X_{opt}, y_{opt}), the batch size, the model class and its search space, the current trial
Ensure: the score that the search maximizes
1: Draw the hyperparameters of the trial from the search space
2: Build the model with them
3: Execute it prequentially on the tuning segment, measuring the execution time t_{total}
4: Drop the first 5% of the predictions
5: Compute the ARI ari between the remaining predictions and their labels
6: **return** $ari - \lambda\, t_{total}$

The search of each model ends at 100 trials or at 300 seconds, whichever comes first. The penalty and the time limit avoid configurations that buy a small gain in ARI with a large cost, and they keep the experiments feasible: some models are slow enough to make an unrestricted search impossible over the runs that the statistics require, so a slow model may complete fewer trials than a fast one. For the same reason, every dataset has at most 10,000 samples. Section 4.6 shows the limits of this search, and the modernized experiment replaces its ranges by rules, lengthens its streams and drops its penalty (Sections 5.3.1 and 5.3.2).

3.6.2 Multi-run Statistics

A run generates its stream from its seed, tunes the model and evaluates it. We aggregate the runs by the median and the interquartile range, and not by the mean and the standard deviation, because the search may end in a poor local optimum and produce outlier runs, to which the median is robust.

We take $N = 1000$ runs as a stable reference, and choose the number of runs k^* that approximates it at the point beyond which more runs bring diminishing returns, Equa-

tion (3.2), which the Kneedle algorithm [39] finds as the point of maximum distance to the secant joining the ends of the min–max normalized curve

$$k^* = \operatorname*{argmax}_k \frac{|(\tilde{k}_{\max} - \tilde{k}_{\min})(\tilde{Y}(k) - \tilde{Y}_{\min}) - (\tilde{Y}_{\max} - \tilde{Y}_{\min})(\tilde{k}(k) - \tilde{k}_{\min})|}{\sqrt{(\tilde{k}_{\max} - \tilde{k}_{\min})^2 + (\tilde{Y}_{\max} - \tilde{Y}_{\min})^2}}, \tag{3.2}$$

$$\tilde{k}(k) = \frac{k - k_{\min}}{k_{\max} - k_{\min}}, \qquad \tilde{Y}(k) = \frac{\hat{Y}_{\text{med}}(k) - \hat{Y}_{\text{med}}(k_{\min})}{\hat{Y}_{\text{med}}(k_{\max}) - \hat{Y}_{\text{med}}(k_{\min})}, \tag{3.3}$$

where $\hat{Y}_{\text{med}}$ is a non-increasing isotonic regression [40] of the worst-case convergence curve of the median

$$\begin{gathered} Y_{\text{med}}(k) = \max_{c \in \mathcal{C}} \text{NMAE}_c^{\text{med}}(k), \\ \mathcal{C} = \{SCL - 2, StreamBenchmark(1), StreamBenchmark(2)\} \\ \times \{\text{ARI}, \text{time}, \text{memory}, \text{microcluster count}\}. \end{gathered} \tag{3.4}$$

For a configuration $c \in \mathcal{C}$ and a subset size $k \in \{k_{\min}, \ldots, N\}$, let $S_{c,k,1}, \ldots, S_{c,k,B}$ be B independent draws, uniform over the size-k subsets of $\{1, \ldots, N\}$, with subsample statistics

$$m_{c,k,b} = \text{median}(\mathcal{D}_c[S_{c,k,b}]), \qquad q_{c,k,b} = Q_3(\mathcal{D}_c[S_{c,k,b}]) - Q_1(\mathcal{D}_c[S_{c,k,b}]), \qquad b = 1, \ldots, B. \tag{3.5}$$

The normalized mean absolute errors

$$\text{NMAE}_c^{\text{med}}(k) = \frac{1}{s_c B} \sum_{b=1}^{B} |m_{c,k,b} - \tilde{m}_c|, \qquad \text{NMAE}_c^{\text{iqr}}(k) = \frac{1}{s_c B} \sum_{b=1}^{B} |q_{c,k,b} - \tilde{q}_c| \tag{3.6}$$

are Monte Carlo estimates [41], with the reference statistics and scale

$$\tilde{m}_c = \text{median}(\mathcal{D}_c), \qquad \tilde{q}_c = Q_3(\mathcal{D}_c) - Q_1(\mathcal{D}_c), \qquad s_c = \tilde{q}_c, \tag{3.7}$$

where $\mathcal{D}_c = \{x_{c,i}\}_{i=1}^N$ holds the value of the metric of configuration c in each run i, taken at the middle batch of the stream. With $N = 1000$, $B = 10^4$ and $k_{\min} = 3$, $k^* = 94$ (Figure 3.10).

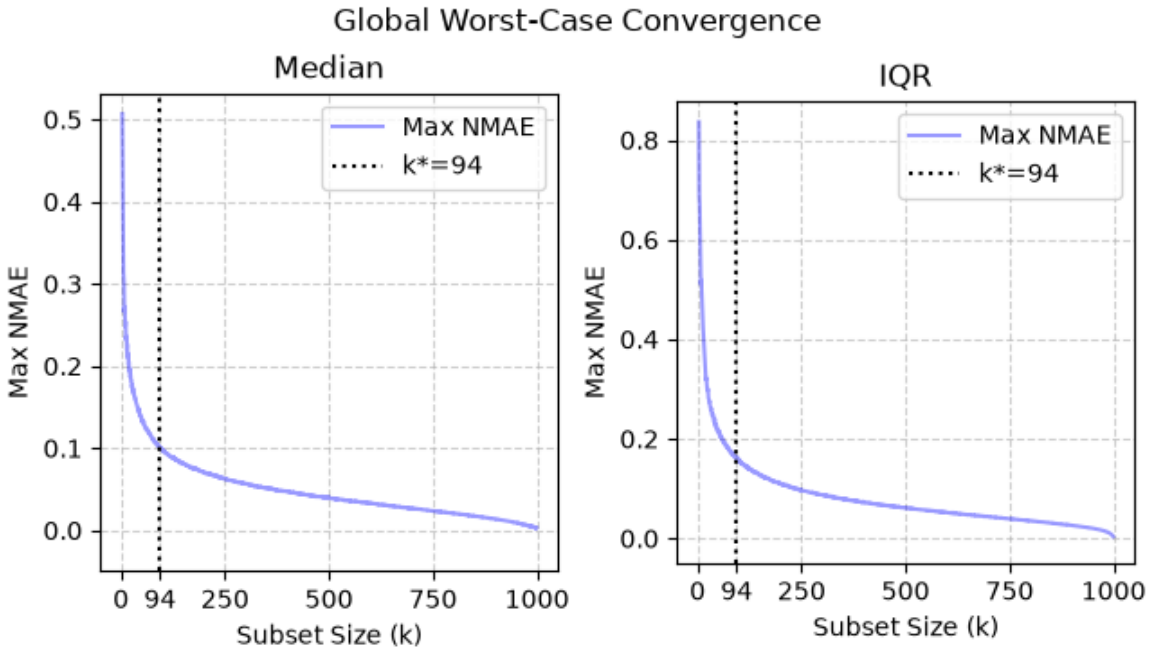


Figure 3.10: Maximum normalized mean absolute error of the median, at left, and of the interquartile range, at right, against the number of runs k. The dashed line marks k^*.

3.7 Statistical Comparison

Before any test, we discretize the metrics to thresholds of practical relevance, so that floating-point noise and hardware jitter do not inflate the power of the rank-based tests:

$$\begin{aligned} \mathrm{ARI}_{\mathrm{disc}} &= 10^{-3}\lfloor 10^3 \cdot \mathrm{ARI} \rceil, & \mathrm{Time}_{\mathrm{disc}}\,[\mathrm{ms}] &= 10\lfloor \mathrm{Time}\,[\mathrm{ms}]\,/10 \rceil, \\ \mathrm{Memory}_{\mathrm{disc}}\,[\mathrm{B}] &= 1024\lfloor \mathrm{Memory}\,[\mathrm{B}]\,/1024 \rceil, & \mathrm{MC}_{\mathrm{disc}} &= \lfloor \mathrm{MC} \rceil. \end{aligned} \tag{3.8}$$

Every comparison of the thesis sets one model against references, as Demšar recommends when a model is compared with others [42]. Two kinds occur:

- **The model in focus against each other model**, in this chapter and whenever a model is compared with the benchmark models. In each dataset and metric, the model in focus is compared with each other model by the paired Wilcoxon signed-rank test [43] over the runs that every model finished, and the p-values of these comparisons are adjusted by the Holm–Bonferroni procedure [44].
- **A variant against its control**, in the improvement of Chapter 4. In each metric, the variant is compared with its control by the same test in each dataset,

over the runs that both finished, and the p-values of the datasets are adjusted together.

With 94 runs, the Wilcoxon test rejects the null hypothesis for differences of no practical consequence, so a difference counts only when it is also large enough. A model m_i is distinguishable from its reference m_{ref} if

$$D(m_i, m_{\text{ref}}) \iff (p_{\text{adj}} < \alpha) \wedge (|d| \geq 0.147), \tag{3.9}$$

where $\alpha = 0.05$, p_{adj} is the adjusted p-value, d is Cliff's delta [45], and $|d| \geq 0.147$ is the boundary of a non-negligible effect [46].

Reading the tables. The tables give medians over the runs. In a comparison with the benchmark models, the model in focus comes first, followed by a row that counts the models it is significantly better than, indistinguishable from and worse than. The median of another model is in bold where the model in focus is significantly better than it, and in red where the model in focus is significantly worse. In the tables of a variant and its control, the value of the variant is in bold where it is significantly better than its control, and in red where it is significantly worse. Higher is better for the ARI; lower is better for the execution time, the memory and the size of a summary.

3.8 MicroTEDAClus Under the Reassessed Methodology

We evaluate MicroTEDAClus against the six benchmark models of Section 3.2 on the six datasets and the seven scalability streams of Section 3.3, over 94 runs, under the methodology of the previous sections.

3.8.1 Clustering Quality

The results do not support the claims of the original work (Table 3.3). Against its benchmark models, MicroTEDAClus is not on par: in the Overall ARI, it is worse than CluStream and StreamKM in the three static datasets and worse than DenStream in StreamBenchmark(1), and it is better than DenStream only in StreamBenchmark(2) and DS3, than StreamKM only in StreamBenchmark(2) and than CluStream only in RandomRBFGeneratorEvents. Nor does it adapt better to concept drift. At the penultimate batch, it is better than none of the three in any dataset and worse than StreamKM in all six. In StreamBenchmark(1) and RandomRBFGeneratorEvents, two

Table 3.3: Clustering quality of MicroTEDAClus and the benchmark models: medians over the runs of the main experiment.

Model	B1	B2	RBF	CSN	DS3	DS4
Overall ARI						
MicroTEDAClus	0.035	0.564	0.359	0.492	0.215	0.202
Better/same/worse	0/3/3	5/1/0	2/3/1	3/1/2	4/0/2	3/1/2
DenStream	0.665	**0.402**	0.360	0.383	**0.174**	0.184
CluStream	0.096	0.566	**0.170**	0.573	0.324	0.319
StreamKM	0.183	**0.498**	0.371	0.696	0.330	0.327
AutoCloud	0.043	**0.001**	**0.051**	**0.076**	**0.023**	**0.030**
CEDAS	0.252	**0.000**	0.178	**0.000**	**0.000**	**0.000**
TEDACloud	0.827	**0.509**	0.495	**0.360**	**0.040**	**0.125**
Steady-State ARI						
MicroTEDAClus	0.000	0.524	0.000	0.436	0.126	0.124
Better/same/worse	0/1/5	2/2/2	0/3/3	2/2/2	2/1/3	2/1/3
DenStream	0.822	0.567	0.340	0.405	0.196	0.274
CluStream	0.174	0.599	0.194	0.589	0.338	0.320
StreamKM	0.164	0.860	0.451	0.569	0.341	0.301
AutoCloud	0.020	**0.135**	0.040	**0.052**	**0.073**	**0.070**
CEDAS	0.211	**0.000**	0.142	**0.000**	**0.000**	**0.000**
TEDACloud	0.760	0.518	0.541	0.366	0.117	0.091

of the streams with concept drift, its median Steady-State ARI is zero, and it is worse than five and three of the six benchmark models, respectively. In the Overall ARI, it is better than most benchmark models only in StreamBenchmark(2) and DS3, mainly the TEDA-based models AutoCloud, CEDAS and TEDACloud, which it beats in every static dataset.

In the scalability streams, the Overall ARI of MicroTEDAClus stays high at every dimension, but its Steady-State ARI falls with 32 dimensions and reaches zero with 64 (Table 3.4). Along the stream with 64 dimensions, its ARI collapses before the end (Figure 3.11), its microcluster count falls (Figure 3.12), and Table 4.5 shows that the *vectorized* model, which executes the same algorithm, ends most runs of the streams with 32 and 64 dimensions predicting a single cluster. We trace the collapse to the dimensionality penalty of Section 2.2.1. From eight dimensions on, it is worse than CluStream, StreamKM and CEDAS in both metrics.

3.8.2 Cost

MicroTEDAClus is slower than every benchmark model but AutoCloud in the learning stage of every dataset, and slower than StreamKM and CEDAS in the prediction stage of every dataset (Table 3.5). Its maximum memory footprint is larger than those of

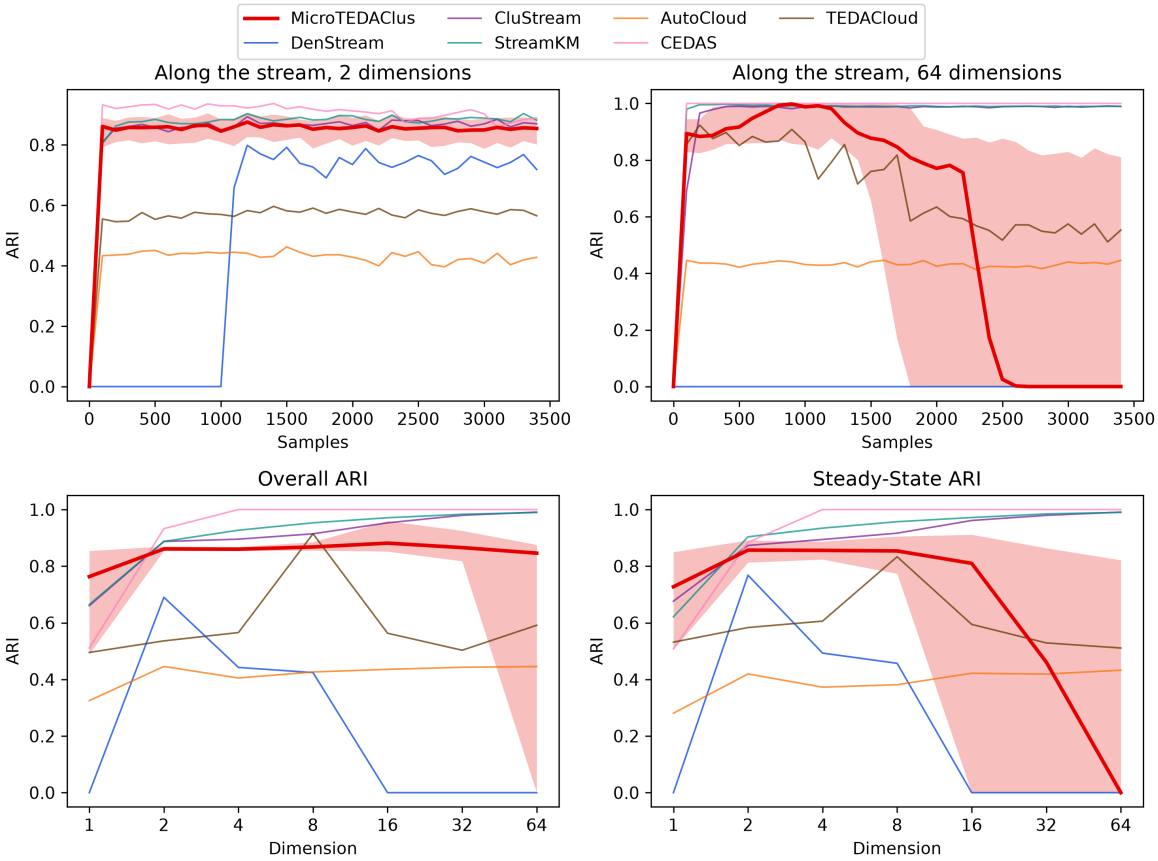


Figure 3.11: ARI of MicroTEDAClus and the benchmark models in the scalability streams: along the stream with 2 and 64 dimensions, and the Overall and Steady-State ARI against the dimension. Medians, with the interquartile band of MicroTEDAClus, over 94 runs.

Table 3.4: Clustering quality of MicroTEDAClus and the benchmark models in the scalability streams: medians over the runs of the main experiment.

Model	S1	S2	S4	S8	S16	S32	S64
Overall ARI							
MicroTEDAClus	0.763	0.861	0.860	0.867	0.883	0.865	0.843
Better/same/worse	4/2/0	3/3/0	2/2/2	2/1/3	2/1/3	2/1/3	2/1/3
DenStream	**0.000**	**0.690**	**0.442**	**0.426**	**0.000**	**0.000**	**0.000**
CluStream	0.661	0.887	0.895	0.913	0.953	0.980	0.991
StreamKM	0.665	0.887	0.927	0.953	0.971	0.983	0.989
AutoCloud	**0.325**	**0.446**	**0.405**	**0.426**	**0.435**	**0.443**	**0.445**
CEDAS	**0.511**	0.932	1.000	1.000	1.000	1.000	1.000
TEDACloud	**0.495**	**0.536**	0.565	0.924	0.567	0.503	0.586
Steady-State ARI							
MicroTEDAClus	0.727	0.857	0.856	0.847	0.810	0.462	0.000
Better/same/worse	4/2/0	3/2/1	2/1/3	2/1/3	2/1/3	2/1/3	1/0/5
DenStream	**0.000**	**0.768**	**0.493**	**0.454**	**0.000**	**0.000**	**0.000**
CluStream	0.677	0.873	0.894	0.916	0.960	0.980	0.991
StreamKM	0.621	0.903	0.934	0.957	0.972	0.985	0.990
AutoCloud	**0.280**	**0.419**	**0.372**	**0.379**	**0.421**	**0.419**	0.433
CEDAS	**0.508**	0.885	1.000	1.000	1.000	1.000	1.000
TEDACloud	**0.532**	**0.583**	0.606	0.903	0.598	0.529	0.510

DenStream, StreamKM, CEDAS and TEDACloud in every dataset, and larger than that of CluStream in the Chameleon datasets. These comparisons measure implementations as much as algorithms, and the times are measured inside the tasks, with other tasks running on the same machine.

In the scalability streams, the steady-state memory footprint and microcluster count of MicroTEDAClus fall as the dimension grows beyond eight (Figure 3.12): the cost falls with the collapse of the microclusters, not with an efficient summary.

3.8.3 Microcluster Count

MicroTEDAClus keeps more microclusters than DenStream, CEDAS and TEDACloud in every dataset (Table 3.6). In the Chameleon datasets it keeps several hundred, more than CluStream, while in the other datasets it keeps no more than CluStream.

Table 3.5: Execution time and memory footprint of MicroTEDAClus and the benchmark models: medians over the runs of the main experiment.

Model	B1	B2	RBF	CSN	DS3	DS4
Learning time [ms]						
MicroTEDAClus	1000	570	8210	590	27850	18815
Better/same/worse	0/1/5	0/1/5	0/1/5	1/0/5	0/0/6	1/0/5
DenStream	180	200	200	190	210	210
CluStream	230	140	4260	120	380	375
StreamKM	60	30	120	25	120	105
AutoCloud	12910	50	460	**16910**	445	**97150**
CEDAS	30	20	90	30	90	70
TEDACloud	110	50	530	270	360	310
Prediction time [ms]						
MicroTEDAClus	100	50	800	60	480	445
Better/same/worse	1/2/3	2/0/4	0/2/4	3/1/2	0/0/6	1/0/5
DenStream	90	**140**	150	**135**	70	110
CluStream	100	**80**	1220	**80**	260	200
StreamKM	10	10	30	10	30	30
AutoCloud	**4770**	20	160	**6200**	160	**35680**
CEDAS	20	10	60	20	70	50
TEDACloud	40	20	150	70	110	100
Maximum memory footprint [KiB]						
MicroTEDAClus	277	141	1030	140	702	628
Better/same/worse	0/2/4	0/2/4	0/2/4	0/2/4	0/0/6	1/0/5
DenStream	15	28	10	24	10	10
CluStream	209	152	1066	148	218	201
StreamKM	27	28	84	31	35	38
AutoCloud	654	3	11	706	9	**1941**
CEDAS	8	3	9	5	8	8
TEDACloud	6	3	11	12	7	7
Steady-state memory footprint [KiB]						
MicroTEDAClus	155	106	423	118	698	626
Better/same/worse	0/2/4	1/1/4	0/2/4	1/1/4	0/0/6	1/0/5
DenStream	12	23	9	18	6	9
CluStream	209	**152**	1066	**148**	218	201
StreamKM	16	17	31	17	20	28
AutoCloud	629	2	9	685	6	**1914**
CEDAS	7	3	8	5	8	8
TEDACloud	6	2	9	8	3	4

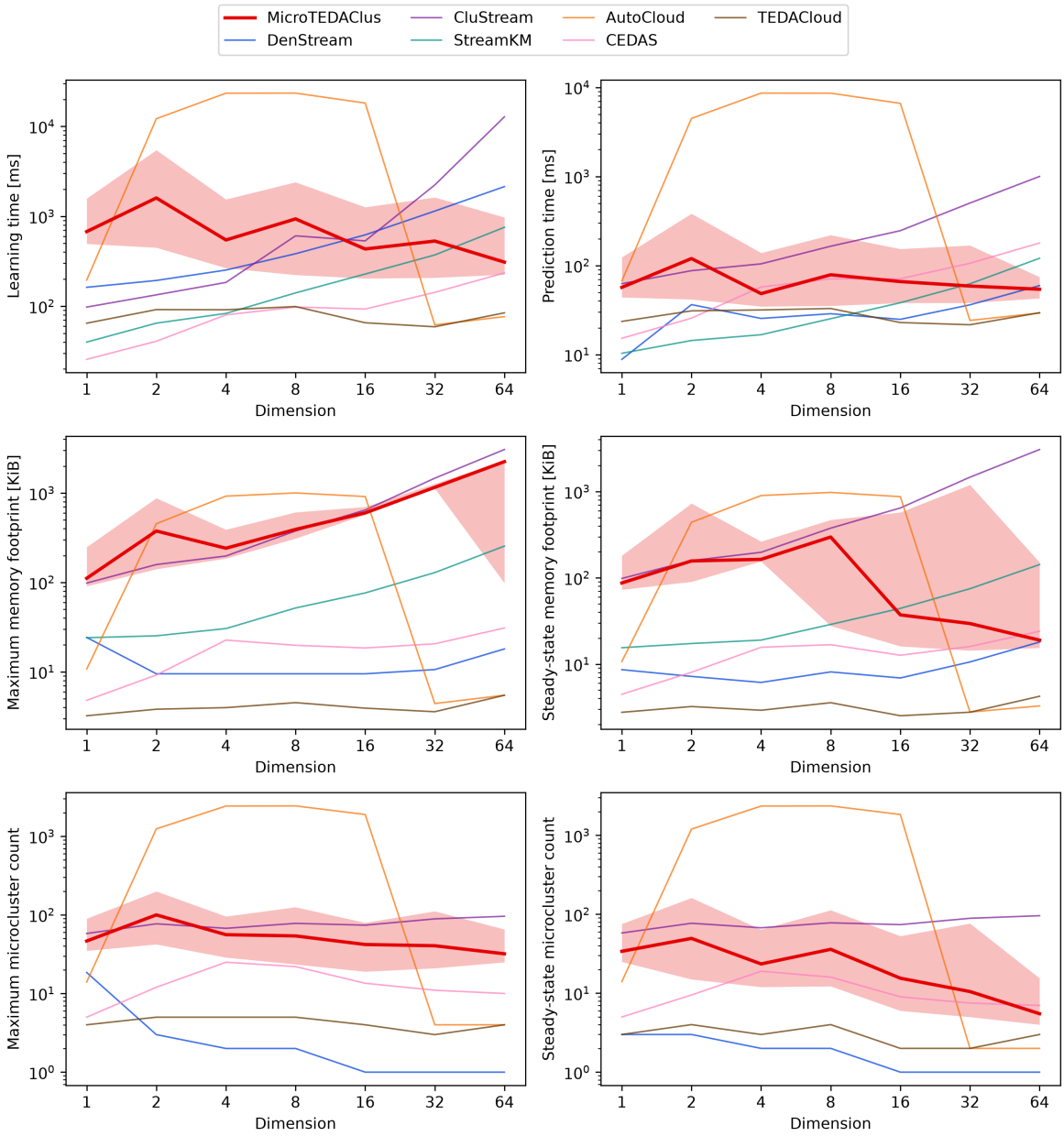


Figure 3.12: Execution time, memory footprint and microcluster count of MicroTEDAClus and the benchmark models in the scalability streams, against the dimension. Medians, with the interquartile band of MicroTEDAClus, over 94 runs.

Table 3.6: Microcluster count of MicroTEDAClus and the benchmark models: medians over the runs of the main experiment. StreamKM keeps no microclusters.

Model	B1	B2	RBF	CSN	DS3	DS4
Maximum microcluster count						
MicroTEDAClus	82	45	215	44	509	430
Better/same/worse	0/2/3	1/1/3	1/1/3	2/0/3	0/0/5	1/0/4
DenStream	7	17	4	14	2	4
CluStream	103	**74**	**530**	**71**	105	97
AutoCloud	1165	3	14	**1882**	12	**5345**
CEDAS	9	2	11	6	10	10
TEDACloud	8	3	14	16	10	9
Steady-state microcluster count						
MicroTEDAClus	42	30	102	34	506	426
Better/same/worse	2/0/3	1/1/3	1/1/3	2/0/3	0/0/5	1/0/4
DenStream	6	12	4	10	2	4
CluStream	**103**	**74**	**530**	**71**	105	97
AutoCloud	**1081**	2	13	**1822**	8	**5266**
CEDAS	8	2	9	5	10	10
TEDACloud	7	2	12	11	4	5

3.9 Hyperparameters of MicroTEDAClus

Table 3.7: Hyperparameters selected for MicroTEDAClus: median [QCD] over the runs of the main experiment.

Model	Hyperparameter	B1	B2	RBF	CSN	DS3	DS4
MicroTEDAClus	variance limit	0.00602 [0.93]	0.00838 [0.98]	0.00093 [0.59]	0.105 [0.44]	4.86 [0.23]	6.86 [0.59]
	decay	363 [0.87]	32.5 [0.87]	1040 [0.94]	35 [0.57]	422 [0.51]	367 [0.54]

The variance limit selected for MicroTEDAClus spans four orders of magnitude across the datasets, from about 10^{-3} in RandomRBFGeneratorEvents to about 7 in DS4, following the scale of each dataset (Table 3.7). With the shift that scaling produces in Section 3.3.1, it contradicts the claim of the original work that the model is robust to this hyperparameter. The dispersion of the selected decay, measured by the quartile coefficient of dispersion, is higher in the three streams with concept drift than in the static datasets.

3.10 Summary

The published results of MicroTEDAClus come from an implementation that departs from its formulation, from benchmark models executed through two layers of interface, from datasets cleaned, scaled with information from the whole stream and ordered by

label, and from an evaluation that learns each batch before predicting it. Under a methodology without these flaws, MicroTEDAClus is not on par with the benchmark models of the original work, it does not adapt better to concept drift, it learns slowly and keeps a large summary, its quality collapses in high dimensions, and its variance limit depends on the scale of the data. Chapter 4 addresses these weaknesses.

Chapter 4

Model Improvement

Chapters 2 and 3 identify issues in the formulation of MicroTEDAClus (Section 2.2.1) and undocumented behaviors of its public implementation (Section 3.1). In this chapter, we act on them in three stages. We first optimize the implementation, whose execution time made iterative experimentation unaffordable. We then evaluate corrections of the flaws, and finally modifications beyond the original formulation, which define MicroTEDAClus+. We close the chapter by comparing MicroTEDAClus+ with MicroTEDAClus and the benchmark models in the main experiment (Section 4.8). Section 4.1 describes how we run the experiments and judge their outcomes.

4.1 Experimental Design

4.1.1 Variants and Controls

An experiment compares a *variant* of the model with its *control*. The variant differs from the control by a stated combination of changes, so any difference between their results is attributable to those changes. To make this exact, we implement every variant as a configuration of a single implementation, in which each design choice under study is a switch. With every switch in its default position, that implementation executes the original algorithm.

4.1.2 Protocol and Datasets

We tune and evaluate every variant by the protocol of the main experiment, with the metrics of Section 3.5. We use eight streams: the six datasets of the main experiment and its two scalability streams of highest dimension, SCL-32 and SCL-64, because the six datasets are two-dimensional and several properties of the model only show in higher dimensions. The tables use the abbreviations of Table 3.2.

4.1.3 Development and Validation Runs

A run is identified by an index, which determines the seed of the stream generator and of the hyperparameter search. The main experiment uses the runs 0 to 93. In this chapter, we use the runs that follow them, so the streams of the main experiment take no part in the development of the model:

- **Development runs** (94 to 125, 32 runs): we execute every experiment that compares variants on these runs, and we made every design choice of this chapter from their results.
- **Validation runs** (126 to 219, 94 runs): we evaluate again on these runs the models accepted during development, and only those.

The separation exists because a model that is chosen for scoring best on a set of runs is expected to score somewhat worse on other runs. The streams of the validation runs come from seeds that no development run uses, and we execute the validation runs once, after we fix the accepted models, so their streams take no part in the choice of the models they measure. Their number is the one that Section 3.6.2 establishes for the main experiment. The two Chameleon datasets are an exception: they are fixed files, so every run processes the same stream and their runs differ only in the seed of the hyperparameter search. For them, the validation repeats the search and not the stream.

4.1.4 Comparisons and Decision Rule

We compare a variant with its control in each dataset and metric, over the runs they share, by the test of Section 3.7, whose conventions the tables follow, and decide the outcome on two metrics, the Overall and the Steady-State ARI. A variant

- **improves** its control if it is significantly better in at least one dataset and significantly worse in none;
- **regresses** if it is significantly worse in at least one dataset and significantly better in none;
- is **mixed** if both happen, and **indistinguishable** if neither does.

Only a variant that improves its control can replace it. The microcluster count and the memory footprint never decide an outcome; Appendix A gives them for the experiments whose cost this chapter discusses.

4.2 Implementation Optimization

The reference implementation stores each microcluster as a dictionary in a Python list. For every sample, it iterates over that list and updates one microcluster at a time; it maintains the intersection graph edge by edge in a graph library, and it computes the membership of each sample to each macrocluster in nested loops. Its execution time is dominated by the interpreter, not by the arithmetic of the algorithm. The reimplementation of Section 2.4 reduces it by changing the language. We keep `Python`, the language of the benchmark models, and change how the algorithm is computed.

The optimized implementation, hereafter the *vectorized* model, stores the population of microclusters as a structure of arrays, with one array per attribute. The algorithm has a part that depends on the order of arrival of the samples: the recursive update of the microclusters, the outlier test, the vitality update and the pruning. This part cannot be vectorized across samples, so it runs in a single compiled kernel that iterates over the samples of a batch and, for each of them, over the arrays. Everything else is expressed as array operations. At each macrocluster update, the intersection graph is rebuilt from the matrix of pairwise distances between the microcluster means, its connected components are extracted by a sparse graph routine, and the membership of a whole batch is obtained from one distance matrix.

The *vectorized* model executes the algorithm of the reference implementation, but does not reproduce its output exactly. Besides the order of the floating-point operations, one behavior differs. The pruning routine of the reference implementation, shown in Listing 3.1, removes elements from the list it is iterating over, so the microcluster that follows a removed one is not visited in that pass. The *vectorized* model visits every microcluster. The same implementation serves as the engine of the experiments of this chapter: each design choice studied in the remainder of this chapter is an option of it, and its default options are the *vectorized* model.

4.2.1 Clustering Quality

Table 4.1 compares the *vectorized* model with the reference implementation, the *original* model of the tables of this chapter, each tuned and evaluated by the protocol on the development runs. By the decision rule, the *vectorized* model improves the *original* one: its Overall ARI is significantly higher in DS3, and it is not significantly lower in any dataset on either metric. The remaining medians differ slightly. The memory footprint, in Table A.1, is significantly smaller in the four datasets in which the models keep the most microclusters.

The two models do not receive the same search. The search ends at 100 trials or at

Table 4.1: Implementation optimization: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
original	0.000	0.532	0.335	0.457	0.206	0.220	0.869	0.857
vectorized	0.000	0.581	0.328	0.520	**0.230**	0.235	0.851	0.859
Steady-State ARI								
original	0.000	0.480	0.000	0.400	0.171	0.171	0.000	0.000
vectorized	0.000	0.541	0.000	0.458	0.176	0.209	0.000	0.000

Table 4.2: Hyperparameter search of the two implementations: medians over the development runs. The search ends at 100 trials or 300 seconds.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Completed trials								
original	100	100	64	100	17	32	100	100
vectorized	100	100	100	100	100	100	100	100
Duration of the search [s]								
original	29	37	302	89	303	303	72	81
vectorized	2	2	7	3	10	9	4	3

300 seconds, whichever comes first, and Table 4.2 shows that the reference implementation exhausts the time before the trials in RandomRBFGeneratorEvents, DS3 and DS4, the datasets in which it is slowest. In DS3, where it gains, the *vectorized* model is tuned with several times the trials of the reference implementation. The gain is consistent with this difference of budget, which the experiment does not separate from the difference of pruning behavior described above. The speed of an implementation is thus also a search budget.

The optimization therefore preserves the clustering quality of the reference implementation, and the *vectorized* model replaces it as the reference of the remaining experiments.

4.2.2 Execution Time

The durations of Table 4.2 do not measure the optimization alone. They are taken while other tasks run on the same machine, and the two models are tuned independently, so a difference between them mixes the implementation, the load of the machine and the hyperparameters that each search selected. We therefore measure the speedup separately.

Each implementation replays the evaluation segment of every development run with

the same hyperparameters: those tuned for the *original* model in that run and, in a second measurement, those tuned for the *vectorized* model. We execute each task alone, on identical processor cores, and run the compiled kernel once before timing it. We execute the whole set of tasks three times, and the time of a task is the minimum of its three executions.

Table 4.3: Execution time of the evaluation segment, learning plus prediction, of the two implementations with the same hyperparameters, and speedup of the vectorized one: median and quartiles over the 32 development runs. Each task runs alone, and its time is the smallest of three executions, which differ by 0.8% in the median.

Dataset	*original* [ms]	*vectorized* [ms]	Speedup
Hyperparameters tuned for the original model			
B1	613 (309, 3906)	35.1 (29.2, 90.9)	17.5 (10.6, 43.4)
B2	1080 (191, 1562)	40.7 (26.2, 47.3)	26.2 (7.3, 33.0)
RBF	17534 (4898, 37424)	300 (134, 612)	55.9 (35.6, 64.3)
CSN	1229 (1008, 1543)	43.2 (39.3, 46.0)	28.5 (25.2, 34.0)
DS3	9628 (9504, 23961)	178 (164, 347)	58.7 (57.2, 68.2)
DS4	9669 (8090, 16455)	166 (144, 236)	59.0 (56.7, 68.7)
S32	780 (445, 1858)	41.2 (34.2, 61.1)	18.9 (12.9, 29.6)
S64	828 (498, 1915)	45.4 (38.0, 68.0)	18.1 (13.1, 25.4)
All	2074 (732, 9510)	65.2 (39.3, 166)	33.1 (16.7, 57.4)
Hyperparameters tuned for the vectorized model			
B1	1092 (461, 2808)	43.4 (32.3, 71.9)	25.2 (14.3, 39.2)
B2	1126 (192, 1485)	42.2 (26.3, 46.3)	26.6 (7.3, 32.8)
RBF	14262 (2907, 30895)	261 (106, 524)	54.8 (27.8, 63.8)
CSN	1233 (1054, 1358)	42.8 (40.2, 46.2)	28.5 (26.2, 30.6)
DS3	9488 (9445, 9557)	166 (163, 176)	57.6 (56.1, 58.9)
DS4	9646 (8031, 15936)	165 (142, 230)	58.4 (57.0, 63.3)
S32	838 (471, 2147)	41.7 (34.7, 65.9)	20.1 (13.6, 33.9)
S64	972 (653, 2443)	47.9 (41.6, 91.7)	19.9 (15.6, 23.6)
All	2291 (919, 9447)	69.2 (41.6, 163)	32.5 (20.9, 57.0)

Table 4.3 verifies the speedup in both measurements, so it does not depend on which model selected the configuration, and the *vectorized* model is at least twice as fast as the *original* one in every run.

The speedup grows with the number of microclusters, as Figure 4.1 shows, and levels off when they pass about a hundred. It is therefore largest in RandomRBFGeneratorEvents, DS3 and DS4, whose tuned configurations keep the most microclusters, and smallest in StreamBenchmark(1), SCL-32 and SCL-64, whose configurations keep the fewest. This is the behavior expected from the two implementations as described above: the cost of the reference implementation is that of interpreted code executed for every microcluster and every sample, while the *vectorized* implementation has a cost

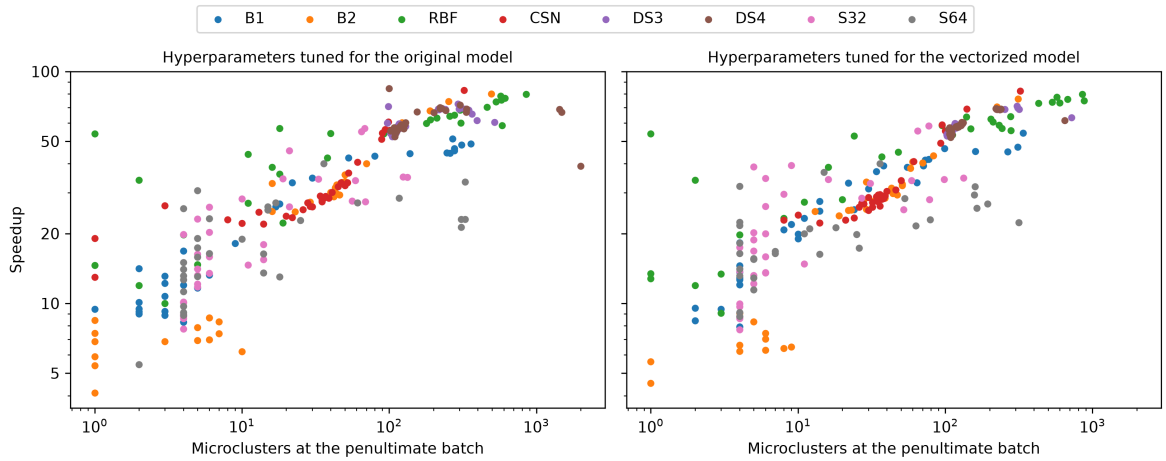


Figure 4.1: Speedup of the *vectorized* implementation in each development run against the number of microclusters that the configuration of the run keeps.

per batch that does not depend on the number of microclusters and dominates when they are few.

The reduction is what makes the remaining stages affordable: the experiments of this chapter execute more than sixteen thousand tuning and evaluation tasks, each with up to 100 trials.

4.3 Corrections

The candidate corrections come from the issues raised in the previous chapters and from two behaviors of the reference implementation that we noticed while reimplementing it. The incorrect condition for microclusters with two samples, Section 2.2.1, is not among them, because the reference implementation already applies Equation (2.24).

4.3.1 Candidate Corrections

Membership. The reference implementation assigns a sample x to the macrocluster that maximizes $\sum_l w^l \left(1 - \zeta^l(x)\right)$ over its active microclusters, see Listing 3.6, where w^l is the share of the microcluster in the density of the macrocluster. For a microcluster with S samples, mean μ and variance σ^2,

$$1 - \zeta(x) = 1 - \frac{1}{2S}\left(1 + \frac{\|x - \mu\|^2}{\sigma^2}\right) \tag{4.1}$$

The standardized squared distance is divided by the number of samples, so the more samples a microcluster has, the less the distance to it matters. A sample five standard deviations away from a microcluster with 500 samples scores 0.974 in Equa-

tion (4.1), more than the 0.950 scored by a sample one standard deviation away from a microcluster with 20 samples. Replacing $1 - \zeta$ by the normalized typicality of the formulation, $t = (1 - \xi)/(S - 2)$, reverses the bias instead of removing it: the values scale with $1/S$, and the microclusters with few samples dominate. The quantity that does not depend on the number of samples is the standardized squared distance itself,

$$q(x) = S\,\xi(x) - 1 = \frac{\|x - \mu\|^2}{\sigma^2} \tag{4.2}$$

which the outlier condition also thresholds, since Equation (2.12) is equivalent to $q(x) > m^2$. We evaluate five alternatives to the membership of the implementation:

- *norm-typicality*: the implemented weights with the normalized typicality $t^l(x)$, as in Equation (2.19).
- *local-weights*: Equation (2.19) with the density of each microcluster evaluated at the sample being classified, $D^l(x) = 1/\zeta^l(x)$, which is how the formulation defines the density in Section 2.2.1.
- *density-sum*: the sum of the densities at the sample, $\sum_l D^l(x)$, with $D^l(x) = 2S^l/(1 + q^l(x))$.
- *density-max*: the largest density at the sample, $\max_l D^l(x)$.
- *nearest*: the smallest standardized distance, $\min_l q^l(x)$. The sample is assigned to the macrocluster of the active microcluster it is closest to in standard deviations.

Density. Section 2.2.1 shows that the density kept by each microcluster is the density of the last sample it absorbed. We evaluate two densities that are properties of the microcluster. The first, *peak-density*, is the density of the formulation evaluated at the mean of the microcluster: since $\xi(\mu) = 1/S$,

$$D = \frac{1}{\zeta(\mu)} = 2S \tag{4.3}$$

It is the largest value the density of the formulation can take, and it depends on the number of samples alone. This is a property of the TEDA variables and not of this particular choice: with the variance $\sigma^2 = \frac{1}{S}\sum_j \|x_j - \mu\|^2$, the mean eccentricity of the samples of a microcluster is

$$\frac{1}{S}\sum_{j=1}^{S} \xi(x_j) = \frac{1}{S}\sum_{j=1}^{S}\left(\frac{1}{S} + \frac{\|x_j - \mu\|^2}{S\sigma^2}\right) = \frac{2}{S} \tag{4.4}$$

whatever its dispersion. The second, *volume-density*, divides the number of samples by the variance, $D = S/(\sigma^2 + \epsilon)$, so that dispersion counts. In both, a microcluster with fewer than three samples has no density.

Activation mean. The reference implementation activates a microcluster whose density is not below the mean density of its component of the intersection graph, and computes that mean over all the microclusters of the component, including those with fewer than three samples, which can never be active. *mature-mean* computes it over the microclusters with at least three samples.

Initial state. The reference implementation creates a microcluster flagged as active. Since only inactive microclusters lose vitality, a new microcluster is exempt from the decay until the next macrocluster update. *born-inactive* creates it inactive.

Dimensionality penalty. *no-penalty* removes the factor $2/d$ of Equation (2.14). The factor cancels in the eccentricity, where it multiplies both the distance and the variance. It acts on the variance that is compared with r_0, on the standard deviations of the intersection condition, Equation (2.16), and on the radius of the vitality update of the reference implementation.

4.3.2 Corrections One at a Time

Table 4.4 compares each candidate with the *vectorized* model. Two candidates improve it: *nearest*, in the Overall ARI of RandomRBFGeneratorEvents and SCL-32 and in the Steady-State ARI of RandomRBFGeneratorEvents, Cassini, SCL-32 and SCL-64, and *no-penalty*, in the Steady-State ARI of SCL-32 and SCL-64.

Among the memberships, only the count-free distance helps. *density-max* is indistinguishable from the implemented membership and *density-sum* is worse. The two memberships built on the normalized typicality of the formulation, *norm-typicality* and *local-weights*, gain in the Steady-State ARI of DS4 and SCL-64 and lose in the Overall ARI of DS3, so the formulation is not preferable to its implementation in this respect.

Neither order-independent density improves the model. *peak-density* is worse in DS3 on both metrics, and *volume-density* trades a gain in the Steady-State ARI of RandomRBFGeneratorEvents for a loss in the Overall ARI of Cassini. The order dependence is real, but in this algorithm the density only decides which microclusters are active and how they are weighted, and for those purposes the alternatives are not better. *mature-mean* and *born-inactive* also regress, in DS3 and DS4 respectively.

Table 4.4: Corrections, one at a time: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
vectorized	0.000	0.581	0.328	0.520	0.230	0.235	0.851	0.859
born-inactive	0.133	0.550	0.410	0.489	0.216	0.207	0.893	0.863
peak-density	0.000	0.583	0.266	0.478	0.214	0.257	0.854	0.859
volume-density	0.004	0.559	0.402	0.319	0.228	0.223	0.817	0.804
mature-mean	0.258	0.553	0.460	0.486	0.231	0.259	0.842	0.858
no-penalty	0.000	0.576	0.328	0.512	0.232	0.256	0.864	0.860
norm-typicality	0.000	0.555	0.399	0.482	0.202	0.219	0.821	0.845
local-weights	0.000	0.562	0.472	0.478	0.194	0.223	0.839	0.861
density-sum	0.000	0.507	0.356	0.487	0.231	0.265	0.861	0.851
density-max	0.000	0.554	0.345	0.518	0.231	0.239	0.861	0.855
nearest	0.037	0.578	**0.498**	0.599	0.239	0.242	**0.964**	0.856
Steady-State ARI								
vectorized	0.000	0.541	0.000	0.458	0.176	0.209	0.000	0.000
born-inactive	0.000	0.528	0.002	0.266	0.177	0.146	0.684	0.000
peak-density	0.000	0.620	0.000	0.408	0.160	0.196	0.000	0.000
volume-density	0.000	0.548	**0.211**	0.291	0.172	0.171	0.000	0.000
mature-mean	0.000	0.498	0.000	0.428	0.161	0.210	0.000	0.366
no-penalty	0.000	0.566	0.000	0.445	0.176	0.211	**0.869**	**0.849**
norm-typicality	0.000	0.540	0.077	0.343	0.193	**0.249**	0.718	**0.821**
local-weights	0.000	0.526	**0.177**	0.431	0.181	**0.248**	0.806	**0.824**
density-sum	0.000	0.000	0.031	0.535	0.176	0.211	0.032	0.000
density-max	0.000	0.536	0.000	0.492	0.176	0.200	0.000	0.000
nearest	0.000	0.680	**0.180**	**0.593**	0.185	0.201	**0.897**	**0.848**

Table 4.5: Corrections, one at a time: predictions of a single cluster. Median share of the batches after the first, and number of the 32 development runs whose penultimate batch holds a single cluster.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Batches with a single predicted cluster [%]								
vectorized	55	0	30	2	0	4	15	12
no-penalty	55	0	30	2	0	5	0	0
nearest	44	0	0	0	0	5	0	0
Runs that end with a single predicted cluster								
vectorized	23	12	21	5	0	0	17	18
no-penalty	24	9	22	5	0	0	0	0
nearest	24	12	8	5	0	0	0	0

The zeros of Table 4.4 come from degenerate partitions, which Table 4.5 counts. In StreamBenchmark(1), the *vectorized* model predicts a single cluster in more than half of the batches of a typical run, and the Overall ARI is the median over the batches. In RandomRBFGeneratorEvents, SCL-32 and SCL-64, it ends more than half of the runs with a single predicted cluster, which gives a Steady-State ARI of zero. *no-penalty* and *nearest* end none of their runs in that state in the two scalability streams, and *nearest* also ends fewer runs so in RandomRBFGeneratorEvents.

4.3.3 Combined Corrections

Table 4.6: Combined corrections: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
nearest	0.037	0.578	0.498	0.599	0.239	0.242	0.964	0.856
nearest + peak-density	0.061	0.603	0.468	0.605	0.220	0.270	0.897	0.857
nearest + peak-density + mature-mean	0.093	0.609	0.399	0.512	0.222	0.254	0.875	0.857
nearest + no-penalty	0.041	0.611	0.504	0.589	0.238	0.241	0.859	0.859
nearest + peak-density + no-penalty	0.055	0.597	0.430	0.605	0.225	0.267	0.860	0.859
nearest + peak-density + mature-mean + no-penalty	0.111	0.611	0.413	0.512	0.219	0.255	0.860	0.859
Steady-State ARI								
nearest	0.000	0.680	0.180	0.593	0.185	0.201	0.897	0.848
nearest + peak-density	0.010	0.827	0.188	0.623	0.166	0.195	0.881	0.833
nearest + peak-density + mature-mean	0.000	0.756	0.017	0.499	0.165	0.191	0.865	0.841
nearest + no-penalty	0.000	0.736	0.162	0.588	0.185	0.200	0.867	0.849
nearest + peak-density + no-penalty	0.010	0.750	0.188	0.623	0.165	0.191	0.867	0.849
nearest + peak-density + mature-mean + no-penalty	0.000	0.789	0.019	0.488	0.153	0.188	0.867	0.849

With *nearest* as the new control, Table 4.6 evaluates whether a second correction improves it. None does. Adding *peak-density* repeats the loss in DS3. Adding *no-penalty*, which improved the *vectorized* model on its own, lowers the Overall ARI of SCL-32. The combinations of three and four corrections inherit these losses.

Table 4.7 shows what *no-penalty* loses in SCL-32, through the partitions that the two models predict. It reports their homogeneity, which is one when every predicted cluster contains samples of a single true class, and their completeness, which is one when every true class is contained in a single predicted cluster [47]. The stream has three clusters and noise, which counts as a fourth class. With the penalty, the model predicts more clusters than that, with a high homogeneity: part of the noise forms macroclusters of its own. Without the penalty it predicts about three clusters and its homogeneity falls, because the noise is absorbed by the clusters. The ARI rewards the first behavior, but the benefit is accidental, since the penalty is not a treatment of noise.

Table 4.7: Combined corrections: predicted partitions, medians over the development runs of the means over the batches.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Predicted clusters								
true clusters	3.0	3.0	4.1	3.0	9.9	8.8	4.0	4.0
nearest	1.6	2.4	5.2	3.6	23.7	19.8	4.5	3.9
nearest + no-penalty	1.7	2.7	4.9	3.8	23.6	19.3	2.9	2.9
Homogeneity								
nearest	0.31	0.64	0.51	0.74	0.77	0.68	0.92	0.76
nearest + no-penalty	0.34	0.70	0.50	0.74	0.77	0.68	0.76	0.76
Completeness								
nearest	0.90	0.95	0.70	0.74	0.56	0.55	0.90	0.90
nearest + no-penalty	0.91	0.92	0.70	0.72	0.56	0.55	0.92	0.92

The corrected model is therefore the *vectorized* model with the *nearest* membership, and it is the control of the experiments that follow. The order-dependent density and the dimensionality penalty remain in it: the experiments do not support replacing the former, and removing the latter costs the accidental separation of the noise.

4.4 Improvements

The candidates of this section modify the algorithm beyond what its formulation states. Most of them answer to what the corrected model still does wrong, which Table 4.8 summarizes through the partitions it predicts.

Table 4.8: Partitions predicted by the *corrected* model: medians over the development runs of the means over the batches. The noise label counts as one cluster, and the first batch, predicted before any sample is learned, is labeled as noise in full, the only noise of a model without a noise label.

Dataset	True clusters	Predicted clusters	Homogeneity	Completeness	True noise [%]	Predicted noise [%]
B1	3.0	1.6	0.31	0.90	9.0	3.1
B2	3.0	2.4	0.64	0.95	0.0	3.1
RBF	4.1	5.2	0.51	0.70	23.2	1.2
CSN	3.0	3.6	0.74	0.74	0.0	3.1
DS3	9.9	23.7	0.77	0.56	7.9	1.2
DS4	8.8	19.8	0.68	0.55	4.0	4.7
S32	4.0	4.5	0.92	0.90	9.0	2.9
S64	4.0	3.9	0.76	0.90	9.1	2.9

Three patterns stand out. In the static shapes of DS3 and DS4, the model predicts more than twice the true number of clusters, with high homogeneity and low completeness: the clusters it finds are pure, but each true cluster is split into several. In

StreamBenchmark(1), it predicts fewer clusters than there are, with low homogeneity: the two moving clusters are merged. Finally, the model never recognizes noise. Six of the eight datasets contain noise, and every noise sample is assigned to a cluster: the noise that the table reports as predicted comes from the first batch alone, which is predicted before any sample is learned.

A second limitation concerns the hyperparameter search. Table 4.9 compares the corrected model with *wide-range*, the same model with the upper bound of the search range of the variance limit extended from 10^2 to 10^4.

Table 4.9: Search range of the variance limit: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
corrected	0.037	0.578	0.498	0.599	0.239	0.242	0.964	0.856
wide-range	0.071	0.617	0.479	0.554	**0.376**	**0.357**	0.958	0.861
Steady-State ARI								
corrected	0.000	0.680	0.180	0.593	0.185	0.201	0.897	0.848
wide-range	0.010	0.810	0.086	0.543	**0.361**	0.000	0.897	0.835

Table 4.10: Search range of the variance limit: limit selected, median over the development runs, and number of the 32 runs whose penultimate batch holds a single predicted cluster.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Variance limit selected								
corrected	0.0259	0.0176	0.000993	0.0824	68.9	35.5	2.54	0.0117
wide-range	0.00643	0.00527	0.000785	0.0909	2.4e+03	1.77e+03	3.4	0.381
Runs that end with a single predicted cluster								
corrected	24	12	8	5	0	0	0	0
wide-range	16	7	8	8	5	19	0	0

By the decision rule the wider range improves the corrected model, in DS3 and DS4. Table 4.10 shows that the limits selected for these two datasets are far above the original bound, and several orders of magnitude above those selected for the other two-dimensional datasets: the variance limit depends on the scale of the data. We nevertheless do not adopt the wider range. It changes the search and not the model, no fixed range suits every scale, and it has a cost that the same tables show: with limits of that size, more than half of the runs of DS4 end with a single predicted cluster. One of the candidates of this section removes the dependence on the scale in the model instead.

4.4.1 Candidate Improvements

Formation of the microclusters.

- *variance-cap*: the variance limit r_0 only constrains a microcluster with two samples; from the third sample on, a microcluster grows for as long as the samples pass the eccentricity test. This variant also treats as an outlier any sample that would raise the variance of a microcluster above r_0, whatever its size, which bounds every microcluster.

- *relative-limit*: r_0 becomes a fraction of the variance of the stream, estimated recursively from the samples seen so far, and is searched in $[10^{-6}, 1]$. The hyperparameter no longer depends on the scale of the data, the limitation that Table 4.10 exposes.

- *single-assign*: a sample updates every microcluster that accepts it, so overlapping microclusters absorb the same samples. This variant updates only the accepting microcluster whose mean is nearest.

- *forgetting*: the sample count used by the recursive mean and variance is capped at $S_{\max}$, searched in $[10, 10^4]$, which makes them exponentially weighted estimates able to follow a moving cluster.

- *chi-threshold*: for a Gaussian cluster in d dimensions, the standardized distance q of Equation (4.2) has mean 1 and standard deviation $\sqrt{2/d}$. This variant replaces the outlier condition $q > m^2$ by $q > 1 + m\sqrt{2/d}$ and removes the dimensionality penalty, as a principled treatment of the dimension.

Pruning.

- *recency*: the rule of Galindo [6], Equation (2.26): every microcluster that the current sample does not update loses vitality, and a microcluster that absorbs a sample recovers it in full.

- *recency-inactive*: the same rule restricted to the inactive microclusters.

Formation of the macroclusters.

- *soft-activation*: a microcluster is active if its density is at least f times the mean density of its component, with f searched in $[0, 1.5]$. The original criterion is $f = 1$.

- *no-activation*: every microcluster with at least three samples is active.
- *single-graph*: the macroclusters are the components of the intersection graph $\mathbb{G}_0$ that contain active microclusters, instead of the components of the active graph $\mathbb{G}_1$ of Algorithm 3.
- *link-factor*: the factor 2 of the intersection condition, Equation (2.16), becomes a hyperparameter ℓ, searched in $[0.5, 4]$. Cardoso [7] proposes an earlier condition with a factor, Equation (2.29), which scales the sum of the radii $m\sigma$ of the two microclusters instead of the sum of their standard deviations.

Prediction.

- *noise-label*: a sample whose standardized distance to every active microcluster exceeds n_σ^2 is labeled as noise, with n_σ searched in $[1, 10]$.

Sensitivity function. Three variants replace the fixed constants of the function m of Equation (2.11) by hyperparameters:

- *m-reparam*: the function

$$m(S) = m_{\max}\left(1 + (m_{\max} - 1)\,(19(m_{\max} - 1))^{-(S-3)/(S_{95}-3)}\right)^{-1} \tag{4.5}$$

  which starts at $m(3) = 1$ and reaches $0.95\,m_{\max}$ at $S = S_{95}$, with $m_{\max} \in [1.05, 10]$ and $S_{95} \in [5, 5000]$.
- *m-constant*: a constant m, searched in $[1, 5]$.
- *m-asymptote*: the original sigmoid with its asymptote, 3, searched in $[1.05, 10]$.

4.4.2 Improvements One at a Time

Table 4.11 compares each candidate with the corrected model. Two candidates improve it. *single-assign* gains in the Overall ARI of StreamBenchmark(2), SCL-32 and SCL-64 and in the Steady-State ARI of StreamBenchmark(1), StreamBenchmark(2), SCL-32 and SCL-64, and *variance-cap* gains in the Steady-State ARI of RandomRBFGeneratorEvents.

Two other candidates produce the largest gains of the table together with a loss, so the rule classifies them as mixed. *noise-label* raises the Overall ARI of StreamBenchmark(1) and that of SCL-64, and loses in DS3. *relative-limit* gains in StreamBenchmark(1), StreamBenchmark(2) and DS3, and loses in the Steady-State ARI of DS4.

Table 4.11: Improvements, one at a time: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
corrected	0.037	0.578	0.498	0.599	0.239	0.242	0.964	0.856
variance-cap	0.043	0.617	0.522	0.502	0.234	0.273	0.867	0.859
soft-activation	0.028	0.567	0.444	0.573	0.215	0.240	0.978	0.855
no-activation	0.004	0.475	0.171	0.422	0.215	0.259	0.978	0.857
single-graph	0.020	0.589	0.434	0.680	0.236	0.253	0.969	0.857
link-factor	0.113	0.598	0.438	0.449	0.220	**0.271**	0.957	0.893
recency	0.052	0.555	0.388	0.497	0.172	**0.522**	0.871	0.856
recency-inactive	0.067	0.592	0.508	0.481	0.237	0.232	0.940	0.856
forgetting	0.048	0.601	0.494	0.560	0.238	0.230	0.954	0.857
noise-label	**0.919**	0.613	0.520	0.540	0.151	0.191	**0.982**	**1.000**
single-assign	0.151	**0.648**	0.512	0.465	0.234	0.246	**0.990**	**0.993**
chi-threshold	0.000	0.000	0.000	0.000	0.000	0.000	0.860	0.859
relative-limit	**0.085**	0.628	0.501	0.595	**0.351**	0.336	0.885	0.856
Steady-State ARI								
corrected	0.000	0.680	0.180	0.593	0.185	0.201	0.897	0.848
variance-cap	0.000	0.929	**0.504**	0.429	0.179	0.208	0.853	0.849
soft-activation	0.000	0.538	0.018	0.617	0.163	0.195	0.964	0.857
no-activation	0.000	0.000	0.015	0.100	0.142	0.203	0.972	0.847
single-graph	0.000	0.637	0.227	0.731	0.184	0.205	0.895	0.855
link-factor	0.025	0.806	0.226	0.321	0.157	0.209	0.900	0.850
recency	0.000	0.430	0.002	0.003	0.185	**0.482**	0.889	0.847
recency-inactive	0.000	0.675	0.140	0.390	0.185	0.185	0.904	0.840
forgetting	0.000	0.773	0.448	0.504	0.185	0.200	0.897	0.853
noise-label	**0.296**	0.855	0.363	0.515	0.163	0.167	**0.982**	**1.000**
single-assign	**0.000**	**0.938**	0.425	0.524	0.176	0.197	**0.983**	**0.990**
chi-threshold	0.000	0.000	0.000	0.000	0.000	0.000	0.867	0.849
relative-limit	**0.010**	**0.909**	0.254	0.573	**0.347**	0.023	0.855	0.835

Table 4.12: Hyperparameters selected for *noise-label* and share of the samples it labels as noise: medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Variance limit selected, searched up to 100								
corrected	0.0259	0.0176	0.000993	0.0824	68.9	35.5	2.54	0.0117
noise-label	0.000376	0.0262	0.00118	0.0765	95.3	94.8	2.28	0.0108
Noise distance selected, searched up to 10								
noise-label	7.1	3.5	5.7	6.8	9.1	9.3	4.8	3.8
Samples labeled as noise [%]								
true noise	9.0	0.0	23.2	0.0	7.9	4.0	9.0	9.1
noise-label	11.0	9.7	8.4	3.1	20.4	19.2	3.8	11.7

Table 4.13: Improvements, one at a time: predictions of a single cluster. Median share of the batches after the first, and number of the 32 development runs whose penultimate batch holds a single cluster.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Batches with a single predicted cluster [%]								
corrected	44	0	0	0	0	5	0	0
relative-limit	26	0	0	0	0	2	0	0
chi-threshold	100	100	99	100	100	100	0	0
Runs that end with a single predicted cluster								
corrected	24	12	8	5	0	0	0	0
relative-limit	16	3	9	5	4	10	0	0
chi-threshold	32	29	30	32	32	32	0	0

Both losses have identifiable causes. Table 4.12 shows that, in the two Chameleon datasets, the search selects for *noise-label* a variance limit and a noise distance at the upper end of their ranges, and that the model still labels as noise more than twice the share of true noise. Its microclusters are too small for the scale of these datasets, the limitation of the search range seen above, which *relative-limit* removes. Table 4.13 shows that *relative-limit* ends about a third of the runs of DS4 with a single predicted cluster, as *wide-range* does: without a bound, a variance limit that suits the first samples lets the microclusters grow until they merge the dataset, the growth that *variance-cap* prevents. The four changes are therefore evaluated in combination in Section 4.4.4.

The remaining candidates do not help. *forgetting* and *single-graph* are indistinguishable from the control. *recency* gains in DS4 and loses in StreamBenchmark(2) and Cassini, and *link-factor* gains in DS4 and loses in Cassini. *soft-activation*, *no-activation* and *recency-inactive* regress. Every relaxation of the activation criterion is thus neutral or harmful: the two-pass construction of the macroclusters, which Section 3.1 describes as a deviation of the reference implementation, is not a defect to be removed. *chi-threshold* collapses in every two-dimensional dataset, where Table 4.13 shows that it predicts a single cluster in practically every batch. For $d = 2$ its limit is $q \leq 1 + m$, which for the values of m close to one that Equation (2.11) gives to young microclusters is twice as permissive as $q \leq m^2$, and the microclusters grow until they form a single macrocluster.

4.4.3 Sensitivity Function

None of the reparameterizations of m improves the corrected model, as Table 4.14 shows. *m-reparam* and *m-asymptote* regress, and *m-constant* gains in StreamBench-

Table 4.14: Reparameterizations of the m function: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
corrected	0.037	0.578	0.498	0.599	0.239	0.242	0.964	0.856
m-reparam	0.009	0.544	0.407	0.581	0.234	0.246	0.857	0.853
m-constant	**0.224**	0.613	0.431	0.282	0.000	0.203	0.855	0.853
m-asymptote	0.048	0.561	0.445	0.450	0.196	0.230	0.859	0.858
Steady-State ARI								
corrected	0.000	0.680	0.180	0.593	0.185	0.201	0.897	0.848
m-reparam	0.000	0.471	0.235	0.502	0.182	0.194	0.831	0.821
m-constant	**0.110**	**0.892**	0.487	0.326	0.000	0.147	0.856	0.823
m-asymptote	0.010	0.499	0.307	0.421	0.113	0.218	0.847	0.831

mark(1) and StreamBenchmark(2) at the cost of five of the remaining six datasets. Each of these variants adds one or two hyperparameters to a search with a fixed budget of trials, and Table 4.15 shows that the values selected for them vary widely between the datasets. We retain the original function.

Table 4.15: Values selected for the hyperparameters of the sensitivity function: medians over the development runs.

Model and hyperparameter	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Selected value								
m-reparam: $m_{\max}$	4.89	3.56	2.52	5.76	4.99	5.44	4.33	4.06
m-reparam: S_{95}	267	348	1.11e+03	1.24e+03	392	478	93.9	42.6
m-constant: m	1.1	1.21	1.16	1.29	1.64	1.36	1.35	1.34
m-asymptote: asymptote	2.5	3.19	2.29	2.61	3.87	2.84	3.51	3.79

4.4.4 Combined Improvements

We evaluate the four changes that helped in isolation in every combination of two, three and four of them. In the names of the combinations, *single*, *cap*, *noise* and *relative* stand for *single-assign*, *variance-cap*, *noise-label* and *relative-limit*.

Table 4.16 shows that nine of the eleven combinations improve the corrected model, and the two that do not, *single* + *cap* and *cap* + *noise*, each lose in one dataset, Cassini and DS3 respectively. The two losses explained in the previous subsection disappear as predicted: *noise* + *relative* gains in DS3, where *noise-label* alone lost, and *cap* + *relative* gains in DS4, where *relative-limit* alone lost.

Since several combinations improve the control, the decision rule does not single

Table 4.16: Combined improvements: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
corrected	0.037	0.578	0.498	0.599	0.239	0.242	0.964	0.856
single + cap	**0.175**	0.637	**0.570**	0.475	0.239	0.222	**0.989**	**0.992**
single + noise	**0.621**	0.581	0.548	0.548	0.168	0.198	**0.990**	**0.992**
single + relative	**0.786**	0.625	**0.552**	0.554	**0.295**	0.246	**0.988**	**0.992**
cap + noise	**0.911**	0.595	0.536	0.556	0.153	0.198	**0.970**	0.995
cap + relative	**0.748**	0.621	**0.534**	0.577	**0.272**	**0.380**	0.941	0.857
noise + relative	**0.808**	**0.646**	0.492	0.581	**0.309**	0.334	**0.995**	**1.000**
single + cap + noise	**0.629**	0.621	0.530	0.514	0.160	0.177	**0.988**	**0.993**
single + cap + relative	**0.812**	0.621	**0.552**	0.478	0.231	0.272	**0.988**	**0.992**
single + noise + relative	**0.915**	**0.643**	0.510	0.455	**0.304**	0.255	**0.987**	**0.990**
cap + noise + relative	**0.775**	0.623	0.532	0.590	0.267	**0.386**	**0.982**	1.000
single + cap + noise + relative	**0.879**	0.649	**0.567**	0.515	0.236	0.271	**0.985**	**0.990**
Steady-State ARI								
corrected	0.000	0.680	0.180	0.593	0.185	0.201	0.897	0.848
single + cap	**0.000**	**0.867**	**0.478**	0.462	0.180	0.176	**0.980**	**0.988**
single + noise	**0.221**	**0.806**	**0.422**	0.576	0.181	0.150	**0.984**	**0.988**
single + relative	**0.710**	**0.867**	**0.473**	0.512	**0.341**	**0.328**	**0.982**	**0.986**
cap + noise	**0.290**	0.747	**0.553**	0.548	0.156	0.159	0.967	1.000
cap + relative	**0.271**	**0.896**	**0.514**	0.564	0.248	**0.351**	0.930	0.833
noise + relative	**0.383**	0.869	0.292	0.551	0.319	0.023	**1.000**	**1.000**
single + cap + noise	**0.215**	**0.878**	**0.489**	0.470	0.160	0.132	**0.984**	**0.990**
single + cap + relative	**0.834**	0.876	**0.444**	0.519	0.148	**0.232**	**0.982**	**0.990**
single + noise + relative	**0.865**	**0.859**	**0.395**	0.451	**0.338**	**0.320**	**0.983**	**0.982**
cap + noise + relative	**0.488**	0.744	**0.502**	0.556	**0.249**	**0.408**	0.992	1.000
single + cap + noise + relative	**0.752**	0.843	**0.397**	0.443	0.161	**0.307**	**0.984**	**0.986**

one out. We choose the combination that becomes the improved model in two steps: it must not be improved upon by any other combination in a direct comparison, and, among those that satisfy this, it must have the largest number of significant gains over the corrected model. *single + relative* and *single + noise + relative* have the most gains, twelve each over the two metrics and eight datasets. Table 4.17 gives the direct comparisons that decide between the leading combinations.

Table 4.17: Direct comparisons between combinations: clustering quality, medians over the development runs.

Model	Control	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI									
single + relative		0.786	0.625	0.552	0.554	0.295	0.246	0.988	0.992
noise + relative		0.808	0.646	0.492	0.581	0.309	0.334	0.995	1.000
single + noise + relative	*single + relative*	**0.915**	0.643	0.510	0.455	0.304	0.255	0.987	0.990
single + noise + relative	*noise + relative*	0.915	0.643	0.510	0.455	0.304	0.255	0.987	0.990
single + noise + relative	*cap + noise + relative*	**0.915**	0.643	0.510	0.455	0.304	0.255	0.987	0.990
cap + noise + relative	*noise + relative*	0.775	0.623	0.532	0.590	0.267	**0.386**	0.982	1.000
single + cap + noise + relative	*single + noise + relative*	0.879	0.649	0.567	0.515	0.236	0.271	0.985	0.990
single + cap + noise + relative	*cap + noise + relative*	0.879	0.649	0.567	0.515	0.236	0.271	0.985	0.990
Steady-State ARI									
single + relative		0.710	0.867	0.473	0.512	0.341	0.328	0.982	0.986
noise + relative		0.383	0.869	0.292	0.551	0.319	0.023	1.000	1.000
single + noise + relative	*single + relative*	**0.865**	0.859	0.395	0.451	0.338	0.320	0.983	0.982
single + noise + relative	*noise + relative*	**0.865**	0.859	0.395	0.451	0.338	**0.320**	0.983	0.982
single + noise + relative	*cap + noise + relative*	0.865	0.859	0.395	0.451	0.338	0.320	0.983	0.982
cap + noise + relative	*noise + relative*	0.488	0.744	0.502	0.556	0.249	**0.408**	0.992	1.000
single + cap + noise + relative	*single + noise + relative*	0.752	0.843	0.397	0.443	0.161	0.307	0.984	0.986
single + cap + noise + relative	*cap + noise + relative*	0.752	0.843	0.397	0.443	0.161	0.307	0.984	0.986

single + noise + relative improves *single + relative*, in StreamBenchmark(1), and no combination improves it. Its comparison with *noise + relative* is mixed, with gains in the Steady-State ARI of StreamBenchmark(1) and DS4 and a loss in that of SCL-64. Adding *variance-cap* to it regresses in DS3. The only other combination that no other improves is *cap + noise + relative*, which has seven gains over the corrected model. The two differ by a trade-off that the rule does not resolve: *single + noise + relative* is better in StreamBenchmark(1) and *cap + noise + relative* is better in DS4.

The improved model is therefore the corrected model with *single-assign*, *noise-label* and *relative-limit*. Compared with the corrected model, it gains in the Overall ARI of StreamBenchmark(1), StreamBenchmark(2), DS3, SCL-32 and SCL-64 and in the Steady-State ARI of every dataset except Cassini, where its medians are lower without the difference being significant.

4.4.5 Refinements of the Improved Model

We evaluate four further changes on the improved model. Two of them, *forgetting* and *recency*, did not help the corrected model, but they target the tracking of mov-

ing clusters, which the improved model now does better, so we give them a second chance. The third is the removal of the dimensionality penalty, which improved the *vectorized* model; we rejected it in Section 4.3 because the penalty accidentally separated part of the noise, and reconsider it now that the noise is labeled explicitly. The fourth, *persistent-labels*, makes each macrocluster keep the label of the macrocluster with which it shares most of its mass across updates, instead of being renumbered at every update. It does not change the partition predicted for a batch, but the objective of the hyperparameter search computes the ARI over the predictions of all batches together, see Section 3.6.1, where a renumbering counts as a disagreement.

Table 4.18: Refinements of the improved model: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
improved	0.915	0.643	0.510	0.455	0.304	0.255	0.987	0.990
improved + no-penalty	0.864	0.696	0.508	0.454	0.304	0.255	**1.000**	**1.000**
improved + persistent-labels	0.887	0.657	0.524	0.536	0.311	0.283	0.990	0.991
improved + forgetting	0.903	0.640	0.550	0.593	0.291	0.264	0.981	0.990
improved + recency	0.890	0.582	0.506	0.476	0.218	0.213	0.988	0.994
Steady-State ARI								
improved	0.865	0.859	0.395	0.451	0.338	0.320	0.983	0.982
improved + no-penalty	0.783	0.913	0.395	0.461	0.338	0.320	**1.000**	**1.000**
improved + persistent-labels	0.833	0.925	0.461	0.540	0.305	0.328	0.985	0.982
improved + forgetting	0.915	0.854	0.498	0.601	0.307	0.289	0.950	0.986
improved + recency	0.732	0.803	0.336	0.457	0.221	0.328	0.974	0.990

Only the removal of the penalty improves the improved model in Table 4.18: it takes the Overall and Steady-State ARI of SCL-32 and SCL-64 to 1.000. *persistent-labels* and *forgetting* are indistinguishable from the control, and *recency* regresses in DS3 and DS4.

The first two rows of each panel also give a measure of the noise of the protocol. In the six two-dimensional datasets the factor $2/d$ is one, so *improved* and *improved + no-penalty* are the same algorithm, tuned twice with the same seeds. Their medians still differ, most of all in StreamBenchmark(1) and StreamBenchmark(2), and none of these differences is significant.

Table 4.19 locates the origin of these differences in the search. The two searches select the same hyperparameters in most of the runs of four datasets and in a minority of the runs of StreamBenchmark(1) and StreamBenchmark(2), and wherever they do, the two evaluations coincide. The sampler of the search is seeded and the algorithm is deterministic, so the execution time that the objective measures is the only input

Table 4.19: Two hyperparameter searches of a single algorithm with the same seeds, *improved* and *improved + no-penalty*, in the two-dimensional datasets: number of the 32 development runs.

	B1	B2	RBF	CSN	DS3	DS4
Runs						
both searches select the same hyperparameters	5	11	31	24	32	27
both evaluations have the same Overall ARI	6	11	31	25	32	28

that differs between the two searches. Differences of this size between distinct models should therefore not be read as effects.

The model that results, MicroTEDAClus+, is therefore the improved model without the dimensionality penalty.

4.4.6 Covariance of the Microclusters

Every model so far describes a microcluster by a scalar variance, which assumes the same dispersion in every direction. This experiment replaces it by a covariance matrix, and the Euclidean distance by the Mahalanobis distance, which Section 2.1 lists among the proximity functions suited to the TEDA metrics. The covariance follows the recursion of the variance,

$$\Sigma_S = \frac{S-1}{S}\Sigma_{S-1} + \frac{1}{S-1}(x-\mu_S)(x-\mu_S)^T, \tag{4.6}$$

whose trace is the scalar variance, and the standardized distance of Equation (4.2) becomes the Mahalanobis distance divided by the dimension,

$$q(x) = \frac{1}{d}(x-\mu)^T\Sigma^{-1}(x-\mu), \tag{4.7}$$

which coincides with it for an isotropic covariance. Two adjustments keep the outlier test sound. The covariance of a microcluster with few samples is singular, so the inverted matrix is shrunk towards the isotropic matrix of the same trace, $\tilde{\Sigma} = (1-\alpha)\,\Sigma + \alpha\,\frac{\operatorname{tr}\Sigma}{d}\,I$ with $\alpha = d/(d+S-1)$, and a young microcluster behaves as an isotropic one. And because the TEDA statistics include the sample under test, a full covariance bounds its distance by $(S-1)/d$, so the test could not hold before the microcluster has more than $1+m^2d$ samples, more than 65 with 64 dimensions. The distance is therefore measured against the covariance before the sample is absorbed, q_0, and converted by the identity of the scalar recursion, $q = c\,q_0/(1+c\,q_0/(S-1))$ with $c = (S-1)/S$.

Two variants generalize the intersection condition. *covariance, directional link* takes

the standard deviation of each microcluster along the line that joins the two means, $s_i^2 = e^T \tilde{\Sigma}_i e$, and requires $\|\mu_i - \mu_j\| \leq 2\sqrt{d}\,(s_i + s_j)$. *covariance, Bhattacharyya link* requires a Bhattacharyya distance of at most $2d$ between the Gaussian distributions of the two microclusters, up to 16 dimensions, and the directional condition above. Both reduce to Equation (2.16) for two microclusters with the same isotropic covariance, and both predict by the nearest microcluster under Equation (4.7).

Table 4.20: Covariance of the microclusters: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
MicroTEDAClus+	0.864	0.696	0.508	0.454	0.304	0.255	1.000	1.000
covariance, directional link	0.897	0.607	0.524	0.467	0.303	0.239	1.000	1.000
covariance, Bhattacharyya link	0.899	0.628	0.551	0.396	0.310	0.227	1.000	1.000
Steady-State ARI								
MicroTEDAClus+	0.783	0.913	0.395	0.461	0.338	0.320	1.000	1.000
covariance, directional link	0.881	0.748	0.434	0.479	0.331	0.233	1.000	1.000
covariance, Bhattacharyya link	0.865	0.863	0.404	0.381	0.335	0.245	1.000	1.000

Applied to MicroTEDAClus+, both regress (Table 4.20): *covariance, directional link* loses in both decision metrics of StreamBenchmark(2) and *covariance, Bhattacharyya link* in its Overall ARI, and neither gains in any dataset, not even in the elongated static shapes. The covariance is also costly: a microcluster stores d^2 values instead of one, which multiplies the steady-state memory footprint of SCL-32 and SCL-64 by one to two orders of magnitude (Table A.2). We did not test why it fails to help. Two explanations fit the results: the tuned models describe each cluster by many microclusters, whose arrangement may already express its shape, and a covariance has $d(d+1)/2$ values to estimate from the samples that estimate a single variance in the original model. We retain the scalar variance.

4.4.7 Order-Independent Densities in MicroTEDAClus+

The corrections of Section 4.3 rejected the two order-independent densities when the density still weighted the membership. In MicroTEDAClus+ the density only decides which microclusters are active, so we evaluate the two densities once more, on it.

Both regress in Table 4.21. *peak-density* loses in DS3, where its Steady-State ARI falls to zero, and *volume-density* loses in five datasets. We retain the density of the last absorbed sample.

Table 4.21: Order-independent densities in MicroTEDAClus+: clustering quality, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
MicroTEDAClus+	0.864	0.696	0.508	0.454	0.304	0.255	1.000	1.000
peak-density	0.852	0.625	0.508	0.577	0.208	0.359	1.000	1.000
volume-density	0.589	0.631	0.435	0.312	0.156	0.229	1.000	1.000
Steady-State ARI								
MicroTEDAClus+	0.783	0.913	0.395	0.461	0.338	0.320	1.000	1.000
peak-density	0.742	0.828	0.347	0.558	0.000	0.345	1.000	1.000
volume-density	0.592	0.716	0.333	0.326	0.120	0.195	1.000	1.000

4.5 Validation

We tuned and evaluated the four models of the chain, *vectorized*, *corrected*, *improved* and MicroTEDAClus+, again on the 94 validation runs, which took no part in any of the choices above, as we set in Section 4.1.3. We executed two experiments of Section 4.4, those of Sections 4.4.6 and 4.4.7, after these runs. Both rejected their candidates, so we changed no model after we saw the validation runs.

4.5.1 Clustering Quality

Table 4.22 compares each model with its predecessor, and Table 4.23 compares MicroTEDAClus+ with the *vectorized* model directly.

Every step of the chain is confirmed: each model improves its predecessor and none regresses on the two metrics of the decision rule.

- The corrected model improves the *vectorized* one in StreamBenchmark(2), DS3, SCL-32 and SCL-64, on both metrics.
- The improved model improves the corrected one in the Overall ARI of every dataset except Cassini, and in the Steady-State ARI of StreamBenchmark(1), RandomRBFGeneratorEvents, DS3, DS4, SCL-32 and SCL-64.
- MicroTEDAClus+ improves the improved model in SCL-32, on both metrics.

From the *vectorized* model to MicroTEDAClus+, the Overall ARI improves in every dataset except Cassini and the Steady-State ARI improves in all eight. The largest changes are in the streams with noise and movement. The gains in the static shapes are significant but modest: in Cassini, DS3 and DS4 MicroTEDAClus+ stays below 0.5.

Table 4.22: Clustering quality of the accepted models: median and quartiles over the validation runs. Each model is compared with the model to its left.

Dataset	*vectorized*	*corrected*	*improved*	MicroTEDAClus+
Overall ARI				
B1	0.000 (0.000, 0.348)	0.059 (0.000, 0.517)	**0.843 (0.635, 0.934)**	0.853 (0.704, 0.937)
B2	0.573 (0.512, 0.634)	**0.607 (0.534, 0.677)**	**0.645 (0.587, 0.704)**	0.627 (0.576, 0.704)
RBF	0.421 (0.000, 0.547)	0.364 (0.064, 0.560)	**0.488 (0.323, 0.627)**	0.488 (0.338, 0.627)
CSN	0.490 (0.352, 0.562)	0.494 (0.329, 0.627)	0.497 (0.406, 0.634)	0.481 (0.396, 0.618)
DS3	0.231 (0.221, 0.232)	**0.238 (0.230, 0.241)**	**0.290 (0.266, 0.322)**	0.290 (0.268, 0.322)
DS4	0.248 (0.218, 0.289)	0.247 (0.223, 0.269)	**0.308 (0.240, 0.355)**	0.308 (0.240, 0.354)
S32	0.869 (0.650, 0.923)	**0.910 (0.855, 0.974)**	**0.988 (0.977, 0.992)**	**1.000 (1.000, 1.000)**
S64	0.675 (0.000, 0.872)	**0.859 (0.849, 0.953)**	**0.993 (0.952, 1.000)**	1.000 (1.000, 1.000)
Steady-State ARI				
B1	0.000 (0.000, 0.000)	0.000 (0.000, 0.056)	**0.712 (0.184, 0.919)**	0.782 (0.195, 0.963)
B2	0.541 (0.000, 0.833)	**0.802 (0.464, 0.921)**	0.842 (0.657, 0.966)	0.853 (0.692, 0.938)
RBF	0.003 (0.000, 0.530)	0.054 (0.000, 0.459)	**0.474 (0.182, 0.663)**	0.481 (0.211, 0.663)
CSN	0.374 (0.113, 0.527)	0.433 (0.249, 0.645)	0.483 (0.378, 0.660)	0.483 (0.349, 0.656)
DS3	0.176 (0.173, 0.184)	**0.184 (0.179, 0.187)**	**0.295 (0.232, 0.364)**	0.296 (0.234, 0.365)
DS4	0.210 (0.176, 0.224)	0.199 (0.175, 0.212)	**0.323 (0.210, 0.358)**	0.322 (0.217, 0.349)
S32	0.140 (0.000, 0.908)	**0.873 (0.812, 0.969)**	**0.975 (0.933, 0.994)**	**1.000 (1.000, 1.000)**
S64	0.000 (0.000, 0.000)	**0.869 (0.818, 0.908)**	**0.990 (0.925, 0.998)**	1.000 (1.000, 1.000)

Table 4.23: MicroTEDAClus+ against the *vectorized* model: medians over the validation runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Overall ARI								
vectorized	0.000	0.573	0.421	0.490	0.231	0.248	0.869	0.675
MicroTEDAClus+	**0.853**	**0.627**	**0.488**	0.481	**0.290**	**0.308**	**1.000**	**1.000**
Steady-State ARI								
vectorized	0.000	0.541	0.003	0.374	0.176	0.210	0.140	0.000
MicroTEDAClus+	**0.782**	**0.853**	**0.481**	**0.483**	**0.296**	**0.322**	**1.000**	**1.000**

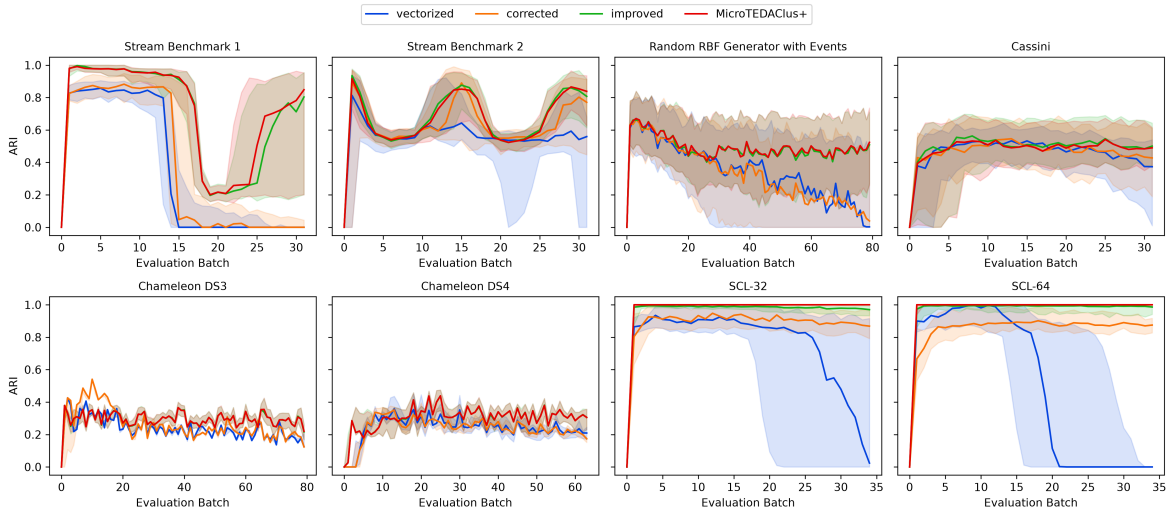


Figure 4.2: ARI of each evaluation batch for the accepted models: median and interquartile range over the 94 validation runs.

Figure 4.2 shows where the gains come from. In StreamBenchmark(1), the *vectorized* and corrected models lose the two clusters when they cross and do not recover, while the improved model and MicroTEDAClus+ separate them again. In RandomRBFGeneratorEvents, the first two models degrade along the stream and the last two hold their level. In SCL-32 and SCL-64, the *vectorized* model degrades along the stream, the corrected model stabilizes below one because it assigns the noise to the clusters, and the models with the noise label reach one.

4.5.2 Cost

We measure the execution times of the validation runs, like those of every experiment of this chapter, under parallel load. Table 4.24 gives the cost of the accepted models and of the reference implementation measured by the procedure of Section 4.2.2: each model replays the evaluation segment of each of the 32 development runs with the hyperparameters that were tuned for it in that run. The times therefore include the effect of the hyperparameters that each model leads the search to select, which is the cost of the model as it is used.

The corrected model costs the same as the *vectorized* one: the median of the ratio between their total times, computed per run, is near one in seven datasets. The exception is SCL-64, where the corrected model keeps more microclusters than the *vectorized* model. MicroTEDAClus+ takes about six tenths of the time of the *vectorized* model in DS3 and DS4, almost the same in StreamBenchmark(2), Cassini, SCL-32 and SCL-64, and more in RandomRBFGeneratorEvents and in StreamBenchmark(1).

Table 4.24: Cost of the original implementation and of the accepted models on the evaluation segment, each with its own tuned hyperparameters: median and quartiles over the 32 development runs. Each task runs alone, and its time is the smallest of three executions, which differ by 0.8% in the median.

Dataset	*original*	*vectorized*	*corrected*	*improved*	MicroTEDAClus+
Learning time [ms]					
B1	552 (271, 3404)	38.0 (27.6, 65.3)	29.1 (25.6, 66.3)	144 (56.4, 233)	152 (74.4, 221)
B2	981 (160, 1426)	36.8 (21.9, 41.0)	34.6 (22.8, 40.2)	34.3 (30.8, 38.7)	32.5 (29.5, 37.1)
RBF	16002 (4426, 32407)	243 (92.0, 502)	420 (133, 679)	441 (176, 797)	500 (182, 798)
CSN	1107 (910, 1383)	37.5 (34.9, 40.8)	38.9 (37.1, 52.7)	36.8 (32.5, 45.9)	36.9 (32.2, 49.0)
DS3	8876 (8711, 21640)	149 (146, 160)	146 (144, 161)	91.1 (88.9, 91.6)	90.7 (88.7, 91.6)
DS4	9154 (7762, 15594)	153 (131, 216)	148 (128, 152)	73.0 (70.0, 428)	73.2 (69.6, 425)
S32	665 (359, 1723)	35.3 (28.7, 56.7)	33.6 (27.6, 77.4)	31.5 (30.0, 33.0)	30.6 (29.2, 43.4)
S64	701 (423, 1679)	41.1 (34.5, 78.7)	66.5 (39.1, 152)	33.5 (30.4, 36.5)	32.1 (30.8, 37.8)
Prediction time [ms]					
B1	61.2 (38.2, 533)	5.2 (4.6, 6.3)	5.0 (4.8, 6.5)	9.3 (7.4, 10.8)	9.4 (8.2, 10.6)
B2	97.0 (31.7, 134)	5.2 (4.5, 5.6)	5.4 (4.7, 5.7)	6.1 (5.9, 6.5)	6.0 (5.8, 6.3)
RBF	1699 (472, 4993)	16.8 (13.7, 25.1)	25.6 (16.5, 31.1)	28.4 (20.5, 37.5)	30.0 (21.0, 37.5)
CSN	115 (98.0, 148)	5.3 (5.2, 5.4)	5.9 (5.7, 6.1)	6.5 (6.0, 6.9)	6.4 (6.0, 7.0)
DS3	807 (745, 1069)	16.6 (16.4, 17.0)	17.5 (17.1, 17.7)	16.2 (16.0, 16.3)	16.1 (16.0, 16.3)
DS4	620 (409, 1092)	12.4 (11.8, 13.9)	12.9 (12.2, 14.2)	13.1 (12.8, 27.4)	13.1 (12.8, 27.4)
S32	105 (76.9, 339)	6.3 (5.9, 8.3)	6.4 (6.1, 7.3)	7.8 (7.4, 8.1)	6.4 (6.4, 6.8)
S64	114 (86.5, 176)	7.1 (6.5, 8.3)	7.9 (6.9, 9.7)	8.1 (7.5, 8.7)	6.7 (6.6, 6.7)
Total time relative to the vectorized model, per run					
B1	13.56 (8.32, 48.93)	1.00 (1.00, 1.00)	0.94 (0.70, 1.08)	2.02 (1.24, 4.80)	2.67 (1.33, 5.11)
B2	25.43 (6.91, 32.56)	1.00 (1.00, 1.00)	0.98 (0.69, 1.43)	1.04 (0.87, 1.34)	0.95 (0.83, 1.34)
RBF	57.53 (32.46, 75.57)	1.00 (1.00, 1.00)	1.19 (0.85, 2.05)	1.42 (0.59, 6.31)	1.41 (0.60, 6.34)
CSN	27.72 (23.82, 38.97)	1.00 (1.00, 1.00)	1.04 (0.98, 1.45)	1.00 (0.92, 1.23)	1.00 (0.91, 1.25)
DS3	59.40 (56.86, 131.65)	1.00 (1.00, 1.00)	0.99 (0.94, 1.04)	0.63 (0.59, 0.66)	0.63 (0.59, 0.66)
DS4	59.68 (50.93, 101.93)	1.00 (1.00, 1.00)	0.98 (0.77, 1.03)	0.62 (0.53, 2.22)	0.62 (0.51, 2.22)
S32	18.09 (12.55, 26.95)	1.00 (1.00, 1.00)	0.99 (0.72, 1.75)	0.96 (0.60, 1.09)	0.94 (0.62, 1.26)
S64	16.63 (11.12, 32.53)	1.00 (1.00, 1.00)	1.47 (1.00, 1.86)	0.88 (0.46, 1.00)	0.81 (0.53, 1.03)
Steady-state microcluster count					
B1	8 (3, 94)	14 (4, 66)	8 (4, 64)	347 (86, 588)	366 (132, 600)
B2	20 (5, 46)	28 (6, 51)	19 (5, 39)	26 (18, 42)	23 (15, 32)
RBF	92 (11, 304)	104 (7, 278)	197 (26, 464)	374 (104, 688)	380 (130, 697)
CSN	36 (22, 53)	36 (30, 42)	45 (35, 60)	36 (23, 52)	36 (23, 58)
DS3	118 (114, 235)	115 (112, 118)	115 (113, 117)	26 (25, 28)	26 (25, 28)
DS4	129 (114, 225)	124 (111, 222)	120 (111, 125)	31 (29, 346)	31 (29, 346)
S32	6 (5, 24)	6 (5, 28)	6 (5, 100)	13 (11, 15)	12 (10, 22)
S64	6 (4, 20)	9 (4, 40)	28 (14, 170)	10 (7, 13)	11 (7, 14)

These differences follow the number of microclusters that the tuned models keep, and not the cost of the changes themselves. MicroTEDAClus+ keeps tens of microclusters in DS3 and DS4, where the *vectorized* model keeps hundreds, and many more than that model in StreamBenchmark(1). Even in StreamBenchmark(1), where it is slowest relative to the *vectorized* model, MicroTEDAClus+ learns the evaluation segment in a lower median time than the reference implementation. The removal of the dimensionality penalty has no cost of its own: the improved model and MicroTEDAClus+ have nearly the same times in SCL-32 and SCL-64, and in the two-dimensional datasets, where they are the same algorithm, they differ only as far as their two hyperparameter searches do. The prediction time changes little along the chain.

The quartiles of Table 4.24 are wide. This dispersion does not come from the measurement, since the three executions of a task agree closely, as the caption of the table states. It comes from the hyperparameter search, which in a quarter of the runs of DS4 selects configurations that keep hundreds of microclusters or more, against a median in the tens.

The memory footprint and the microcluster count of the 94 validation runs repeat the pattern in Table A.3. At the steady state, MicroTEDAClus+ keeps fewer microclusters than the *vectorized* model in DS3 and DS4, with medians in the tens against hundreds, and more in StreamBenchmark(1) and RandomRBFGeneratorEvents. Its memory footprint is lower in DS3 and DS4, and higher in StreamBenchmark(1). The implementation reserves room in its arrays for the microclusters that the next batch can create and enlarges them in steps, so its footprint is that of the allocated capacity and changes in steps with the number of microclusters.

4.6 Limits of the Hyperparameter Selection

MicroTEDAClus+ stays below an Overall ARI of 0.5 in the three datasets of static shapes, Cassini, DS3 and DS4, as Table 4.22 shows. Every result of this chapter measures a model as the protocol tunes it, so a low score has two possible origins: the model is not able to do better, or it is, with hyperparameters that the search does not select. In this section, we separate the two for MicroTEDAClus+.

4.6.1 Reachability Probe

The probe draws 400 configurations at random from a search space, with the distributions that the search itself samples from, and scores each of them twice on a stream. The first score is the objective of the hyperparameter search, computed on the first

20% of the stream by Algorithm 4: it is what the search sees. The second is the set of evaluation metrics, computed on the remaining 80%: it is what the search is meant to maximize.

We probe two spaces. The first is the search space of MicroTEDAClus+, with its three hyperparameters. The second, the *extended* space, adds to them six design choices studied in this chapter, as categorical hyperparameters: *single-assign* and *variance-cap*, each on or off; the density of the last absorbed sample or *peak-density*; the original pruning rule, *recency* or *recency-inactive*; the activation factor $f \in \{0, 0.25, 0.5, 0.75, 1\}$; and the link factor $\ell \in \{1, 1.5, 2\}$. In Cassini, we score each configuration on the streams of the first four development runs and average its scores. DS3 and DS4 are the fixed files presented in Section 4.1.3 and offer a single stream each.

Table 4.25: Reachability probe: for each search space, the configuration of highest objective and the one of highest Overall ARI. The first row of each dataset is MicroTEDAClus+ as the protocol tunes it, with medians over the development runs; the microcluster count is that of the penultimate batch.

Dataset	Space	Configuration	Objective	Overall ARI	Steady-State ARI	Microclusters	r	decay
CSN	final	tuned by the protocol	-	0.454	0.460	36	0.0609	46
	MicroTEDAClus+	highest objective	0.436	0.467	0.426	32	0.0621	40
	MicroTEDAClus+	highest Overall ARI	0.150	0.883	0.917	418	0.0086	1944
	extended	highest objective	0.445	0.423	0.043	4	0.2391	45
	extended	highest Overall ARI	0.092	0.977	0.996	610	0.0064	3507
DS3	final	tuned by the protocol	-	0.304	0.337	26	0.0610	34
	MicroTEDAClus+	highest objective	0.395	0.275	0.267	30	0.0570	52
	MicroTEDAClus+	highest Overall ARI	0.057	0.587	0.625	1238	0.0032	8694
	extended	highest objective	0.261	0.302	0.265	47	0.1648	50
	extended	highest Overall ARI	0.049	0.721	0.797	1445	0.0024	3857
DS4	final	tuned by the protocol	-	0.254	0.320	31	0.0559	51
	MicroTEDAClus+	highest objective	0.335	0.281	0.341	27	0.0570	52
	MicroTEDAClus+	highest Overall ARI	0.043	0.514	0.474	1104	0.0032	8694
	extended	highest objective	0.274	0.352	0.352	23	0.0609	16
	extended	highest Overall ARI	0.113	0.594	0.657	426	0.0050	159

Table 4.25 and Figure 4.3 show that the search space of MicroTEDAClus+ holds configurations far better than those the protocol selects. The best configuration has an Overall ARI above one half in Cassini, DS3 and DS4, where MicroTEDAClus+ as the protocol tunes it stays below it.

The objective does not rank them high. The best configuration of each dataset has less than half of the objective of the configuration that the objective prefers, and the evaluation scores of the latter are close to those of the protocol. The search therefore does what it is asked to do: it finds the configurations that the objective prefers. Over the whole space the two scores are positively correlated because most configurations fail under both. It is among the configurations that work that the objective and the evaluation disagree.

The two kinds of configuration differ in granularity. Those that the objective prefers

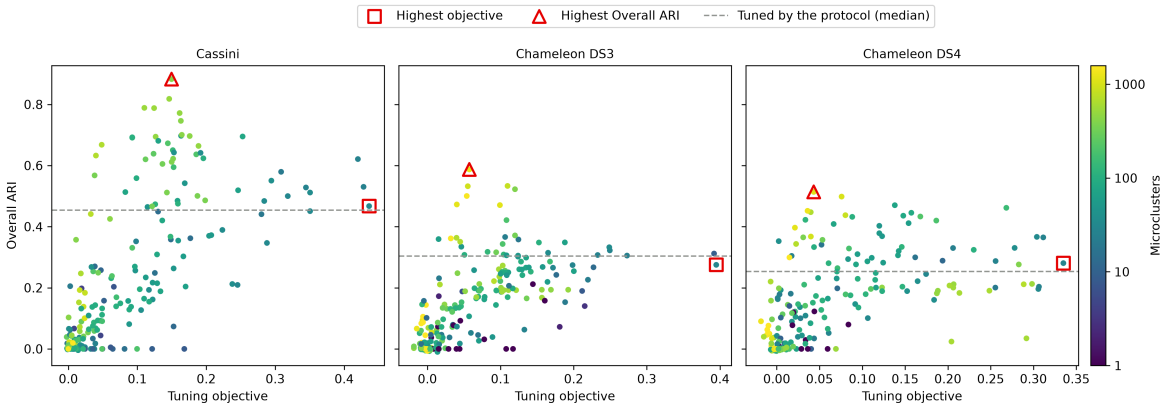


Figure 4.3: Objective of the hyperparameter search against Overall ARI of the 400 configurations drawn from the search space of MicroTEDAClus+. The color gives the number of microclusters at the penultimate batch.

have higher relative variance limit and a lower decay. The best ones have a lower variance limit and a decay in the thousands, and keep far more microclusters.

Table 4.26 follows the two configurations along the stream and shows why the objective prefers the coarse one. In the first fifth of the stream, which is the segment that the search sees, the coarse configuration is the better of the two in the three datasets. From then on it declines, while the fine-grained configuration improves. The second and third panels of the table show what changes. In the first fifth the fine-grained configuration predicts about three times the true number of clusters or more: its many small microclusters are not yet connected into macroclusters. As the stream advances its microclusters multiply and that number falls towards the true one. In DS3 it falls below the true number, and the advantage of the fine-grained configuration is less regular there, with a higher score than the coarse configuration in the third and last fifths and a lower one in the second and fourth. The objective is thus not wrong about the segment it observes. It observes a segment in which the configuration that is better in the long run has not matured. The penalty that the objective applies to the execution time does not explain the preference: it amounts to 10^{-3} per second of execution, and these executions take at most a few seconds.

The extended space holds better configurations still, and the objective profits even less from it. A larger space searched with the same objective does not lead to better models, which agrees with the outcome of the reparameterizations of the sensitivity function in Section 4.4.3. The probe draws the six design choices at random together with the three hyperparameters, and is not designed to attribute the advantage of the best configurations to any of them. What these configurations share with the best ones of the first space is a small variance limit and hundreds of microclusters.

Table 4.26: The two configurations of the search space of MicroTEDAClus+ in Table 4.25 along the stream, in fifths, when the model learns the whole stream from its first sample. The first fifth is the segment that the hyperparameter search sees.

Dataset	Configuration	0–20%	20–40%	40–60%	60–80%	80–100%
Median batch ARI						
CSN	highest objective	0.673	0.586	0.472	0.446	0.442
	highest Overall ARI	0.355	0.803	0.937	0.981	0.998
DS3	highest objective	0.442	0.382	0.311	0.302	0.277
	highest Overall ARI	0.272	0.267	0.486	0.243	0.388
DS4	highest objective	0.387	0.383	0.345	0.346	0.286
	highest Overall ARI	0.247	0.500	0.573	0.555	0.559
Median number of predicted clusters						
CSN	true clusters	3.0	3.0	3.0	3.0	3.0
	highest objective	4.5	6.2	7.4	8.1	8.5
	highest Overall ARI	11.8	6.0	4.4	3.9	3.0
DS3	true clusters	10.0	10.0	10.0	10.0	10.0
	highest objective	7.0	8.0	8.0	8.0	7.0
	highest Overall ARI	27.5	10.0	7.0	5.0	5.0
DS4	true clusters	9.0	9.0	9.0	9.0	9.0
	highest objective	7.0	7.5	7.0	7.5	7.0
	highest Overall ARI	32.0	20.5	14.0	10.0	8.0
Microclusters at the end of the segment						
CSN	highest objective	35	34	34	34	33
	highest Overall ARI	216	327	396	409	420
DS3	highest objective	32	35	30	29	26
	highest Overall ARI	544	824	1038	1231	1333
DS4	highest objective	29	35	29	27	27
	highest Overall ARI	490	742	943	1128	1283

4.6.2 Consequences

Three consequences follow.

First, the modest scores of MicroTEDAClus+ in the static shapes are, at least in part, a limitation of the selection of its hyperparameters and not of the algorithm. With a fine-grained configuration the model separates the three shapes of Cassini almost completely, on four different streams, and roughly doubles its Overall ARI in DS3 and DS4. The figures of DS3 and DS4 are optimistic, since the best of 400 configurations is chosen on the stream on which it is scored and there is no second stream to confirm it.

Second, every comparison of this chapter is conditional on the protocol. We accepted a candidate if it improved its control when both are tuned by the protocol, so we rejected a candidate whose benefit requires a configuration that the objective does not select. The same holds for MicroTEDAClus and the benchmark models of the main experiment, which are tuned by the same protocol. We kept the protocol unchanged throughout this chapter for that reason: it is what places MicroTEDAClus+ and the benchmark models on equal terms.

Third, the search is itself a source of variation. Two searches of a single algorithm with the same seeds produce medians that differ in the Overall ARI and in the Steady-State ARI (Section 4.4.5). In DS3 and DS4, where every run processes the same stream, the dispersion of the results across runs is entirely that of the search.

The probe does not change the model. It indicates that the most direct way of improving its results in static shapes is not a change of the algorithm but an objective that observes a longer segment of the stream. The modernized experiment of Chapter 5 lengthens the streams and their tuning segments and replaces the search ranges by rules (Sections 5.3.1 and 5.3.2).

4.7 MicroTEDAClus+

MicroTEDAClus+ keeps the structure of MicroTEDAClus: microclusters updated recursively with the TEDA statistics, macroclusters as connected components of the graph of intersecting active microclusters, and pruning of the inactive microclusters by vitality. It differs from the algorithm of the reference implementation in the five points that the experiments accepted, listed in Table 4.27. It has three hyperparameters: the relative variance limit r, the decay, with $\rho = 1/\text{decay}$, and the noise distance n_σ.

Table 4.27: Differences between MicroTEDAClus+ and the algorithm of the reference implementation of MicroTEDAClus.

	MicroTEDAClus	MicroTEDAClus+
Membership	macrocluster with the largest density-weighted sum of $1-\zeta$	macrocluster of the active microcluster with the smallest standardized distance
Update	every microcluster that accepts the sample	the nearest microcluster that accepts the sample
Noise	every sample is assigned to a macrocluster	a sample farther than n_σ standard deviations from every active microcluster is noise
Variance limit	absolute, r_0	relative to the variance of the stream, $r_0 = r\,\sigma^2_{\text{stream}}$
Distances	scaled by $2/d$	not scaled

4.7.1 Microclusters

A microcluster i is described by its number of samples S^i, its mean μ^i, its variance $(\sigma^i)^2$, its density D^i, its vitality v^i and a flag that tells whether it is active. The model also keeps the variance of the stream, σ^2_{stream}, the mean squared distance of the samples seen so far to their mean, updated before each batch is learned. For a sample x and a microcluster i,

$$S' = S^i + 1, \qquad \mu' = \frac{S'-1}{S'}\mu^i + \frac{x}{S'}, \qquad \sigma'^2 = \frac{S'-1}{S'}(\sigma^i)^2 + \frac{\|x-\mu'\|^2}{S'-1}, \qquad \xi' = \frac{1}{S'} + \frac{\|x-\mu'\|^2}{S'\sigma'^2} \tag{4.8}$$

where $\xi' = 1/S'$ when $\sigma'^2 = 0$. The sample is an outlier to the microcluster, outlier $\in$ [false, true], if

$$\text{outlier} = \begin{cases} \sigma'^2 > r\,\sigma^2_{\text{stream}}, & S' < 3 \\ \dfrac{\xi'}{2} > \dfrac{m(S')^2+1}{2S'}, & S' \geq 3 \end{cases} \tag{4.9}$$

Algorithm 5 processes a sample.

4.7.2 Macroclusters

The macroclusters are rebuilt after each batch. Two microclusters intersect when Equation (2.16) holds between their means and standard deviations, which defines the

Algorithm 5 Microcluster update of MicroTEDAClus+ for a sample x

Require: x, the microclusters, r, ρ, σ^2_{stream}
$A \leftarrow$ the microclusters to which x is not an outlier by Equation (4.9)
if $A = \emptyset$ **then**
    create a microcluster with $S = 1$, $\mu = x$, $\sigma^2 = 0$, $D = 0$, $v = 1$, flagged as active
else
    $i \leftarrow \arg\min_{l \in A} \|x - \mu^l\|$
    $v^i \leftarrow v^i + \rho\,(\sigma^i - \|x - \mu^i\|)/\sigma^i$ if $\sigma^i > 0$, otherwise $v^i \leftarrow 1$
    $S^i \leftarrow S'$, $\mu^i \leftarrow \mu'$, $(\sigma^i)^2 \leftarrow \sigma'^2$, $D^i \leftarrow 2/\xi'$, from Equation (4.8)
end if
for each microcluster l that is not active **do**
    $v^l \leftarrow v^l - \rho$
    remove l if $v^l < 0$
end for

graph $\mathbb{G}_0$. In each connected component of $\mathbb{G}_0$, a microcluster is active if it has at least three samples and its density is not below the mean density of the microclusters of the component. The macroclusters are the connected components of $\mathbb{G}_1$, the subgraph of $\mathbb{G}_0$ induced by the active microclusters.

4.7.3 Prediction

For $q^l(x) = \|x - \mu^l\|^2/(\sigma^l)^2$, the standardized squared distance of a sample to an active microcluster l, and $l^* = \arg\min_l q^l(x)$,

$$l_x = \begin{cases} -1, & q^{l^*}(x) > n_\sigma^2 \\ l_{\text{macro}}(l^*), & \text{otherwise} \end{cases} \tag{4.10}$$

Before the first batch is learned there is no active microcluster, and every sample is labeled as noise.

4.7.4 What Remains of the Original Model

The function m, the density, the activation criterion, the two-pass construction of the macroclusters and the vitality mechanism are those of the reference implementation. We challenged each of them in this chapter with at least one alternative and none of the alternatives improved the model. In particular, the density of a microcluster is still the density of the last sample it absorbed, the order dependence identified in Section 2.2.1: in MicroTEDAClus+ it only decides which microclusters are active, and Section 4.4.7 verifies whether an order-independent density can replace it there.

We search the hyperparameters in $r \in [10^{-6}, 1]$ and decay $\in [10, 10^4]$, both on a logarithmic scale, and $n_\sigma \in [1, 10]$.

In the experiments of this chapter, we execute MicroTEDAClus+ as a configuration of the implementation of Section 4.2. For the main experiment, we implement the model on its own, with its three hyperparameters and without the options of the experimental implementation, and we verified that the two implementations produce the same predictions and the same number of microclusters, batch by batch, on scalability streams with 2, 8 and 32 dimensions.

4.8 MicroTEDAClus+ in the Main Experiment

The experiments above compare models that share one implementation, on runs reserved for the development of the model. In this section, we return to the main experiment of Chapter 3. We tune and evaluate MicroTEDAClus+ on its 94 runs, six datasets and seven scalability streams, by the same code and on the same machine as the benchmark models, and compare it with MicroTEDAClus and the six benchmark models as Section 3.8 compares MicroTEDAClus. The streams of these runs took no part in the development of the model, except those of DS3 and DS4, which are the same in every run (Section 4.1.3).

4.8.1 Clustering Quality

In the Overall ARI, MicroTEDAClus+ is better than MicroTEDAClus in five datasets and indistinguishable from it in Cassini, and in the Steady-State ARI it is better in all six (Table 4.28). Against the benchmark models, it reverses the position of MicroTEDAClus in the streams with concept drift. Its Overall ARI is better than that of every other model in StreamBenchmark(1) and StreamBenchmark(2), and than all but TEDACloud in RandomRBFGeneratorEvents, and no model is better than it at the penultimate batch of these streams. In the static shapes, it is better than MicroTEDAClus, DenStream and the TEDA-based models, but worse than CluStream and StreamKM in Cassini and DS3, on both metrics, and worse than CluStream in the Overall ARI of DS4.

In RandomRBFGeneratorEvents, DS3 and DS4, the comparison with MicroTEDAClus also carries a difference of search budget, since the reference implementation exhausts the time of the search before its trials in these datasets (Table 4.2). The validation of Section 4.5, in which every model completes its trials, shows that the gains over the *vectorized* model do not depend on that difference.

Table 4.28: Clustering quality of MicroTEDAClus+, MicroTEDAClus and the benchmark models: medians over the runs of the main experiment.

Model	B1	B2	RBF	CSN	DS3	DS4
Overall ARI						
MicroTEDAClus+	0.894	0.634	0.491	0.496	0.288	0.296
Better/same/worse	7/0/0	7/0/0	6/1/0	4/1/2	5/0/2	5/1/1
MicroTEDAClus	**0.035**	**0.564**	**0.359**	0.492	**0.215**	**0.202**
DenStream	**0.665**	**0.402**	**0.360**	**0.383**	**0.174**	**0.184**
CluStream	**0.096**	**0.566**	**0.170**	0.573	0.324	0.319
StreamKM	**0.183**	**0.498**	**0.371**	0.696	0.330	0.327
AutoCloud	**0.043**	**0.001**	**0.051**	**0.076**	**0.023**	**0.030**
CEDAS	**0.252**	**0.000**	**0.178**	**0.000**	**0.000**	**0.000**
TEDACloud	**0.827**	**0.509**	0.495	**0.360**	**0.040**	**0.125**
Steady-State ARI						
MicroTEDAClus+	0.869	0.842	0.495	0.500	0.300	0.316
Better/same/worse	6/1/0	6/1/0	4/3/0	5/0/2	5/0/2	4/3/0
MicroTEDAClus	**0.000**	**0.524**	**0.000**	**0.436**	**0.126**	**0.124**
DenStream	**0.822**	**0.567**	0.340	**0.405**	**0.196**	0.274
CluStream	**0.174**	**0.599**	**0.194**	0.589	0.338	0.320
StreamKM	**0.164**	0.860	0.451	0.569	0.341	0.301
AutoCloud	**0.020**	**0.135**	**0.040**	**0.052**	**0.073**	**0.070**
CEDAS	**0.211**	**0.000**	**0.142**	**0.000**	**0.000**	**0.000**
TEDACloud	0.760	**0.518**	0.541	**0.366**	**0.117**	**0.091**

Table 4.29: Clustering quality of MicroTEDAClus+, MicroTEDAClus and the benchmark models in the scalability streams: medians over the runs of the main experiment.

Model	S1	S2	S4	S8	S16	S32	S64
Overall ARI							
MicroTEDAClus+	0.694	0.947	1.000	1.000	1.000	1.000	1.000
Better/same/worse	4/3/0	7/0/0	6/1/0	6/1/0	6/1/0	6/1/0	6/1/0
MicroTEDAClus	0.763	**0.861**	**0.860**	**0.867**	**0.883**	**0.865**	**0.843**
DenStream	**0.000**	**0.690**	**0.442**	**0.426**	**0.000**	**0.000**	**0.000**
CluStream	0.661	**0.887**	**0.895**	**0.913**	**0.953**	**0.980**	**0.991**
StreamKM	0.665	**0.887**	**0.927**	**0.953**	**0.971**	**0.983**	**0.989**
AutoCloud	**0.325**	**0.446**	**0.405**	**0.426**	**0.435**	**0.443**	**0.445**
CEDAS	**0.511**	**0.932**	1.000	1.000	1.000	1.000	1.000
TEDACloud	**0.495**	**0.536**	**0.565**	**0.924**	**0.567**	**0.503**	**0.586**
Steady-State ARI							
MicroTEDAClus+	0.677	0.950	1.000	1.000	1.000	1.000	1.000
Better/same/worse	4/3/0	7/0/0	7/0/0	6/1/0	6/1/0	6/1/0	6/1/0
MicroTEDAClus	0.727	**0.857**	**0.856**	**0.847**	**0.810**	**0.462**	**0.000**
DenStream	**0.000**	**0.768**	**0.493**	**0.454**	**0.000**	**0.000**	**0.000**
CluStream	0.677	**0.873**	**0.894**	**0.916**	**0.960**	**0.980**	**0.991**
StreamKM	0.621	**0.903**	**0.934**	**0.957**	**0.972**	**0.985**	**0.990**
AutoCloud	**0.280**	**0.419**	**0.372**	**0.379**	**0.421**	**0.419**	**0.433**
CEDAS	**0.508**	**0.885**	**1.000**	1.000	1.000	1.000	1.000
TEDACloud	**0.532**	**0.583**	**0.606**	**0.903**	**0.598**	**0.529**	**0.510**

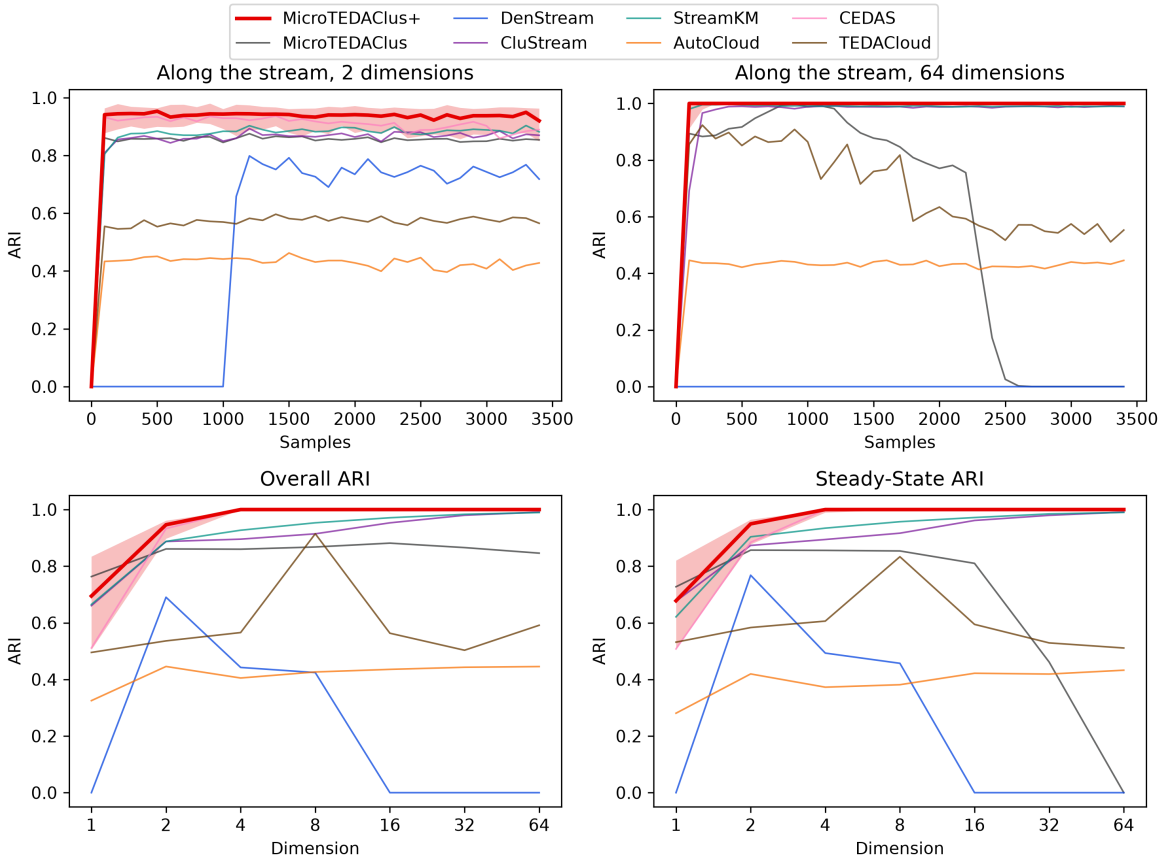


Figure 4.4: ARI of MicroTEDAClus+, MicroTEDAClus and the benchmark models in the scalability streams: along the stream with 2 and 64 dimensions, and the Overall and Steady-State ARI against the dimension. Medians, with the interquartile band of MicroTEDAClus+, over 94 runs.

In the scalability streams, MicroTEDAClus+ is better than every other model from two dimensions on, on both metrics, except CEDAS, which reaches the same median of 1.000 and is indistinguishable from it from four or eight dimensions on (Table 4.29). The collapse of MicroTEDAClus along the streams of high dimension does not occur (Figure 4.4). The stream with one dimension is the exception: there, MicroTEDAClus+ is indistinguishable from MicroTEDAClus, CluStream and StreamKM, with a lower median than MicroTEDAClus. No one-dimensional stream took part in the development of the model.

4.8.2 Cost

MicroTEDAClus+ is faster than every other model in the learning stage of every dataset, except CEDAS in StreamBenchmark(1) and RandomRBFGeneratorEvents, from which it is indistinguishable, and faster than every other model in the prediction stage of every dataset (Table 4.30). Its times are at or below the resolution of 10 ms to which the analysis discretizes them, which is why most of its prediction times are printed as zero. Its learning time is one to two orders of magnitude below that of MicroTEDAClus in four datasets and close to three in the Chameleon datasets. These ratios combine the optimization of Section 4.2 with the number of microclusters that each tuned model keeps, an order of magnitude smaller for MicroTEDAClus+ in the Chameleon datasets (Table 4.31). Like those of Section 3.8, these comparisons measure implementations as much as algorithms: MicroTEDAClus+ runs the sequential part of the algorithm in compiled code, and Section 4.2.2 measures what that implementation alone contributes for MicroTEDAClus.

Its memory footprint is smaller than those of MicroTEDAClus and CluStream in every dataset and larger than those of CEDAS and TEDACloud in every dataset. The model stores the microclusters in arrays that always have room for one new microcluster per sample of the next batch and that are enlarged to twice the required size when that room runs out, so the footprint we measure is that of the allocated capacity, the same for every run that never keeps more than 100 microclusters.

In the scalability streams, the learning and prediction times of MicroTEDAClus+ hardly change with the dimension and stay below those of every other model (Figure 4.5). Its maximum memory footprint grows with the dimension, since each microcluster stores a mean, and stays below those of MicroTEDAClus, CluStream and StreamKM at every dimension and above those of DenStream, CEDAS and TEDACloud from eight dimensions on.

Table 4.30: Execution time and memory footprint of MicroTEDAClus+, MicroTEDAClus and the benchmark models: medians over the runs of the main experiment.

Model	B1	B2	RBF	CSN	DS3	DS4
Learning time [ms]						
MicroTEDAClus+	30	10	70	10	30	30
Better/same/worse	6/1/0	7/0/0	6/1/0	7/0/0	7/0/0	7/0/0
MicroTEDAClus	**1000**	**570**	**8210**	**590**	**27850**	**18815**
DenStream	**180**	**200**	**200**	**190**	**210**	**210**
CluStream	**230**	**140**	**4260**	**120**	**380**	**375**
StreamKM	**60**	**30**	**120**	**25**	**120**	**105**
AutoCloud	**12910**	**50**	**460**	**16910**	**445**	**97150**
CEDAS	30	**20**	90	**30**	**90**	**70**
TEDACloud	**110**	**50**	**530**	**270**	**360**	**310**
Prediction time [ms]						
MicroTEDAClus+	0	0	10	0	0	0
Better/same/worse	7/0/0	7/0/0	7/0/0	7/0/0	7/0/0	7/0/0
MicroTEDAClus	**100**	**50**	**800**	**60**	**480**	**445**
DenStream	**90**	**140**	**150**	**135**	**70**	**110**
CluStream	**100**	**80**	**1220**	**80**	**260**	**200**
StreamKM	**10**	**10**	**30**	**10**	**30**	**30**
AutoCloud	**4770**	**20**	**160**	**6200**	**160**	**35680**
CEDAS	**20**	**10**	**60**	**20**	**70**	**50**
TEDACloud	**40**	**20**	**150**	**70**	**110**	**100**
Maximum memory footprint [KiB]						
MicroTEDAClus+	25	13	25	13	13	13
Better/same/worse	2/3/2	4/1/2	3/1/3	5/0/2	3/0/4	4/0/3
MicroTEDAClus	**277**	**141**	**1030**	**140**	**702**	**628**
DenStream	15	**28**	10	**24**	10	10
CluStream	**209**	**152**	**1066**	**148**	**218**	**201**
StreamKM	27	**28**	**84**	**31**	**35**	**38**
AutoCloud	654	3	11	**706**	9	**1941**
CEDAS	8	3	9	5	8	8
TEDACloud	6	3	11	12	7	7
Steady-state memory footprint [KiB]						
MicroTEDAClus+	25	13	24	13	13	13
Better/same/worse	2/1/4	4/1/2	3/1/3	5/0/2	3/0/4	4/0/3
MicroTEDAClus	**155**	**106**	**423**	**118**	**698**	**626**
DenStream	12	**23**	9	**18**	6	9
CluStream	**209**	**152**	**1066**	**148**	**218**	**201**
StreamKM	16	**17**	**31**	**17**	**20**	**28**
AutoCloud	629	2	9	**685**	6	**1914**
CEDAS	7	3	8	5	8	8
TEDACloud	6	2	9	8	3	4

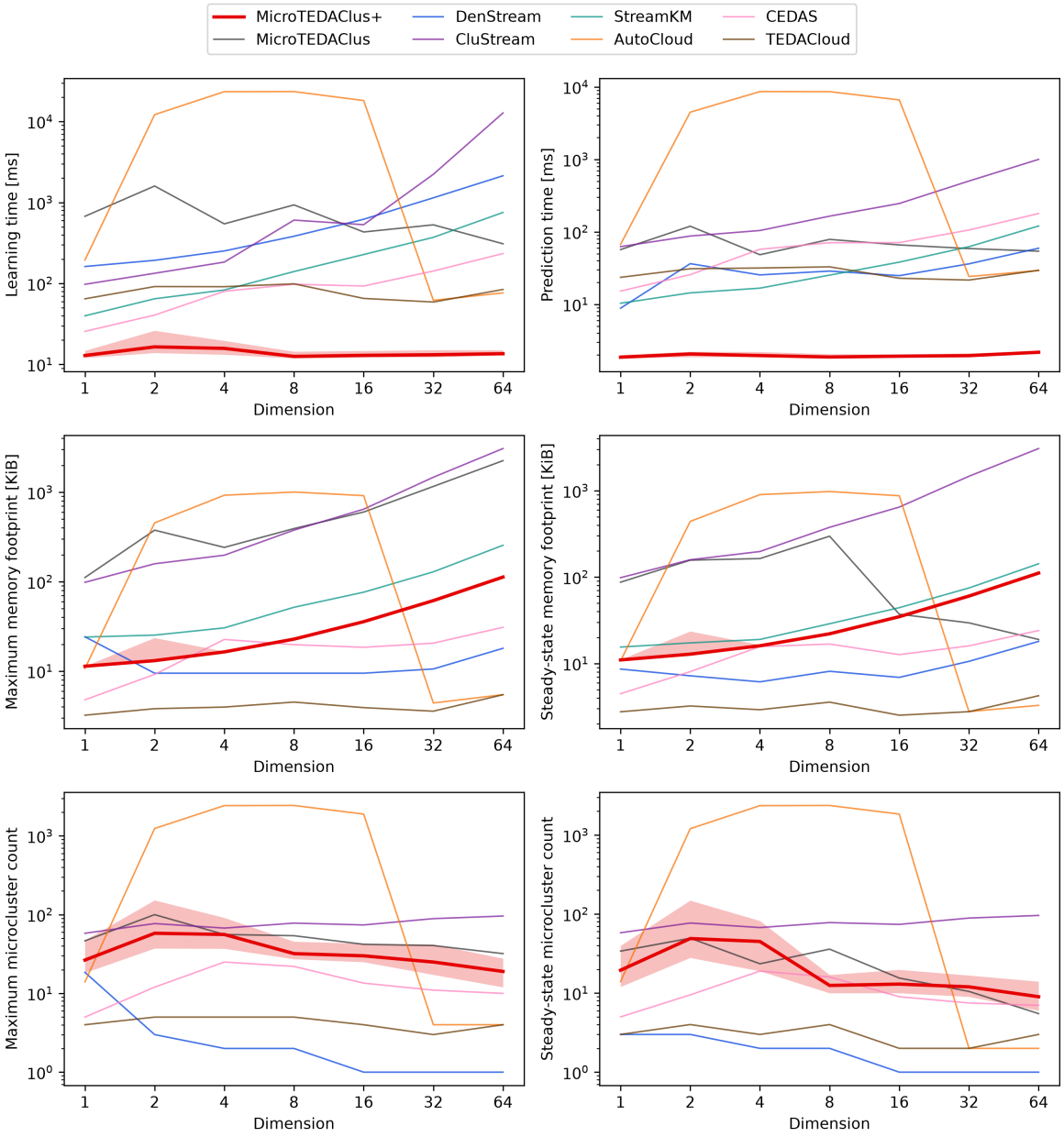


Figure 4.5: Execution time, memory footprint and microcluster count of MicroTEDAClus+, MicroTEDAClus and the benchmark models in the scalability streams, against the dimension. Medians, with the interquartile band of MicroTEDAClus+, over 94 runs.

4.8.3 Microcluster Count

Table 4.31: Microcluster count of MicroTEDAClus+, MicroTEDAClus and the benchmark models: medians over the runs of the main experiment. StreamKM keeps no microclusters.

Model	B1	B2	RBF	CSN	DS3	DS4
Maximum microcluster count						
MicroTEDAClus+	174	31	170	44	40	40
Better/same/worse	0/1/5	1/2/3	1/2/3	1/2/3	2/0/4	3/0/3
MicroTEDAClus	82	45	215	44	**509**	**430**
DenStream	7	17	4	14	2	4
CluStream	103	**74**	**530**	71	**105**	**97**
AutoCloud	1165	3	14	**1882**	12	**5345**
CEDAS	9	2	11	6	10	10
TEDACloud	8	3	14	16	10	9
Steady-state microcluster count						
MicroTEDAClus+	167	24	146	38	26	31
Better/same/worse	0/1/5	1/2/3	1/2/3	2/1/3	2/0/4	3/0/3
MicroTEDAClus	42	30	102	34	**506**	**426**
DenStream	6	12	4	10	2	4
CluStream	103	**74**	**530**	**71**	**105**	**97**
AutoCloud	1081	2	13	**1822**	8	**5266**
CEDAS	8	2	9	5	10	10
TEDACloud	7	2	12	11	4	5

MicroTEDAClus+ keeps more microclusters than DenStream, CEDAS and TEDACloud in every dataset (Table 4.31). It keeps fewer than MicroTEDAClus and CluStream in the Chameleon datasets and more than both in StreamBenchmark(1), where only AutoCloud is not exceeded. The count reflects the granularity of the configuration that the hyperparameter search selects as much as the model: the small counts of the Chameleon datasets are those of the coarse configurations that Section 4.6 examines. In the scalability streams, its maximum count stays between those of CEDAS and CluStream at every dimension (Figure 4.5).

4.8.4 Hyperparameters of MicroTEDAClus+

Table 4.32: Hyperparameters selected for MicroTEDAClus+: median [QCD] over the runs of the main experiment.

Model	Hyperparameter	B1	B2	RBF	CSN	DS3	DS4
MicroTEDAClus+	relative variance limit	0.00364 [0.98]	0.0348 [0.75]	0.0172 [0.74]	0.058 [0.41]	0.0622 [0.07]	0.0599 [0.19]
	decay	629 [0.78]	64.5 [0.50]	896 [0.85]	35 [0.80]	34 [0.30]	32 [0.79]
	noise distance	7.32 [0.16]	6.51 [0.27]	5.72 [0.20]	7.58 [0.18]	7.56 [0.16]	6.67 [0.19]

The relative variance limit selected for MicroTEDAClus+ spans about one order of magnitude across the datasets, against four for the absolute limit of MicroTEDAClus (Tables 3.7 and 4.32), and is nearly the same in the three static datasets, whose coordinates differ in scale by two orders of magnitude. The relative limit thus removes the dependence on the scale of the data that Section 3.9 reports. Its dispersion is larger in the three streams with concept drift than in the static datasets. The median decay is also larger in the streams with concept drift, while its dispersion is high in almost every dataset. The noise distance has a low dispersion in every dataset. In the Chameleon datasets, every run processes the same stream, so the dispersion of their columns is that of the hyperparameter search alone.

4.8.5 Summary

In the main experiment, MicroTEDAClus+ improves on MicroTEDAClus and changes its position among the benchmark models.

- **Clustering quality.** It is better than MicroTEDAClus in the Overall ARI of five of the six datasets and in the Steady-State ARI of all six. No benchmark model is better than it in the streams with concept drift, where MicroTEDAClus was worse than most of them, but CluStream and StreamKM remain better in the static shapes. In the scalability streams, it is better than every model but CEDAS from two dimensions on, without the collapse of MicroTEDAClus in high dimensions.

- **Cost.** It is the fastest model in the learning and prediction stages of every dataset and at every dimension, tied with CEDAS in the learning stage of two datasets. Its memory footprint is smaller than those of MicroTEDAClus and CluStream and larger than those of CEDAS and TEDACloud.

- **Hyperparameters.** Its variance limit no longer depends on the scale of the data.

The weakness that remains is the clustering of static shapes, which Section 4.6 relates to the configurations that the hyperparameter search selects. Chapter 5 repeats the experiment under a modernized protocol, and Chapter 6 on real data.

Chapter 5

Experiment Modernization

The main experiment of the previous chapters runs part of its generators in `R` and `Java`, as Chapter 3 describes, and its protocol carries the constraints of that environment. Its hyperparameter search penalizes the execution time and stops at a time limit, and its datasets have at most 10000 samples, because some of the models are too slow for an unrestricted search, as seen in Section 3.6.1. We set the ranges of that search by hand, as values that seemed reasonable after a brief survey of the literature [1,2,17,18,24,25]. In Section 4.6, we found that the objective of the search observes too short a segment of the stream to recognize the configurations that are best in the long run, and that the search is itself a source of variation.

In this chapter, we replace the environment, the benchmark models and the hyperparameter selection, and repeat the main experiment under them. We evaluate MicroTEDAClus+ as Section 4.7 defines it, without changes.

5.1 Environment

We write the experiment in `Python` only. Its numerical parts run in compiled code through the libraries that provide them, and no other runtime is involved. MicroTEDAClus+ keeps the implementation of Section 4.7, and we use every model through one interface: it predicts the labels of a batch, learns the batch, and reports the number of structures of its summary of the stream, which generalizes the microcluster count of Section 3.5. We label as noise a sample that a model predicts before it has any cluster.

5.1.1 Stream Generators

Four generators of Section 3.3 ran in `R`: that of Cassini and those of StreamBenchmark(1), StreamBenchmark(2) and RandomRBFGeneratorEvents, the last from MOA

[48] through `streamMOA` [23]. We port them to `Python`, following their reference implementations step by step. The Chameleon datasets remain the files of Section 3.3, and the scalability streams were already generated in `Python`. A stream is determined by its dataset and its run, which seed its generator.

We check each port against the streams that `R` generated for the previous experiment through the ARI of three models with fixed hyperparameters (Table 5.1), compared by the Mann-Whitney test with the Holm correction and the threshold on Cliff's delta of Section 3.7. No test finds a difference.

Table 5.1: Models with fixed hyperparameters on the streams generated by R and by the port: medians over the streams. A value in red is significantly different from that of R.

Model and source	B1	B2	RBF	CSN
Overall ARI				
MicroTEDAClus+, R	0.956	0.663	0.598	0.200
MicroTEDAClus+, port	0.953	0.666	0.617	0.197
mini-batch k-means, R	0.821	0.638	0.428	0.562
mini-batch k-means, port	0.816	0.647	0.431	0.540
BIRCH, R	0.132	0.691	0.377	1.000
BIRCH, port	0.133	0.688	0.366	1.000
Steady-State ARI				
MicroTEDAClus+, R	0.973	0.783	0.599	0.171
MicroTEDAClus+, port	0.956	0.777	0.587	0.165
mini-batch k-means, R	0.832	0.832	0.376	0.571
mini-batch k-means, port	0.828	0.847	0.455	0.572
BIRCH, R	0.138	0.931	0.381	1.000
BIRCH, port	0.127	0.940	0.389	1.000

The interface through which the previous experiment generated RandomRBFGeneratorEvents gives the noise the class value of the first cluster that the generator creates after the initial ones, so the samples of that cluster are labeled as noise. The port labels the noise apart from every cluster, and in the comparison above we apply the labeling of the reference to both sources. Table 5.2 shows the share of samples labeled as noise along the stream under each labeling.

Table 5.2: Share of the samples of the Random RBF stream that carry the noise label, in consecutive blocks of 1000 samples: medians over the streams.

Source	1	2	3	4	5	6	7	8	9	10
R	0.10	0.33	0.33	0.31	0.29	0.24	0.22	0.20	0.12	0.12
port, labels of the R wrapper	0.10	0.33	0.29	0.30	0.26	0.24	0.22	0.14	0.11	0.11
port	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10

5.1.2 Length and Order of the Streams

The generated streams are five times longer than those of the previous experiment, and so are their tuning segments, in answer to Section 4.6. The Chameleon files keep their size. We shuffle the streams without concept drift, which are those of Cassini, of the Chameleon datasets and of the scalability datasets, anew in every run. Each run of a Chameleon dataset thus has a stream of its own, where in the previous experiment all its runs processed the same stream (Section 4.6.2). The concept drift streams keep their order, which carries the drift.

5.2 Benchmark Models

We choose the benchmark models by three criteria:

1. They are established in the literature of stream clustering;
2. Their implementations are maintained libraries that the experiment imports instead of copying;
3. They are fast enough to be tuned without a time limit.

We retain five models: DenStream [2] and DBSTREAM [49], from `River` [10]; BIRCH [50] and Mini-batch k-means [51], from `scikit-learn` [11]; and BICO [12], from the `bico` package. They summarize the stream by density-based microclusters that fade, DenStream and DBSTREAM; by a tree of cluster features, BIRCH; by a coreset, BICO; and by centroids, Mini-batch k-means. This set replaces that of Section 3.2.

Table 5.3 lists the hyperparameters that the search selects for each model. We search a parameter if it is part of the clustering method, and keep the default of its library if it only sets a unit of time or the shape of a data structure. DenStream and DBSTREAM forget at a rate that we search as its half-life, in samples. We search the outlier threshold $\beta\mu$ of DenStream, which its formulation requires to exceed one, as its position between one and μ.

Table 5.3: Hyperparameters that the search selects for each model, and the category that gives each its range.

Model	Hyperparameter	Category
MicroTEDAClus+	`variance_limit`	Relative variance
	`decay`	Horizon
	`noise_distance`	Multiplier
DenStream	`epsilon`	Distance
	`half_life`	Horizon
	`mu`	Minimum points
	`outlier_ratio`	Ratio
	`n_samples_init`	Horizon
DBSTREAM	`clustering_threshold`	Distance
	`half_life`	Horizon
	`cleanup_interval`	Horizon
	`intersection_factor`	Ratio
	`minimum_weight`	Minimum points
BIRCH	`threshold`	Distance
	`n_clusters`	Clusters
Mini-batch k-means	`n_clusters`	Clusters
	`reassignment_ratio`	Ratio
BICO	`n_clusters`	Clusters
	`summary_size`	Summary size

5.3 Hyperparameter Selection

The search remains the Bayesian optimization of Section 3.6.1 on the first 20% of each stream, and we evaluate the selected model prequentially on the rest. Three things change: where the ranges come from, what the objective measures, and what bounds the search.

5.3.1 Ranges by Rule

Each hyperparameter belongs to a category, given in Table 5.3, and each category has one rule that computes its range from the tuning segment, given in Table 5.4. A rule uses only what a deployment of the model also has before the stream continues: the samples of the segment and their labels. Below the median distance from a sample to its nearest neighbor, a structure of that radius holds a single sample, and above the median distance between two samples it holds half of the segment. The range of the distances extends a decade beyond both, so that the search reaches past the scales at which a structure summarizes the data, and the range of the relative variance is that of the distances, squared, so that in the logarithmic scale of the search both give the same room. The longest horizon is the length of the segment, the longest that the segment can tell apart from not forgetting at all. The minimum points start at one, the weight of a single sample, and we search the ratios in the logit scale, which gives every decade towards zero and towards one the same room. The number of clusters reaches twenty times the labeled clusters, since a model may represent a class by several clusters, and at most the samples of a batch, the most clusters that a batch can show. The rules replace ranges set by hand, which Table 4.10 showed to depend on the scale of the data. Table 5.5 gives the ranges that they produce in each dataset.

5.3.2 Objective and Search

The objective is the median of the batch ARI over the second half of the tuning segment. It is the quantity that the Overall ARI reports, computed after the configuration has had the first half of the segment to form its summary, which addresses the slow maturation shown in Table 4.26. It has no term of execution time.

Every model receives 100 trials, without a time limit. The search of a task is seeded by its dataset, its model and its run, and its objective involves no measurement of time, so the selection depends neither on the speed of the machine nor on its load. What bounds the search instead is the size of the summary: no model may keep a summary of more structures than a quarter of the tuning segment, the upper end of the rule of

Table 5.4: Rule of each category of hyperparameter, for a tuning segment of n samples with k labeled clusters. Horizons, summary sizes and numbers of clusters are integers. We measure the median distance between two samples on at most a million pairs of samples drawn at random.

Category	Lower end	Upper end	Scale
Distance	a tenth of the median distance from a sample to its nearest neighbor	ten times the median distance between two samples	logarithmic
Relative variance	square of the lower distance, as a fraction of the total variance	square of the upper distance, as a fraction of the total variance	logarithmic
Horizon	$n/1000$ samples	n samples	logarithmic
Summary size	$2k$	$n/4$	logarithmic
Minimum points	1	$\sqrt{n}$	logarithmic
Ratio	0.0001	0.999	logit
Multiplier	1	10	linear
Clusters	2	$20k$, and at most the 100 samples of a batch	linear

Table 5.5: Search ranges that the rules give to each category of hyperparameter: medians over 94 runs of the two ends of each range.

Dataset	Distance	Relative variance	Horizon	Summary size	Minimum points	Clusters
B1	2.1×10^{-4} to 3.3	5.5×10^{-7} to 138	4 to 4000	4 to 1000	1 to 63	2 to 40
B2	5.3×10^{-4} to 6.5	7.0×10^{-7} to 106	4 to 4000	6 to 1000	1 to 63	2 to 60
RBF	2.5×10^{-4} to 3.2	7.8×10^{-7} to 136	10 to 10000	16 to 2500	1 to 100	2 to 100
CSN	0.002 to 16	2.2×10^{-6} to 142	4 to 4000	6 to 1000	1 to 63	2 to 60
DS3	0.44 to 2802	3.5×10^{-6} to 146	2 to 2000	18 to 500	1 to 45	2 to 100
DS4	0.48 to 2598	4.9×10^{-6} to 143	2 to 1600	16 to 400	1 to 40	2 to 100
S1	4.9×10^{-5} to 30	1.2×10^{-10} to 65	5 to 4400	6 to 1100	1 to 66	2 to 60
S2	0.0026 to 60	1.8×10^{-7} to 106	5 to 4400	6 to 1100	1 to 66	2 to 60
S4	0.021 to 113	4.9×10^{-6} to 167	5 to 4400	6 to 1100	1 to 66	2 to 60
S8	0.067 to 195	2.5×10^{-5} to 216	5 to 4400	6 to 1100	1 to 66	2 to 60
S16	0.15 to 285	6.0×10^{-5} to 230	5 to 4400	6 to 1100	1 to 66	2 to 60
S32	0.26 to 419	9.7×10^{-5} to 252	5 to 4400	6 to 1100	1 to 66	2 to 60
S64	0.43 to 612	1.3×10^{-4} to 270	5 to 4400	6 to 1100	1 to 66	2 to 60

the summary size. At the fine end of the ranges a model can keep one structure per sample, and a search over such configurations is not affordable. A configuration whose summary exceeds the limit fails, as one that raises an error does, and a selected model whose summary exceeds it during the evaluation fails its task. To measure what the change of protocol does, we also tune MicroTEDAClus+ by the previous protocol of Section 3.6.1, with its previous ranges, on the same streams.

5.4 Execution and Measurements

A task is a dataset, a model and a run, and the experiment executes the tasks of 94 runs, 13 datasets and the six models in parallel processes, which give the quality of the models and the size of their summaries. Parallel execution disturbs the execution time, under which the main experiment of the previous chapters timed its tasks. We therefore measure time in a separate pass, by the procedure of Section 4.2.2: we execute the tasks of the first 32 runs again, one at a time, with their selected hyperparameters. A new model of the task first processes the first three batches of the evaluation segment and is discarded, so that what is created on first use is in place, and we then time another new model over the evaluation segment once. A single execution replaces the minimum of three, since the three executions of a task agree closely (Table 4.24).

We measure memory in another pass, over the tasks of the same 32 runs, by tracking every allocation of the process with `memray` [52], which counts the memory allocated by compiled code as well as that allocated by `Python`. We keep two quantities: the memory still allocated when the stream ends, which is what the model holds, and the peak during the evaluation, which adds the working memory of its steps. Each task runs in a new process, so that it inherits nothing that an earlier task left allocated. A model of the task first processes the whole evaluation segment without tracking, so that what is created on first use is in place however late it is created, and we then track new models of the task three times and keep the smallest values. These quantities are not comparable with the memory reported in the previous chapters, which counted only the `Python` objects of a model.

We compare MicroTEDAClus+ with each benchmark model by the test of Section 3.7, whose conventions the tables follow, with the abbreviations of Table 3.2. We leave out of a dataset any model that failed in more than half of its runs, and compare the others over the runs that all of them finished. The comparisons of time and memory rest on 32 runs instead of 94, so they separate fewer models.

5.5 Results

5.5.1 Failures

Table 5.6: Number of runs in which a model failed. Every failure is a summary that exceeded a quarter of the tuning segment.

Model	B1	B2	RBF	CSN	DS3	DS4	S1	S2	S4	S8	S16	S32	S64
Failed runs													
MicroTEDAClus+	0	0	0	0	2	2	0	0	0	0	0	1	0
DenStream	0	0	0	0	0	0	0	0	0	0	0	0	0
DBSTREAM	1	0	0	0	0	0	0	0	0	0	0	0	0
BIRCH	86	1	7	3	1	2	1	2	31	82	79	31	29
Mini-batch k-means	0	0	0	0	0	0	0	0	0	0	0	0	0
BICO	0	0	0	0	0	0	0	0	0	0	0	0	0

Every failure is a summary that exceeded the limit, and nearly all are failures of BIRCH (Table 5.6). It fails in more than half of the runs of StreamBenchmark(1) and of the scalability streams with eight and sixteen dimensions, which leaves it out of them, and in fewer than half of those with four, thirty-two and sixty-four dimensions, which leaves their comparisons with fewer runs. Its tree of cluster features does not fade, unlike the microclusters of DenStream and DBSTREAM (Section 5.2), and in the scalability streams its largest summary grows with the dimension far faster than those of the other models, up to four dimensions (Figure 5.2).

5.5.2 Clustering Quality

MicroTEDAClus+ is better than DenStream and Mini-batch k-means in the Overall ARI of StreamBenchmark(1) and than DBSTREAM and BICO in that of Cassini, and worse than none in the Overall ARI of either (Table 5.7). It is worse than every benchmark model but DenStream in StreamBenchmark(2), on both metrics, worse than all five at the penultimate batch of RandomRBFGeneratorEvents, and worse than three or four of them in DS3. In the Overall ARI of DS4, it is indistinguishable from all of them. In Section 4.8.1, no benchmark model was better than MicroTEDAClus+ in the streams with concept drift. The benchmark models, the streams and the protocol all differ here, and the protocol alone changes its results in StreamBenchmark(2) little (Table 5.9).

In the scalability streams, no benchmark model is better than MicroTEDAClus+ (Table 5.8 and Figure 5.1). From two dimensions on, it is better than most of them on both metrics, and than every one compared in the Overall ARI up to sixteen dimen-

Table 5.7: Clustering quality of MicroTEDAClus+ and the benchmark models: medians over 93 runs, except 87 in RBF, 91 in CSN, 91 in DS3, 90 in DS4. A dash marks a model that failed in more than half of the runs of a dataset.

Model	B1	B2	RBF	CSN	DS3	DS4
Overall ARI						
MicroTEDAClus+	0.895	0.598	0.607	0.925	0.329	0.356
Better/same/worse	2/2/0	0/1/4	0/3/2	2/3/0	0/2/3	0/5/0
DenStream	**0.830**	0.612	0.591	0.646	0.368	0.347
DBSTREAM	0.894	0.792	0.629	**0.636**	0.359	0.361
BIRCH	-	0.695	0.630	0.639	0.396	0.364
Mini-batch k-means	**0.848**	0.640	0.589	0.635	0.391	0.352
BICO	0.882	0.761	0.645	**0.596**	0.405	0.370
Steady-State ARI						
MicroTEDAClus+	0.783	0.565	0.236	0.798	0.309	0.336
Better/same/worse	1/2/1	0/1/4	0/0/5	0/5/0	0/1/4	0/4/1
DenStream	**0.811**	0.580	0.601	0.692	0.368	0.344
DBSTREAM	0.904	0.775	0.575	0.603	0.337	0.352
BIRCH	-	0.759	0.716	0.665	0.402	0.366
Mini-batch k-means	0.913	0.666	0.579	0.620	0.391	0.356
BICO	0.821	0.806	0.629	0.584	0.409	0.368

Table 5.8: Clustering quality of MicroTEDAClus+ and the benchmark models in the scalability streams: medians over 94 runs, except 93 in S1, 92 in S2, 63 in S4, 62 in S32, 65 in S64. A dash marks a model that failed in more than half of the runs of a dataset.

Model	S1	S2	S4	S8	S16	S32	S64
Overall ARI							
MicroTEDAClus+	0.703	0.942	0.997	1.000	1.000	1.000	1.000
Better/same/worse	0/5/0	5/0/0	5/0/0	4/0/0	4/0/0	4/1/0	4/1/0
DenStream	0.785	**0.863**	**0.951**	**0.968**	**0.978**	**0.893**	**0.992**
DBSTREAM	0.780	**0.921**	**0.957**	**0.977**	**0.977**	**0.980**	**1.000**
BIRCH	0.781	**0.929**	**0.971**	-	-	**0.862**	**0.861**
Mini-batch k-means	0.789	**0.858**	**0.895**	**0.958**	**0.983**	**1.000**	**1.000**
BICO	0.800	**0.878**	**0.893**	**0.962**	**0.982**	1.000	1.000
Steady-State ARI							
MicroTEDAClus+	0.721	0.909	0.983	1.000	1.000	1.000	1.000
Better/same/worse	0/5/0	3/2/0	4/1/0	4/0/0	4/0/0	4/1/0	3/2/0
DenStream	0.766	**0.855**	**0.950**	**0.969**	**0.972**	**0.939**	**0.996**
DBSTREAM	0.759	0.899	**0.941**	**0.982**	**0.976**	**0.988**	**1.000**
BIRCH	0.755	0.925	0.967	-	-	**0.902**	**0.874**
Mini-batch k-means	0.777	**0.838**	**0.893**	**0.935**	**0.982**	**1.000**	1.000
BICO	0.770	**0.876**	**0.872**	**0.942**	**0.983**	1.000	1.000

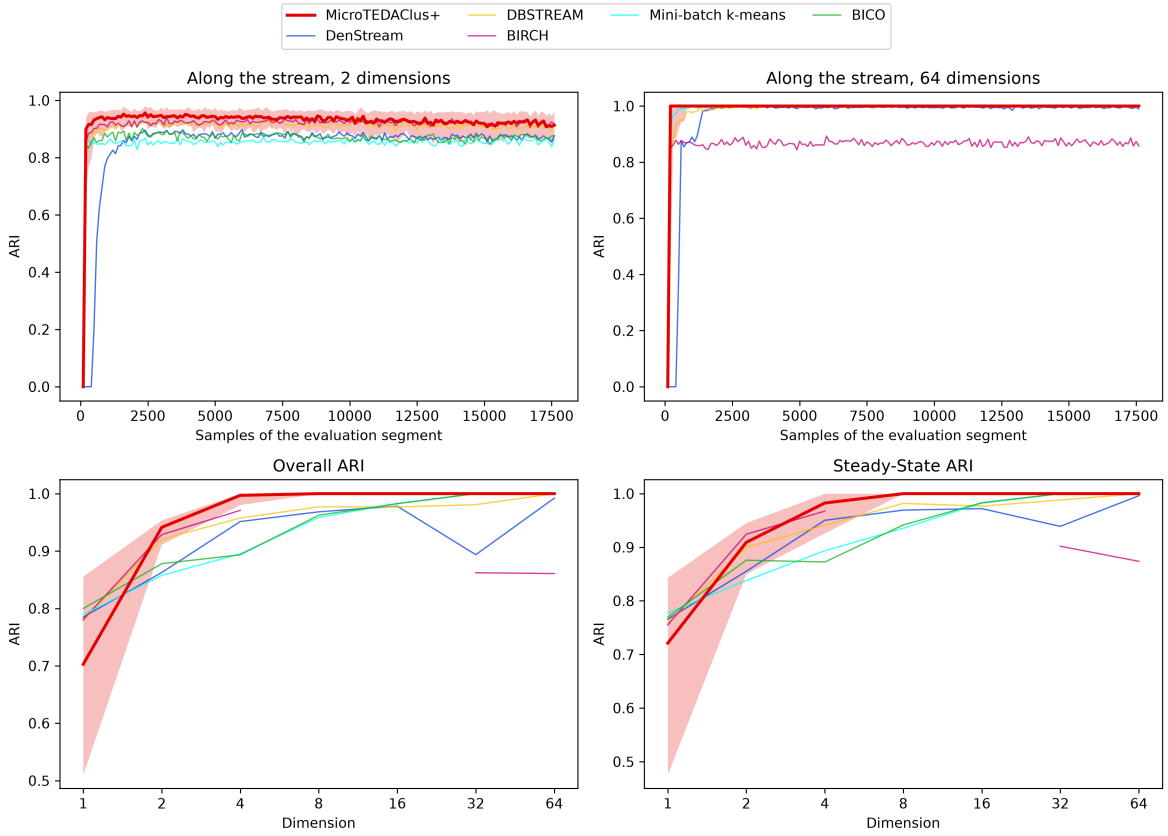


Figure 5.1: ARI of MicroTEDAClus+ and the benchmark models in the scalability streams: along the stream with 2 and 64 dimensions, and the Overall and Steady-State ARI against the dimension. Medians, with the interquartile band of MicroTEDAClus+, over the runs of Table 5.8; BIRCH, left out of two streams, has no point there.

sions. With one dimension, the stream that was already the exception in Section 4.8.1, it is indistinguishable from all of them.

Along the streams of Figure 5.1, DenStream scores zero over the first several hundred samples: it labels every sample as noise until its initialization buffer, whose length the search selects, is full. The objective scores only the second half of the tuning segment, where a late start costs nothing, while the Overall ARI counts the whole stream.

5.5.3 The Change of Protocol

Table 5.9: MicroTEDAClus+ with its hyperparameters selected by the previous protocol and by the protocol by rules, on the same streams: medians over 94 runs, except 92 in DS3, 92 in DS4, 93 in S32.

Selection	B1	B2	RBF	CSN	DS3	DS4	S1	S2	S4	S8	S16	S32	S64
Overall ARI													
Previous protocol	0.867	0.606	0.546	0.575	0.283	0.338	0.716	0.937	1.000	1.000	1.000	1.000	1.000
Protocol by rules	**0.895**	0.598	0.600	0.917	**0.332**	0.356	0.704	0.942	1.000	1.000	1.000	1.000	1.000
Steady-State ARI													
Previous protocol	0.216	0.565	0.274	0.482	0.264	0.325	0.663	0.926	1.000	1.000	1.000	1.000	1.000
Protocol by rules	**0.780**	0.565	0.186	**0.794**	**0.308**	0.337	0.730	0.909	0.996	1.000	1.000	1.000	1.000

Table 5.9 compares MicroTEDAClus+ under the two protocols on the same streams. The protocol by rules raises its Overall ARI in StreamBenchmark(1) and DS3 and lowers it slightly in StreamBenchmark(2) and in the scalability stream with four dimensions, and raises its Steady-State ARI in StreamBenchmark(1), Cassini and DS3 without lowering it in any dataset. The largest differences are at the penultimate batch of StreamBenchmark(1) and Cassini. This agrees with Section 4.6.2, which located the most direct improvement of the model in the selection of its hyperparameters rather than in its algorithm.

5.5.4 Cost

MicroTEDAClus+ is faster than every benchmark model compared in the prediction stage of every dataset (Table 5.10). It has no such advantage in the learning stage, where it is slower than BICO in five datasets and than Mini-batch k-means in all but StreamBenchmark(2), and faster than DBSTREAM in four. Mini-batch k-means, which keeps only its centroids, holds less memory and reaches a lower peak than MicroTEDAClus+ in every dataset. Against the other benchmark models, the memory that MicroTEDAClus+ holds is smaller in some datasets and larger in others, while its peak is the largest of all models in Cassini and the Chameleon datasets, in which it keeps more structures than four or all five of them (Table 5.11).

Table 5.10: Execution time and memory of MicroTEDAClus+ and the benchmark models: medians over 32 runs, except 31 in B1, 29 in RBF, 31 in CSN, 30 in DS3. A dash marks a model that failed in more than half of the runs of a dataset.

Model	B1	B2	RBF	CSN	DS3	DS4
Learning time [ms]						
MicroTEDAClus+	80	60	190	380	100	30
Better/same/worse	2/0/2	3/0/2	2/2/1	0/1/4	0/2/3	2/0/3
DenStream	**160**	50	170	50	30	20
DBSTREAM	**300**	**70**	**1140**	100	55	**80**
BIRCH	-	**190**	**840**	230	90	**80**
Mini-batch k-means	70	**70**	170	70	40	30
BICO	30	10	70	10	10	0
Prediction time [ms]						
MicroTEDAClus+	10	10	30	20	10	0
Better/same/worse	4/0/0	5/0/0	5/0/0	5/0/0	5/0/0	5/0/0
DenStream	**110**	**25**	**150**	**20**	**15**	**10**
DBSTREAM	**120**	**30**	**170**	**30**	**20**	**20**
BIRCH	-	**70**	**190**	**70**	**30**	**30**
Mini-batch k-means	**30**	**30**	**80**	**30**	**10**	**10**
BICO	**670**	**205**	**1160**	**190**	**110**	**90**
Memory held at the end of the stream [KiB]						
MicroTEDAClus+	11	11	11	44	23	11
Better/same/worse	3/0/1	1/1/3	4/0/1	0/2/3	1/0/4	2/1/2
DenStream	**30**	7	**19**	6	8	8
DBSTREAM	**65**	5	**64**	6	8	**29**
BIRCH	-	7	**46**	12	7	8
Mini-batch k-means	7	6	6	6	6	6
BICO	**159**	**38**	**237**	20	**46**	**34**
Peak memory [KiB]						
MicroTEDAClus+	251	122	240	2675	894	136
Better/same/worse	2/0/2	0/2/3	2/2/1	0/0/5	0/0/5	0/0/5
DenStream	**712**	126	**639**	49	70	57
DBSTREAM	104	44	138	45	46	68
BIRCH	-	34	119	40	30	38
Mini-batch k-means	45	24	32	22	26	30
BICO	**327**	120	**411**	101	100	85

Table 5.11: Summary size of MicroTEDAClus+ and the benchmark models: medians over 93 runs, except 87 in RBF, 91 in CSN, 91 in DS3, 90 in DS4. A dash marks a model that failed in more than half of the runs of a dataset.

Model	B1	B2	RBF	CSN	DS3	DS4
Largest summary						
MicroTEDAClus+	65	28	65	423	150	58
Better/same/worse	1/2/1	1/0/4	1/2/2	0/0/5	0/1/4	0/1/4
DenStream	49	4	21	4	6	5
DBSTREAM	62	3	46	7	6	23
BIRCH	-	7	69	6	6	7
Mini-batch k-means	30	3	14	3	5	6
BICO	**510**	**261**	**853**	26	100	100
Steady-state summary						
MicroTEDAClus+	57	4	13	397	124	50
Better/same/worse	1/1/2	2/2/1	3/2/0	0/0/5	0/1/4	0/1/4
DenStream	19	3	10	3	5	5
DBSTREAM	54	3	**36**	7	6	23
BIRCH	-	**7**	**69**	6	6	7
Mini-batch k-means	30	3	14	3	5	6
BICO	**386**	**233**	**765**	24	93	76

In the three static datasets, MicroTEDAClus+ keeps more structures than DenStream, DBSTREAM, BIRCH and Mini-batch k-means (Table 5.11). In the Chameleon datasets, whose streams have the same length in both experiments, it keeps more microclusters than the configurations that the main experiment selected, most of all in DS3 (Table 4.31). The protocol by rules thus selects there the finer configurations that Section 4.6 found to score better, and the Overall ARI of DS3 rises with them (Table 5.9).

In the scalability streams, the prediction time of MicroTEDAClus+ stays nearly constant across the dimensions, below those of every benchmark model, and its learning time stays below that of DBSTREAM at every dimension and below that of BIRCH wherever it is compared (Figure 5.2). The memory it holds grows with the dimension and stays above that of Mini-batch k-means at every dimension.

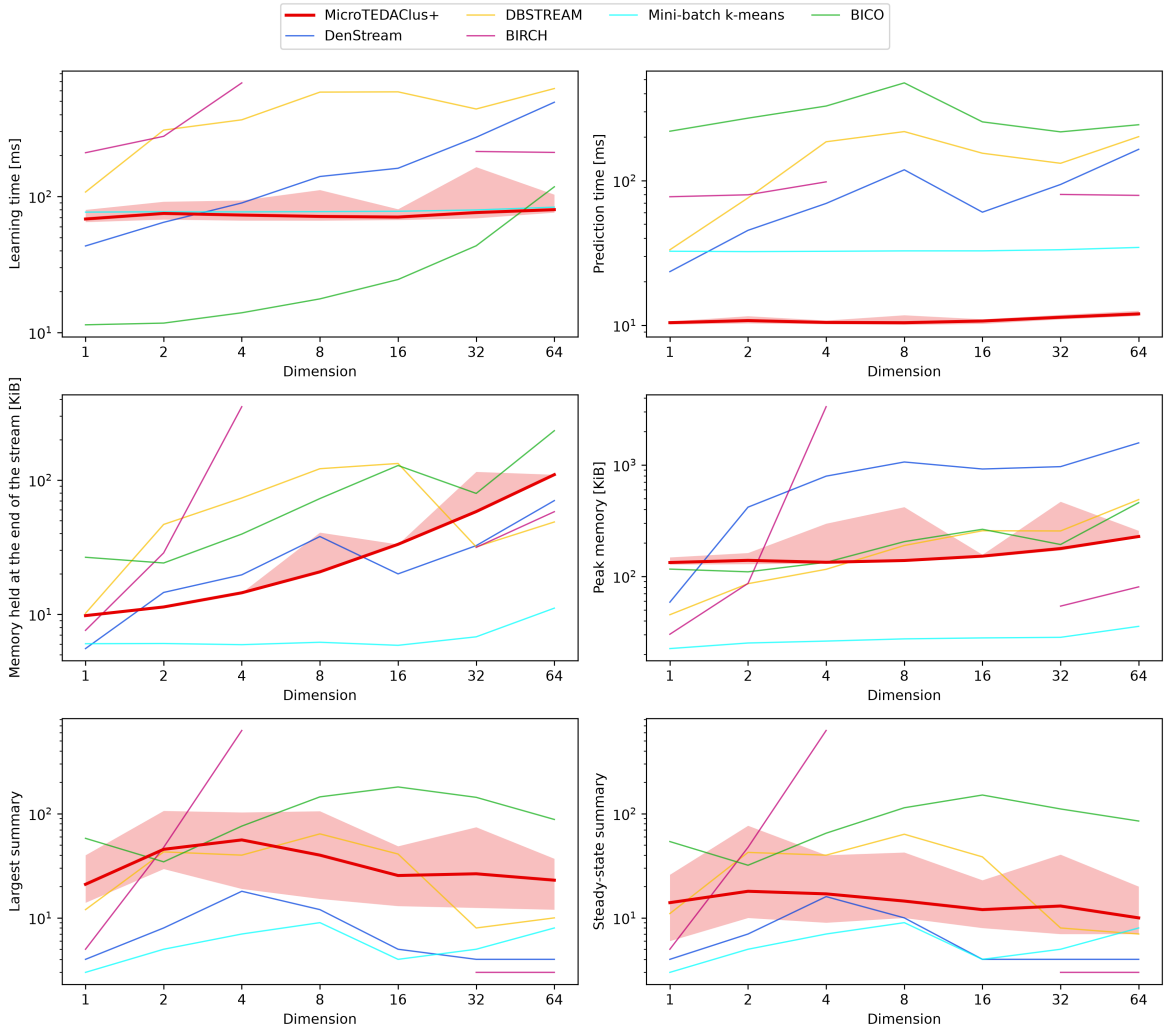


Figure 5.2: Execution time and memory of MicroTEDAClus+ and the benchmark models in the scalability streams, and their summary sizes, against the dimension. Medians over the runs that every model kept in a stream finished, out of 32 for time and memory and 94 for the summary sizes, with the interquartile band of MicroTEDAClus+; BIRCH, left out of two streams, has no point there.

5.6 Summary

The modernized experiment runs in `Python` only, on ported generators validated against those of `R` (Section 5.1.1), with five benchmark models imported from maintained libraries. Its hyperparameter selection takes its ranges from the tuning segment by rule and measures no time, so it depends neither on the speed nor on the load of the machine (Section 5.3). Under it, MicroTEDAClus+ is not worse than any benchmark model in the Overall ARI of StreamBenchmark(1) and Cassini nor in any scalability stream, and better than every one compared in the Overall ARI of those from two to sixteen dimensions, but worse than most of them in StreamBenchmark(2), at the penultimate batch of RandomRBFGeneratorEvents and in DS3. It predicts faster than every benchmark model compared in every dataset, but does not learn faster. The protocol itself improves its results in several datasets, most of all at the penultimate batch of StreamBenchmark(1) and Cassini (Table 5.9).

Chapter 6

Real Data

The datasets of the previous chapters are synthetic: their clusters, drifts and noise are known by construction. In this chapter, we evaluate MicroTEDAClus+ and the benchmark models of Chapter 5 on datasets of real data, recorded from physical processes and systems. Their classes are not drawn from chosen distributions, their drifts come from the conditions of the recording, and their dimension reaches the hundreds. The models, the evaluation and the statistical analysis are those of Chapter 5, and we change the protocol only where these datasets require it.

6.1 Datasets

We choose the datasets by three criteria:

1. They are public and documented, so that their streams can be traced to their sources;
2. Their classes are groups of samples of the same origin, such as a species, a gas or a digit, and not the values of a variable to be predicted;
3. Their features are numeric.

The second criterion excludes datasets of stream classification whose label is a quantity to anticipate, such as the direction of a price. We retain eight datasets, listed in Table 6.1, which also gives the abbreviations of the tables of this chapter.

The three INSECTS datasets record flying insects with an optical sensor, labeled by species and sex [53]. Their authors varied the temperature of the recording, which changes the flight of the insects and therefore the features of each class, and ordered the samples so that the temperature changes abruptly, incrementally, or incrementally and back again. Gas Sensor Array Drift records an array of chemical sensors exposed to

Table 6.1: The real datasets: their number of samples, the features that the experiment uses, the number of classes, and the order of their streams.

Dataset	Abbreviation	Samples	Features	Classes	Order
INSECTS abrupt [53]	INS-A	52,848	33	6	as recorded
INSECTS incremental [53]	INS-I	57,018	33	6	as recorded
INSECTS incremental-reoccurring [53]	INS-R	79,986	33	6	as recorded
Gas Sensor Array Drift [54]	GAS	13,910	128	6	periods in order, each shuffled
KDD Cup 1999 [55]	KDD	494,021	34	23	shuffled
Forest Covertype [56]	COV	581,012	10	7	as recorded
Pendigits [57]	PEN	10,992	16	10	shuffled
Shuttle [58]	SHU	58,000	8	7	shuffled

six gases over three years, in ten recording periods across which the sensors drift [54]. KDD Cup 1999 records network connections, labeled by the attack to which they belong [55]. It is a frequent dataset of stream clustering [1, 2] and a criticized one, for its redundant records among other reasons [59]. Forest Covertype describes forest cells by cartographic variables, labeled by their cover type [56], Pendigits describes handwritten digits by points of their trajectory [57], and Shuttle records the sensors of a space shuttle, labeled by one of seven conditions [58].

6.1.1 Features

Each dataset contributes the features that its documentation describes as measurements. We leave out symbolic features, binary indicators and the time of a record: KDD Cup 1999 keeps its continuous features, Forest Covertype its quantitative ones, and Shuttle every attribute but the first, which is time. No dataset has a missing value.

The features of a real dataset come in units of their own, so we standardize them by the mean and the standard deviation that each feature has in the tuning segment of the stream. A feature that is constant in the tuning segment cannot be scaled by it, and we leave it out. Section 3.3.1 explains that scaling is expected in any application of the model, and that it leaks information when it is computed over the whole dataset. The tuning segment is what a deployment has before the stream continues, so nothing of the evaluation segment enters the transformation. We do not standardize the datasets of the previous chapters.

6.1.2 Order

The INSECTS datasets and Forest Covertype keep the order of their files. The drifts of the INSECTS datasets are in that order, and the order of Forest Covertype follows the location of its cells [53]. Pendigits and Shuttle record no meaningful order, so we shuffle their streams in every run, as we do the static streams of Section 5.1.2. Gas

Sensor Array Drift keeps the order of its ten recording periods, which carries the drift of its sensors, and we shuffle each period in every run, since the order of the samples within a period is not recorded.

Table 6.2: Batches of a single class in the evaluation segment of the real datasets, in the order of the files and in the streams evaluated, and runs whose penultimate batch holds a single class: percentages.

	INS-A	INS-I	INS-R	GAS	KDD	COV	PEN	SHU
Batches of a single class								
Recorded order	6.5	0.0	11.0	15.3	96.5	5.7	0.0	0.0
Streams evaluated	6.5	0.0	11.0	0.0	0.0	5.7	0.0	0.0
Runs								
Penultimate batch of a single class	100	0	100	0	0	100	0	0

We shuffle KDD Cup 1999 as well. In its recorded order, nearly every batch of its evaluation segment holds a single class (Table 6.2). The ARI of a batch of a single class only tells whether the model placed the whole batch in one cluster, which is the flaw that we found in the ordered streams of the original work (Section 3.3). Shuffled, no batch of it holds a single class.

Three of the streams evaluated still hold some batches of a single class, and in all of their runs the penultimate batch is one of them (Table 6.2). The Steady-State ARI reads the penultimate batch (Section 3.5), so we do not report it for these three datasets.

6.2 Protocol

The protocol is that of Chapter 5: its rules for the search ranges, its limit on the summary, its runs and its rule for failed runs. This section describes what these datasets add.

Noise label. The real datasets have no noise class: every sample belongs to a class. Once they have a cluster, the benchmark models assign every sample to one, so on these datasets MicroTEDAClus+ assigns every sample to the macrocluster of its nearest active microcluster, without the noise label of Equation (4.10), and the noise distance is no longer a hyperparameter. Before any microcluster is active, it labels every sample as noise, as every model does before it has a cluster. On the synthetic datasets, whose streams carry noise, the label stays (Section 4.4).

Tuning segment. The tuning segment is the first fifth of the stream, up to 10000 samples, the longest tuning segment of Chapter 5, because a search over the whole first fifth of the longest of these streams is not affordable. The evaluation segment is the rest of the stream, and the limit on the summary remains a quarter of the tuning segment (Section 5.3.2).

Comparison. A dataset that leaves MicroTEDAClus+ out, by the rule for failed runs of Section 5.4, has no comparison.

6.3 Results

6.3.1 Failures

Table 6.3: Number of runs in which a model failed on a real dataset. Every failure is a summary that exceeded a quarter of the tuning segment.

Model	INS-A	INS-I	INS-R	GAS	KDD	COV	PEN	SHU
Failed runs								
MicroTEDAClus+	5	1	0	0	0	51	15	0
DenStream	0	0	0	0	0	0	0	0
DBSTREAM	1	2	2	1	0	0	4	0
BIRCH	10	20	13	39	1	3	5	0
Mini-batch k-means	0	0	0	0	0	0	0	0
BICO	0	0	0	0	0	0	0	0

BIRCH fails in most datasets, most often in Gas Sensor Array Drift, but in fewer than half of the runs of any of them, so it stays in every dataset and leaves their comparisons with fewer runs (Table 6.3). It also failed most often on the synthetic datasets, where its tree of cluster features, which does not fade, grew past the limit (Section 5.5.1). MicroTEDAClus+ fails in 51 of the 94 runs of Forest Covertype, which leaves it out of that dataset. Its summary passes the limit late in the stream, in the median failing run after about two thirds of the evaluation segment, which is some 57 times as long as the tuning segment on which the configuration was selected. It also fails in some runs of Pendigits, the shortest stream and therefore the one with the smallest limit, and in a few runs of INSECTS abrupt and INSECTS incremental. Every failure is a summary that exceeded the limit.

Table 6.4: Clustering quality of MicroTEDAClus+ and the benchmark models on the real datasets: medians over 94 runs, except 78 in INS-A, 71 in INS-I, 79 in INS-R, 54 in GAS, 93 in KDD, 71 in PEN. A dash marks a value left out: the Steady-State ARI of a stream whose penultimate batch holds a single class, a model that failed in more than half of the runs of a dataset, and every model of a dataset that leaves MicroTEDAClus+ out, where no comparison is made.

Model	INS-A	INS-I	INS-R	GAS	KDD	COV	PEN	SHU
Overall ARI								
MicroTEDAClus+	0.150	0.135	0.135	0.181	0.688	-	0.492	0.448
Better/same/worse	1/1/3	2/0/3	0/3/2	2/2/1	1/1/3	-	0/0/5	5/0/0
DenStream	0.166	0.171	0.158	0.181	0.947	-	0.522	**0.363**
DBSTREAM	0.149	**0.094**	0.086	**0.142**	0.932	-	0.546	**0.378**
BIRCH	**0.121**	**0.111**	0.092	**0.151**	**0.192**	-	0.536	**0.318**
Mini-batch k-means	0.179	0.152	0.126	0.205	0.933	-	0.544	**0.433**
BICO	0.186	0.169	0.140	0.161	0.548	-	0.550	**0.365**
Steady-State ARI								
MicroTEDAClus+	-	0.161	-	0.114	0.633	-	0.533	0.413
Better/same/worse	-	1/1/3	-	0/4/1	1/1/3	-	0/5/0	4/1/0
DenStream	-	0.225	-	0.121	0.947	-	0.515	**0.336**
DBSTREAM	-	**0.108**	-	0.106	0.920	-	0.558	**0.350**
BIRCH	-	0.146	-	0.101	**0.081**	-	0.544	**0.000**
Mini-batch k-means	-	0.192	-	0.143	0.919	-	0.536	0.376
BICO	-	0.217	-	0.110	0.500	-	0.563	**0.372**

6.3.2 Clustering Quality

MicroTEDAClus+ is better than every benchmark model in the Overall ARI of Shuttle, and worse than most of them in INSECTS abrupt, INSECTS incremental, KDD Cup 1999 and Pendigits, where it is worse than all five (Table 6.4). In Gas Sensor Array Drift it is better than two and worse than Mini-batch k-means, and in INSECTS incremental-reoccurring it is worse than two and indistinguishable from the other three. At the penultimate batch, where it is reported, it is worse than most benchmark models in INSECTS incremental and KDD Cup 1999, worse than Mini-batch k-means alone in Gas Sensor Array Drift, indistinguishable from all of them in Pendigits, and better than four in Shuttle. Forest Covertype, which leaves it out, has no comparison.

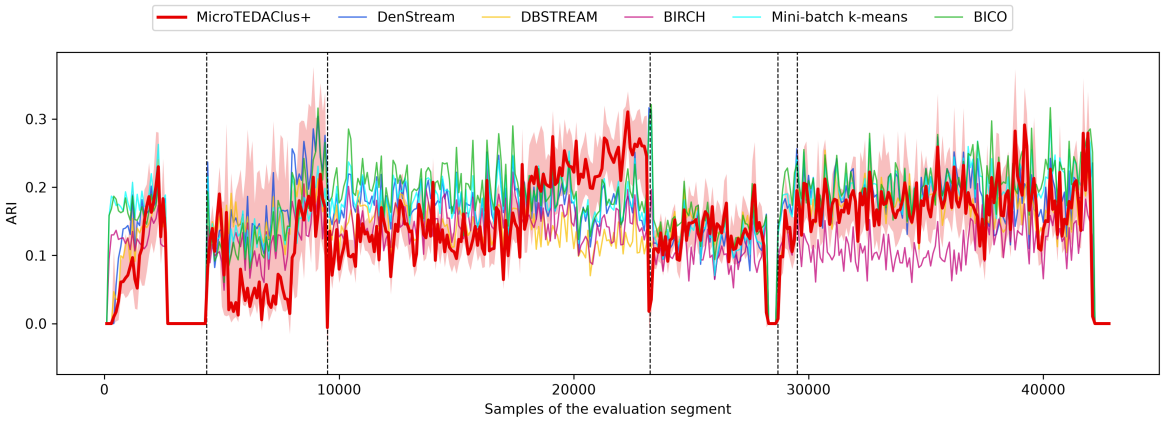


Figure 6.1: Batch ARI of MicroTEDAClus+ and the benchmark models along the evaluation segment of INSECTS abrupt, with the change points of the stream as dashed lines. Medians, with the interquartile band of MicroTEDAClus+, over the 78 runs that every model finished.

Figure 6.1 follows the models along INSECTS abrupt. MicroTEDAClus+ is neither consistently above nor below the benchmark models: it falls below all of them for a stretch after the first change point and rises above all of them for a stretch before the third. In three stretches the median ARI of every model is zero. They are batches of a single class (Table 6.2), whose ARI is one for a model that placed the whole batch in one cluster and zero otherwise (Section 6.1.2).

6.3.3 Cost

MicroTEDAClus+ is faster than every benchmark model in the prediction stage of every dataset compared, or indistinguishable from one of them (Table 6.5). In the learning stage it is slower than Mini-batch k-means and BICO in five of the seven datasets compared and than one of the two in the others, and faster than DBSTREAM in four.

Table 6.5: Execution time and memory of MicroTEDAClus+ and the benchmark models on the real datasets: medians over 32 runs, except 28 in INS-A, 25 in INS-I, 26 in INS-R, 22 in GAS, 31 in KDD, 25 in PEN. A dash marks a model left out of a dataset, and every model of a dataset that leaves MicroTEDAClus+ out.

Model	INS-A	INS-I	INS-R	GAS	KDD	COV	PEN	SHU
Learning time [ms]								
MicroTEDAClus+	5005	460	8780	225	2020	-	170	620
Better/same/worse	0/1/4	3/0/2	0/2/3	2/2/1	3/1/1	-	1/1/3	0/2/3
DenStream	770	**890**	1200	**860**	**7240**	-	90	220
DBSTREAM	3630	**8770**	6515	**1210**	**38730**	-	**520**	800
BIRCH	1325	**1680**	6715	410	**12180**	-	190	635
Mini-batch k-means	170	190	280	50	1910	-	30	185
BICO	130	150	210	270	1360	-	10	45
Prediction time [ms]								
MicroTEDAClus+	65	30	130	20	270	-	10	40
Better/same/worse	4/1/0	5/0/0	4/1/0	4/1/0	5/0/0	-	4/1/0	5/0/0
DenStream	**290**	**370**	**430**	**310**	**3070**	-	**70**	**70**
DBSTREAM	**665**	**310**	**1195**	**235**	**3270**	-	**110**	**120**
BIRCH	**230**	**260**	**545**	**75**	**2320**	-	**40**	**190**
Mini-batch k-means	70	**80**	120	20	**800**	-	10	**80**
BICO	**815**	**890**	**1040**	**240**	**7160**	-	**170**	**490**
Memory held at the end of the stream [KiB]								
MicroTEDAClus+	396	124	374	461	57	-	90	44
Better/same/worse	1/2/2	3/0/2	1/1/3	2/1/2	3/1/1	-	3/0/2	1/1/3
DenStream	65	88	49	262	62	-	57	7
DBSTREAM	256	**722**	224	378	**295**	-	**219**	20
BIRCH	**600**	**829**	**1652**	**2058**	**475**	-	**146**	58
Mini-batch k-means	8	8	8	26	7	-	9	6
BICO	612	**592**	224	**946**	**412**	-	**155**	**94**
Peak memory [KiB]								
MicroTEDAClus+	12496	582	14556	928	231	-	1471	1562
Better/same/worse	0/0/5	1/3/1	0/1/4	2/2/1	4/0/1	-	0/0/5	0/0/5
DenStream	528	1450	834	**4858**	**381**	-	366	134
DBSTREAM	526	1016	472	1322	**534**	-	344	92
BIRCH	1774	**2949**	10368	**3590**	**992**	-	305	132
Mini-batch k-means	34	33	30	85	22	-	46	26
BICO	1208	1129	475	1714	**612**	-	301	226

The memory it holds at the end of the stream is smaller than that of some benchmark models and larger than that of others, and always larger than that of Mini-batch k-means. Its peak during the evaluation is the largest of all models in INSECTS abrupt, Pendigits and Shuttle, and of all but BIRCH in INSECTS incremental-reoccurring.

6.4 Summary

The real datasets add drifts that come from the conditions of the recording, classes that are not drawn from chosen distributions, and up to 128 dimensions. Evaluating on them required standardizing the features by the tuning segment, an order for each stream, MicroTEDAClus+ without its noise label, and a limit on the tuning segment. On them, MicroTEDAClus+ is better than every benchmark model in Shuttle, worse than most of them in four datasets, and left out of Forest Covertype, where its summary exceeds the limit in most runs (Tables 6.3 and 6.4). It predicts at least as fast as every benchmark model, learns more slowly than the fastest of them, and reaches the largest peak of memory in three datasets (Table 6.5).

Chapter 7

Discussion

In the previous chapters we report each experiment on its own terms; in this chapter we read them together: what they establish about the claims of the original work, how the model behaves on synthetic and on real data, how much of each comparison depends on the protocol that selects the hyperparameters, and where the evidence stops.

7.1 The Claims of the Original Work

The original work reports that MicroTEDAClus performs on par with established stream clustering algorithms and adapts to concept drift better than them. Under the reassessed methodology, neither claim holds. Against the established benchmark models of the original work, DenStream, CluStream and StreamKM, MicroTEDAClus is worse than CluStream and StreamKM in the Overall ARI of the three static datasets, and at the penultimate batch it is better than none of the three in any dataset and worse than StreamKM in all six (Section 3.8). In two of the three streams with concept drift, its median Steady-State ARI is zero. It learns more slowly than every benchmark model but AutoCloud, and in the scalability streams of high dimension its microclusters collapse along the stream, which we trace to the dimensionality penalty of the formulation (Section 2.2.1).

The published results were obtained under the conditions that we found in the original methodology (Chapter 3). The datasets were cleaned of noise, scaled with statistics of the whole stream and ordered by label, which made the clustering of most batches trivial (Section 3.3); the evaluation learned each batch before predicting it (Section 3.4); the benchmark models ran through layers of interface that inflated their times (Figure 3.1); and the implementation that produced the results departs from the formulation of the paper (Section 3.1). The published comparisons therefore measured these conditions together with the models. The reassessed methodology removes them

for every model alike, and under it the original model does not confirm the results published for it.

7.2 Synthetic and Real Data

On the synthetic datasets, MicroTEDAClus+ corrected the weaknesses of the original model. It is better than MicroTEDAClus in the Overall ARI of five of the six datasets and in the Steady-State ARI of all six, faster in both stages and smaller in memory; no benchmark model is better than it in the streams with concept drift, and it is the fastest model of the main experiment (Section 4.8.5). Its weakness is the static shapes, where CluStream and StreamKM remain better. The modernized experiment, with other benchmark models, longer streams and the selection by rules, moves its position. It is not worse than any benchmark model in the Overall ARI of StreamBenchmark(1) and Cassini nor in any scalability stream, but most of them are better than it in StreamBenchmark(2), at the penultimate batch of RandomRBFGeneratorEvents and in DS3, and it predicts faster than all of them in every dataset without learning faster (Section 5.6). On the real datasets, with the same rules for the search ranges and without its noise label, it is better than every benchmark model in Shuttle, worse than most of them in four datasets, and left out of Forest Covertype (Tables 6.3 and 6.4). These datasets differ from the synthetic ones in what Chapter 6 names: their drifts come from the conditions of the recording, their classes are not drawn from chosen distributions, and their dimension reaches the hundreds.

Chapter 6 also shows where it falls short. Along INSECTS abrupt, it falls below every benchmark model for a stretch after the first change point, and rises above all of them for a stretch before the third (Figure 6.1). Its summary grows with the stream: it exceeds the limit in most runs of Forest Covertype, the longest stream, and in some runs of Pendigits, whose limit is the smallest, and its peak of memory is the largest of all models in three datasets (Tables 6.3 and 6.5). In the prediction stage it is never slower than a benchmark model, as on the synthetic data (Table 6.5).

The noise label is a choice that depends on the data. It improved the model on the synthetic datasets, whose streams carry noise (Section 4.4), and the real datasets have no noise class, so we run the model without it there (Section 6.2). The model that no benchmark model surpassed in the synthetic streams with concept drift falls behind several of them in the real ones, the INSECTS datasets, where two or three of the five benchmark models are better (Table 6.4).

7.3 The Weight of the Protocol

Every result that we report measures a model as its protocol tunes it, and the protocol weighs on the results as much as some changes to the model do. The probe of Section 4.6 found configurations in the search space of MicroTEDAClus+ far better than those that the search selected. The modernized protocol raised the results of the same model in several datasets without any change to it, most of all at the penultimate batch of StreamBenchmark(1) and Cassini (Table 5.9), and in DS3 it selected finer configurations, of the kind that the probe had found to score better (Section 5.5.4). A comparison between models is thus a comparison between models and their selection, which is why we tune every model of a comparison by the same protocol.

The search is also a source of variation. Two searches of a single algorithm give medians that differ without significance (Table 4.19). Differences of that size are not effects of a model, and the tests over many runs are what separate the effects from this variation.

7.4 Limits of the Evidence

Five limits qualify these results.

- **Streams that repeat.** The Chameleon datasets are fixed files in the main experiment (Section 4.1.3), and the INSECTS datasets and Forest Covertype keep the order of their files (Section 6.1.2). Their runs repeat the search and not the stream, so on them the dispersion over the runs is that of the search, and on the Chameleon datasets the validation of Chapter 4 confirms the outcome of the search more than that of the model.

- **Number of runs.** The time and the memory of the modernized and the real-data experiments rest on 32 runs (Section 5.4). With fewer runs the tests separate fewer models.

- **Models and runs left out.** The limit on the summary leaves MicroTEDAClus+ out of Forest Covertype, which therefore has no comparison, and BIRCH out of three synthetic datasets, and the failures of BIRCH leave several other datasets with fewer runs to compare (Tables 5.6 and 6.3).

- **Cost.** The execution time and the memory measure implementations and not only algorithms, as Section 4.8.2 notes and as the reimplementation of Section 2.4 shows for a single algorithm, and the models of a comparison are implemented in different libraries.

- **Datasets.** The synthetic datasets are streams and shapes of two dimensions and scalability streams of up to sixty-four, and the real datasets are eight, chosen by the criteria of Section 6.1, one of them criticized for its redundant records. Our conclusions about real data extend to data of that kind and not beyond.

Chapter 8

Conclusions

In this work, we set out to reassess MicroTEDAClus, an evolving clustering algorithm for data streams built on the eccentricity and typicality of the TEDA framework, and to propose an improved model from that reassessment. We then modernized the experiment and evaluated the improved model on real data.

8.1 Final Remarks

The formulation of MicroTEDAClus has issues of its own, an order-dependent density, an outlier condition that cannot hold for microclusters with two samples and an unjustified scaling by the dimension (Section 2.2.1), and the implementation that produced the published results departs from it (Section 3.1). The methodology of the original experiments leaks information about the streams, makes most batches trivial to cluster and evaluates each batch after learning it (Chapter 3). Under a methodology without these flaws, the original model is not on par with the benchmark models of the original work, and it does not adapt better than them to concept drift (Section 7.1).

MicroTEDAClus+, which we defined by controlled experiments on separate development and validation runs, differs from MicroTEDAClus in the five points of Table 4.27. On the synthetic datasets it improves on the original model in clustering quality, execution time and memory, no benchmark model is better than it in the datasets with concept drift, and it is the fastest model of the main experiment (Section 4.8). The modernized experiment, which runs in a single environment with benchmark models from maintained libraries, search ranges by rule and the cost measured with each task alone, changes that position. MicroTEDAClus+ is not worse than any benchmark model in the Overall ARI of StreamBenchmark(1) and Cassini nor in any scalability stream, but most of them are better than it in StreamBenchmark(2), at the penultimate batch of RandomRBFGeneratorEvents and in DS3, and it no longer

learns fastest. The protocol itself raises its results in several datasets (Chapter 5).

On real datasets, with the same rules for the search ranges and without its noise label, since these datasets have no noise class, MicroTEDAClus+ is better than every benchmark model in Shuttle and worse than most of them in four datasets. Its summary exceeds the limit in most runs of Forest Covertype, which leaves it out of that dataset (Tables 6.3 and 6.4). It predicts at least as fast as every benchmark model, learns more slowly than the fastest of them, and reaches the largest peak of memory in three datasets (Table 6.5).

We met the objectives of Section 1.2. Our reassessment identified the issues of the formulation, of the implementation and of the methodology, and we replaced the latter by a methodology that evaluates every model alike. The improved model overcomes most weaknesses of the original one on synthetic data, and keeps part of its standing in the modernized experiment. On real data, the evaluation places it ahead of the benchmark models in one dataset and behind most of them in four, within the limits stated in Section 7.4.

8.2 Future Work

The results point to the following directions.

- **Hyperparameter selection.** The search selects configurations worse than those its space holds (Section 4.6), and its outcome varies with its seed (Section 7.3). An objective that observes a longer segment of the stream, more trials, or repeated searches would reduce these effects, applied to every model of a comparison alike.

- **One model for both kinds of data.** The noise label helps on data with noise, and on data whose classes cover every sample we run the model without it (Section 7.2). Searching whether to label noise, as one more hyperparameter, would let a single model serve both.

- **The real streams with concept drift.** They are where the model stays behind several benchmark models, and along INSECTS abrupt it falls below all of them after the first change point (Table 6.4 and Figure 6.1); they are the main case to understand.

- **Size of the summary.** The summary of the model exceeds the limit in most runs of Forest Covertype and in some of Pendigits, and its peak of memory is the largest of the models compared in three real datasets (Tables 6.3 and 6.5). A

bound on the number of microclusters that the model enforces itself would make its cost predictable.

- **An improvement effort on real data.** The improvement of Chapter 4 rests on synthetic data only. An effort of the same kind on real data, with development runs of its own, could address the weaknesses above.

- **Explore its strengths.** It predicts faster than every benchmark model in every real dataset compared, or as fast as one of them (Table 6.5), which suits applications in which labels must be returned quickly.

References

[1] AGGARWAL, C. C. et al. A framework for clustering evolving data streams. In: VLDB ENDOWMENT. *Proceedings of the 29th International Conference on Very Large Data Bases - Volume 29*. [S.l.], 2003. p. 81–92.

[2] CAO, F. et al. Density-based clustering over an evolving data stream with noise. In: SIAM. *Proceedings of the 2006 SIAM International Conference on Data Mining*. [S.l.], 2006. p. 328–339.

[3] WANG, X. et al. Data stream clustering: An in-depth empirical study. *Proc. ACM Manag. Data*, Association for Computing Machinery, New York, NY, USA, v. 1, n. 2, jun. 2023. Disponível em: <https://doi.org/10.1145/3589307>.

[4] MAIA, J. et al. Evolving clustering algorithm based on mixture of typicalities for stream data mining. *Future Generation Computer Systems*, v. 106, p. 672–684, 2020. ISSN 0167-739X. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0167739X19312786>.

[5] ANGELOV, P. Outside the box: an alternative data analytics framework. *Journal of Automation, Mobile Robotics and Intelligent Systems*, p. 29–35, 2014.

[6] GALINDO, J. C. F. *Mineração de Dados Aplicada a Problemas de Last-Mile na Logística*. Tese (Doutorado) — Escola de Engenharia, Universidade Federal de Minas Gerais, Belo Horizonte, MG, dez. 2021.

[7] CARDOSO, M. B. dos R. *Implementação do algoritmo MicroTEDAclus para classificação de dados*. Monografia (Graduação em Engenharia de Controle e Automação) — Escola de Engenharia, Universidade Federal de Minas Gerais, Belo Horizonte, MG, jul. 2022.

[8] KAUFMAN, S. et al. Leakage in data mining: Formulation, detection, and avoidance. *ACM Trans. Knowl. Discov. Data*, Association for Computing Machinery, New York, NY, USA, v. 6, n. 4, dez. 2012. ISSN 1556-4681. Disponível em: <https://doi.org/10.1145/2382577.2382579>.

[9] KAPOOR, S.; NARAYANAN, A. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, Elsevier, v. 4, n. 9, p. 100804, 2023. ISSN 2666-3899. Disponível em: <https://doi.org/10.1016/j.patter.2023.100804>.

[10] MONTIEL, J. et al. River: machine learning for streaming data in python. *Journal of Machine Learning Research*, v. 22, n. 110, p. 1–8, 2021. Disponível em: <https://jmlr.org/papers/v22/20-1380.html>.

[11] PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011. Disponível em: <https://jmlr.org/papers/v12/pedregosa11a.html>.

[12] FICHTENBERGER, H. et al. BICO: BIRCH meets coresets for k-means clustering. In: *Algorithms – ESA 2013*. [S.l.]: Springer, 2013. (Lecture Notes in Computer Science, v. 8125), p. 481–492.

[13] MAHALANOBIS, P. C. Reprint of: Mahalanobis, p.c. (1936) "on the generalised distance in statistics.". *Sankhya A*, v. 80, n. 1, p. 1–7, Dec 2018. ISSN 0976-8378. Disponível em: <https://doi.org/10.1007/s13171-019-00164-5>.

[14] ESTER, M. et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In: AAAI PRESS. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*. [S.l.], 1996. p. 226–231.

[15] ANGELOV, P. Anomaly detection based on eccentricity analysis. In: *2014 IEEE Symposium on Evolving and Autonomous Learning Systems (EALS)*. [S.l.: s.n.], 2014. p. 1–8.

[16] EDUARDO. *evolving_clustering*. 2026. https://github.com/eduardo-ufmg/evolving_clustering. Fork of Carlos Alberto Severiano's repository available at https://github.com/cseveriano/evolving_clustering. Accessed at commit 50dd2b4.

[17] ACKERMANN, M. R. et al. Streamkm++: A clustering algorithm for data streams. *ACM Journal of Experimental Algorithmics*, v. 17, p. 2.1–2.30, 2012.

[18] HYDE, R.; ANGELOV, P.; MACKENZIE, A. R. Fully online clustering of evolving data streams into arbitrarily shaped clusters. *Information Sciences*, Elsevier, v. 382–383, p. 96–114, 2017.

[19] AKINOSHO, T.; TABANE, E.; WANG, Z. A comparative analysis of data stream clustering algorithms. *International Journal of Computing*, p. 439–446, 12 2023.

[20] REZAEI, Z.; SAJEDI, H.; HASHEMI, M. Data stream clustering with concept drift using fractal dimension. *Knowledge and Information Systems*, v. 67, n. 10, p. 8715–8749, Oct 2025. ISSN 0219-3116. Disponível em: <https://doi.org/10.1007/s10115-025-02484-5>.

[21] MUTAMBIK, I. An entropy-based clustering algorithm for real-time high-dimensional iot data streams. *Sensors*, v. 24, n. 22, 2024. ISSN 1424-8220. Disponível em: <https://www.mdpi.com/1424-8220/24/22/7412>.

[22] JAHN, P. et al. Going offline: An evaluation of the offline phase in stream clustering. In: RIBEIRO, R. P. et al. (Ed.). *Machine Learning and Knowledge Discovery in Databases. Research Track*. Cham: Springer Nature Switzerland, 2026. p. 308–326. ISBN 978-3-032-06109-6.

[23] HAHSLER, M.; FORREST, J. *streamMOA: Interface for MOA Stream Clustering Algorithms*. [S.l.], 2024. R package version 1.3-1. Disponível em: <https://CRAN.R-project.org/package=streamMOA>.

[24] BEZERRA, C. G. et al. A new evolving clustering algorithm for online data streams. In: *2016 IEEE Conference on Evolving and Adaptive Intelligent Systems (EAIS)*. [S.l.: s.n.], 2016. p. 162–168.

[25] BEZERRA, C. G. et al. An evolving approach to data streams clustering based on typicality and eccentricity data analytics. *Information Sciences*, v. 518, p. 13–28, 2020. ISSN 0020-0255. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0020025519311363>.

[26] ZHANG, X.; SHEN, X.; OUYANG, T. Extension of dbscan in online clustering: An approach based on three-layer granular models. *Applied Sciences*, v. 12, n. 19, 2022. ISSN 2076-3417. Disponível em: <https://www.mdpi.com/2076-3417/12/19/9402>.

[27] NGUYEN-LE, H.-H. et al. Beyond binary classification: A semi-supervised approach to generalized ai-generated image detection. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. [S.l.: s.n.], 2026. v. 40, n. 42, p. 35733–35741.

[28] ZUBAROğLU, A.; ATALAY, V. Online embedding and clustering of evolving data streams. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, v. 16, n. 1, p. 29–44, 2023. First published online: 6 July 2022.

[29] KARYPIS, G.; HAN, E.-H.; KUMAR, V. Chameleon: Hierarchical clustering using dynamic modeling. *Computer*, IEEE, v. 32, n. 8, p. 68–75, 1999.

[30] DAWID, A. P. Present position and potential developments: Some personal views: Statistical theory: The prequential approach. *Journal of the Royal Statistical Society. Series A (General)*, Wiley, v. 147, n. 2, p. 278–292, 1984.

[31] ARIAS-GARCIA, J. et al. Improved design for hardware implementation of graph-based large margin classifiers for embedded edge computing. *IEEE Transactions on Neural Networks and Learning Systems*, v. 35, n. 1, p. 1320–1329, 2024.

[32] HALKIDI, M.; BATISTAKIS, Y.; VAZIRGIANNIS, M. On clustering validation techniques. *Journal of Intelligent Information Systems*, Kluwer Academic Publishers, v. 17, n. 2–3, p. 107–145, 2001. ISSN 0925-9902.

[33] STONE, M. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B*, v. 36, n. 2, p. 111–133, 1974.

[34] KOHAVI, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*. [S.l.: s.n.], 1995. v. 2, p. 1137–1143.

[35] ALIFERIS, C.; SIMON, G. Overfitting, underfitting and general model overconfidence and under-performance pitfalls and best practices in machine learning and ai. In: _____. *Artificial Intelligence and Machine Learning in Health Care and Medical Sciences: Best Practices and Pitfalls*. [S.l.]: Springer International Publishing, 2024. p. 477–524. ISBN 978-3-031-39355-6.

[36] BAYAT, R. et al. The pitfalls of memorization: When memorization hurts generalization. In: YUE, Y. et al. (Ed.). *International Conference on Learning Representations*. [S.l.: s.n.], 2025. v. 2025, p. 86017–86043.

[37] HUBERT, L.; ARABIE, P. Comparing partitions. *Journal of Classification*, v. 2, n. 1, p. 193–218, 1985.

[38] OZAKI, Y.; WATANABE, S.; YANASE, T. OptunaHub: A platform for black-box optimization. *arXiv preprint arXiv:2510.02798*, 2025.

[39] SATOPAA, V. et al. Finding a "kneedle" in a haystack: Detecting knee points in system behavior. In: *2011 31st International Conference on Distributed Computing Systems Workshops*. [S.l.: s.n.], 2011. p. 166–171.

[40] BARLOW, R. E. et al. *Statistical Inference Under Order Restrictions: The Theory and Application of Isotonic Regression*. [S.l.]: Wiley, 1972.

[41] ROBERT, C. P.; CASELLA, G. *Monte Carlo Statistical Methods*. 2. ed. New York, NY: Springer, 2004. (Springer Texts in Statistics).

[42] DEMŠAR, J. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine learning research*, v. 7, n. Jan, p. 1–30, 2006.

[43] WILCOXON, F. Individual comparisons by ranking methods. *Biometrics bulletin*, JSTOR, v. 1, n. 6, p. 80–83, 1945.

[44] HOLM, S. A simple sequentially rejective multiple test procedure. *Scandinavian journal of statistics*, JSTOR, v. 6, n. 2, p. 65–70, 1979.

[45] CLIFF, N. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological bulletin*, American Psychological Association, v. 114, n. 3, p. 494–509, 1993.

[46] ROMANO, J. et al. Appropriate statistics for exploring effect size and power in research. In: *Annual meeting of the Florida Educational Research Association*. [S.l.: s.n.], 2006.

[47] ROSENBERG, A.; HIRSCHBERG, J. V-measure: A conditional entropy-based external cluster evaluation measure. In: *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*. Prague, Czech Republic: Association for Computational Linguistics, 2007. p. 410–420.

[48] BIFET, A. et al. MOA: Massive online analysis. *Journal of Machine Learning Research*, v. 11, p. 1601–1604, 2010. Disponível em: <https://jmlr.org/papers/v11/bifet10a.html>.

[49] HAHSLER, M.; BOLAÑOS, M. Clustering data streams based on shared density between micro-clusters. *IEEE Transactions on Knowledge and Data Engineering*, v. 28, n. 6, p. 1449–1461, 2016.

[50] ZHANG, T.; RAMAKRISHNAN, R.; LIVNY, M. BIRCH: An efficient data clustering method for very large databases. In: *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*. [S.l.]: ACM, 1996. p. 103–114.

[51] SCULLEY, D. Web-scale k-means clustering. In: *Proceedings of the 19th International Conference on World Wide Web*. [S.l.]: ACM, 2010. p. 1177–1178.

[52] Bloomberg. *Memray: a memory profiler for Python*. 2026. https://github.com/bloomberg/memray. Version 1.20.0. Accessed: 2026-10-14.

[53] SOUZA, V. M. A. et al. Challenges in benchmarking stream learning algorithms with real-world data. *Data Mining and Knowledge Discovery*, v. 34, n. 6, p. 1805–1858, 2020.

[54] VERGARA, A. et al. Chemical gas sensor drift compensation using classifier ensembles. *Sensors and Actuators B: Chemical*, v. 166, p. 320–329, 2012.

[55] STOLFO, S. et al. *KDD Cup 1999 Data*. 1999. UCI Machine Learning Repository.

[56] BLACKARD, J. A.; DEAN, D. J. Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. *Computers and Electronics in Agriculture*, v. 24, n. 3, p. 131–151, 1999.

[57] ALPAYDIN, E.; ALIMOGLU, F. *Pen-Based Recognition of Handwritten Digits*. 1996. UCI Machine Learning Repository.

[58] STATLOG (Shuttle). UCI Machine Learning Repository.

[59] TAVALLAEE, M. et al. A detailed analysis of the KDD CUP 99 data set. In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*. [S.l.]: IEEE, 2009. p. 1–6.

Appendix A

Complementary Tables of the Model Improvement Experiments

This appendix collects the cost metrics that Chapter 4 cites but that do not take part in the decision rule of Section 4.1.4: the microcluster count and the memory footprint. The tables follow the conventions of that chapter.

A.1 Implementation Optimization

Table A.1: Implementation optimization: steady-state microcluster count and memory footprint, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Steady-state microcluster count								
original	8	20	92	36	118	129	6	6
vectorized	14	28	104	36	115	124	6	9
Steady-state memory footprint [KiB]								
original	69	97	450	119	312	284	19	22
vectorized	23	23	**41**	**24**	**57**	**53**	72	122

A.2 Covariance of the Microclusters

Table A.2: Covariance of the microclusters: steady-state microcluster count and memory footprint, medians over the development runs.

Model	B1	B2	RBF	CSN	DS3	DS4	S32	S64
Steady-state microcluster count								
MicroTEDAClus+	366	23	380	36	26	31	12	11
covariance, directional link	305	24	158	37	27	33	12	14
covariance, Bhattacharyya link	343	22	282	32	24	33	12	14
Steady-state memory footprint [KiB]								
MicroTEDAClus+	84	23	90	24	24	24	71	122
covariance, directional link	96	37	84	38	37	38	3296	13019
covariance, Bhattacharyya link	122	37	106	38	37	38	3296	13019

A.3 Validation

Table A.3: Microcluster count and memory footprint of the accepted models: median and quartiles over the validation runs. Each model is compared with the model to its left.

Dataset	*vectorized*	*corrected*	*improved*	MicroTEDAClus+
Maximum microcluster count				
B1	56 (18, 161)	57 (19, 158)	212 (60, 424)	241 (67, 528)
B2	44 (15, 55)	44 (23, 59)	31 (21, 48)	30 (21, 44)
RBF	226 (106, 620)	178 (86, 493)	206 (74, 368)	210 (74, 378)
CSN	44 (38, 56)	56 (41, 100)	40 (31, 103)	41 (30, 103)
DS3	125 (120, 128)	126 (120, 130)	**40 (38, 41)**	40 (38, 41)
DS4	120 (112, 133)	123 (115, 131)	**40 (38, 42)**	40 (38, 42)
S32	34 (20, 70)	30 (19, 120)	**13 (10, 17)**	26 (13, 56)
S64	27 (23, 40)	56 (29, 112)	**11 (9, 14)**	18 (10, 26)
Steady-state microcluster count				
B1	18 (5, 106)	20 (5, 87)	210 (54, 414)	231 (63, 515)
B2	28 (4, 50)	28 (8, 50)	26 (13, 42)	25 (13, 38)
RBF	108 (26, 512)	106 (14, 430)	188 (60, 353)	188 (60, 374)
CSN	34 (23, 50)	46 (30, 95)	34 (24, 90)	35 (24, 90)
DS3	114 (112, 117)	114 (113, 116)	**26 (22, 27)**	26 (22, 27)
DS4	120 (112, 133)	123 (114, 131)	**30 (29, 34)**	30 (29, 34)
S32	7 (5, 25)	8 (4, 113)	12 (9, 15)	11 (8, 18)
S64	5 (4, 8)	18 (6, 69)	**9 (7, 12)**	8 (6, 12)
Maximum memory footprint [KiB]				
B1	25 (23, 45)	25 (23, 46)	56 (24, 93)	54 (24, 94)
B2	24 (23, 25)	24 (23, 25)	24 (23, 25)	24 (23, 24)
RBF	55 (42, 122)	52 (29, 102)	54 (27, 94)	58 (27, 95)
CSN	24 (24, 25)	25 (24, 30)	25 (24, 38)	25 (24, 39)
DS3	56 (55, 59)	56 (55, 58)	**25 (24, 25)**	25 (24, 25)
DS4	53 (49, 58)	54 (50, 57)	**25 (24, 25)**	24 (24, 25)
S32	77 (75, 89)	76 (74, 137)	**75 (73, 76)**	**72 (71, 73)**
S64	134 (131, 137)	146 (133, 244)	**126 (124, 128)**	**123 (122, 123)**
Steady-state memory footprint [KiB]				
B1	23 (23, 40)	24 (23, 40)	53 (24, 91)	54 (24, 93)
B2	23 (23, 24)	23 (23, 24)	23 (23, 24)	23 (23, 24)
RBF	48 (38, 109)	44 (28, 79)	54 (27, 84)	56 (27, 90)
CSN	24 (23, 25)	24 (24, 28)	24 (24, 37)	24 (24, 38)
DS3	56 (55, 59)	56 (55, 58)	**24 (23, 24)**	24 (23, 24)
DS4	52 (48, 57)	53 (49, 56)	**24 (24, 24)**	24 (24, 24)
S32	72 (71, 76)	72 (71, 135)	75 (73, 76)	**71 (71, 71)**
S64	122 (122, 123)	125 (122, 237)	126 (124, 128)	**122 (122, 122)**